

Table of Contents

1. Einführung und Ziele	3
1.1. Aufgabenstellung	3
1.2. Qualitätsziele	3
1.3. Stakeholder	5
2. Randbedingungen	6
2.1. Technische Randbedingungen	6
2.2. Organisatorische Randbedingungen	6
2.3. Konventionen und Sicherheitsvorgaben	7
3. Kontextabgrenzung	8
3.1. Fachlicher Kontext	8
3.2. Technischer Kontext	9
3.3. Mapping fachliche auf technische Schnittstellen	10
4. Lösungsstrategie	11
4.1. Erreichung der wichtigsten Qualitätsziele	11
4.2. Fundamentale Architekturentscheidungen (Top-Level)	13
4.3. Technologie-Entscheidungen (Tech Stack)	13
4.4. Integrations- und Schnittstellenstrategie	13
4.5. Organisatorisches & DevOps	14
5. Bausteinsicht	15
5.1. Whitebox Gesamtsystem (Level 1)	15
5.2. Whitebox Interface Layer (Level 2)	16
5.3. Whitebox Business Layer (Level 2)	16
5.4. Whitebox Storage Layer & Zugriffslogik (Level 2)	21
6. Laufzeitsicht	23
6.1. Szenario 1: Daten-Ingestion & Grenzwert-Alarm (Pipeline Flow)	23
6.2. Szenario 2: Manueller Upload & Admin-Freigabeprozess	24
6.3. Szenario 3: Initialisierung des Digitalen Zwillings (3D View)	25
6.4. Szenario 4: Räumliches & Fachliches Filtern	26
6.5. Szenario 5: Visualisierung langer Zeitreihen (FDW & Downsampling)	27
6.6. Szenario 6: Authentifizierung, Security Context & Row-Level Security	28
6.7. Szenario 7: On-Demand Datenabruf aus Umsystemen (Fulcrum Integration)	30
6.8. Szenario 8: TimeScaleDB Daten-Lebenszyklus (Downsampling & Kompression)	32
6.9. Datenkonsistenz & Unabhängiges Routing (Kafka Consumer Groups)	33
7. Verteilungssicht	36

7.1. Infrastruktur Ebene 1: AWS Cloud Ressourcen.	36
8. Querschnittliche Konzepte.	39
8.1. Architekturmuster: Domain-Driven Design & CQRS-Lite (Read/Write Flows).	39
8.2. Core-API Datenzugriff & Sicherheit.	40
8.3. Core-API Optimistic Locking und Versionierung/Auditierung.	42
8.4. Core-API Validierung (Validation & Error Handling).	46
8.5. Architektur- und Skalierungskonzept: Data Pipeline.	48
8.6. Integration neuer externer Datenquellen.	53
8.7. 5.4. Ingress Listener implementieren (Deklarativer EIP-Adapter).	55
8.8. Datenbankschemata.	57
8.9. API-Schnittstellen (Contracts).	62
8.10. Teststrategie.	64
8.11. Entwickler-Workflow & CI/CD Pipeline.	65
9. Architekturentscheidungen.	68
10. Qualitätsanforderungen.	74
10.1. Übersicht der Qualitätsanforderungen.	74
10.2. Qualitätsszenarien.	74
11. Risiken und technische Schulden.	75
12. Glossar.	76

Über arc42

Diese Dokumentation basiert auf dem arc42-Template und wurde davon abgeleitet (© Dr. Peter Hruschka, Dr. Gernot Starke und Mitwirkende).

Original-Template: <https://arc42.org>

Lizenz: Creative Commons Attribution-ShareAlike 4.0 International <https://creativecommons.org/licenses/by-sa/4.0/>

Es wurden Änderungen am originalen arc42-Template vorgenommen.

Dieses angepasste Werk wird unter derselben Creative Commons Attribution-ShareAlike 4.0 International Lizenz bereitgestellt.

Dokumentationsversion: 1.0.0-rc. Draft, 2026-05-19

1. Einführung und Ziele

Dieses Dokument beschreibt das Mont Terri Experiment Information System, kurz MONTEIS. Mit MONTEIS soll ein Zentrales System für das Mont Terri Konsortium von Grund auf neu gebaut werden, welches die Bedürfnisse der Forscher im Bereich der Geologie im Felslabor Mont Terri automatisiert.

1.1. Aufgabenstellung

MONTEIS (Mont Terri Experiment Information System) ist ein neues zentrales Informationssystem für das Mont Terri Felslabor. Es dient der strukturierten Erfassung, Verwaltung und Bereitstellung sämtlicher experimentbezogener Daten aus dem Felslaborumfeld.

Die bestehende Systemlandschaft ist historisch gewachsen, heterogen und technisch fragmentiert. Sie erfüllt die heutigen Anforderungen an Datenvolumen, Datenvielfalt, Nachvollziehbarkeit und langfristige Verfügbarkeit aufgrund veralteter Architekturen nicht mehr ausreichend. Daher wurde im Rahmen einer Vorstudie festgestellt, dass eine komplette Neuentwicklung (Greenfield-Ansatz) eines zentralen Systems notwendig ist.

Ziel der Aufgabenstellung ist die Bereitstellung einer zukunftsfähigen, offenen und modular erweiterbaren Plattform, welche die vollständige Digitalisierung und Integration aller relevanten Datenquellen des Mont Terri Felslabors ermöglicht. Dazu gehören insbesondere experimentbezogene Messdaten, geologische Modelle, Sensordaten sowie wissenschaftliche Dokumente.

Die Dimensionen des Systems sind enorm: Bisher wurden ca. 50 Milliarden Messungen durchgeführt. Täglich kommen mehr als 2 Millionen neue Messungen hinzu. Über die geplante Lebenszeit des Systems wird ein Datenvolumen von mehr als 200 Milliarden Messungen (ca. 10 TB) erwartet.

Das System soll den internationalen Partnern des Mont Terri Konsortiums einen einheitlichen, webbasierten Zugang zu diesen Informationen ermöglichen. Dabei spielt die permanente Datenverfügbarkeit und die georeferenzierte Strukturierung der Daten eine zentrale Rolle.

Zwingende technologische Vorgabe: MONTEIS muss auf 100 % Open-Source-Technologien basieren. Zudem ist der Source-Code zwingend auf GitHub zu publizieren, um maximale Transparenz und eine unabhängige Weiterentwicklung sicherzustellen.

1.2. Qualitätsziele

Die Anforderungen an MONTEIS lassen sich in funktionale und nicht-funktionale Aspekte sowie übergeordnete Qualitätsziele gliedern. Der Schwerpunkt liegt auf dem Umgang mit grossen, heterogenen Datenmengen sowie der langfristigen Sicherstellung von Datenzugriff, Integrität und Systemstabilität.

1.2.1. Zentrale funktionale Anforderungen (Überblick)

Das System muss insbesondere folgende fachliche Fähigkeiten unterstützen:

- Zentrale Verwaltung von experimentbezogenen Daten über den gesamten Lebenszyklus (teilweise >20 Jahre).
- Erfassung und Integration heterogener Datenquellen (Sensoren, Zeitreihen, Dokumente, geologische Modelle).
- Verwaltung von Experimenten, Messstellen und zugehörigen Metadaten.
- Bereitstellung eines performanten webbasierten Zugriffs, inklusive interaktiver 3D-Ansichten und räumlicher Navigation.
- Unterstützung der Suche (z. B. Region of Interest-Suche in 2D/3D, Volltext) und Wiederauffindbarkeit von Daten.
- Automatisierter Import (via Daten-Pipelines) aus bestehenden Datenerfassungssystemen (DAS) wie Geomonitor über FTP oder Kafka.
- Bereitstellung von Daten für die wissenschaftliche Auswertung über strukturierte Exportfunktionen.

1.2.2. Übergeordnete Qualitätsziele (ISO 25010-orientiert)

Die folgenden Qualitätsziele sind für MONTEIS besonders relevant und wurden mit messbaren Kriterien aus dem Pflichtenheft hinterlegt:

Nr.	Qualitätsziel	Beschreibung & Messkriterien
1	Sicherheit	Schutz sensibler Forschungsdaten durch rollenbasierte Zugriffskontrolle (RBAC) , sichere Authentifizierung via eIAM und Einhaltung des IKT-Grundschutzes des Bundes.
2	Datenintegrität	Sicherstellung der verlustfreien Speicherung (absolut kein Datenverlust). Jederzeitige Verfügbarkeit unprozessierter Originaldaten und Speicherung standardmässig in SI-Einheiten.
3	Verfügbarkeit	Gewährleistung eines hochverfügbaren Systems mit minimalen Ausfallzeiten (gefordert ist eine Downtime von < 2 %).
4	Performanz	Effiziente interaktive Nutzung (z. B. durch Resampling-Funktionen beim Plotten langer Zeitreihen) , die gewährleistet ist, auch wenn bis zu 30 Benutzer gleichzeitig arbeiten.
5	Skalierbarkeit	Fähigkeit zur performanten Verarbeitung eines Zielvolumens von über 200 Milliarden Messungen. Die Architektur muss so modular sein, dass Daten-Pipelines unabhängig operieren, ohne das System zu blockieren.
6	Interoperabilität	Nahtlose Anbindung an Umsysteme (wie BIM, Fulcrum und Citavi) über Schnittstellen. Problemlose Verarbeitung der stark heterogenen Datenformate (TXT, CSV, LAS, PDF etc.).

Nr.	Qualitätsziel	Beschreibung & Messkriterien
7	Offenheit & Transparenz	Konsequente Nutzung von 100 % Open-Source-Komponenten. Publikation des vollständigen Quellcodes auf GitHub.
8	Nachvollziehbarkeit	Lückenlose Dokumentation von Systemoperationen. Sicherstellung der Verifizierbarkeit bei manuellen Daten-Uploads sowie Nutzung von Zeitstempeln.
9	Benutzerfreundlichkeit	Intuitive Bedienung im Browser ohne zusätzliche Plugins. Selbstständige Konfigurierbarkeit von Alarmierungen und Dashboards durch die Nutzer.

1.3. Stakeholder

Rolle	Kontakt	Erwartungshaltung
Auftraggeberin / Betreiberin	swisstopo (Landesgeologie)	Wünscht eine Effizienzsteigerung, automatisierte Validierung, Ablösung veralteter Systeme zur Kostenersparnis und eine Vorreiterrolle durch innovative Open-Source-Technologien.
Projektpartner & Forscher	Mont Terri Konsortium (22 Partner, inkl. ENSI, ETH, nagra)	Benötigen einen performanten, automatisierten Zugang zu den Daten, einfache Wiederauffindbarkeit (z.B. über 3D-Viewer) und Exportmöglichkeiten für wissenschaftliche Auswertungen.
Administratoren	swisstopo-interne Mitarbeiter	Einfache Verwaltung von Benutzerkonten, Systemüberwachung via Cockpit-View, voll konfigurierbare Daten-Pipelines und Alerting-Funktionen.

2. Randbedingungen

Dieser Abschnitt beschreibt die Vorgaben und Einschränkungen, die bei der Konzeption und Umsetzung von MONTEIS zwingend zu beachten sind. Sie schränken den Lösungsraum für Architekten und Entwickler bewusst ein, um die strategischen und rechtlichen Vorgaben des Bundes und des Mont Terri Konsortiums zu erfüllen.

2.1. Technische Randbedingungen

Randbedingung	Beschreibung
100 % Open Source	Das System, inklusive aller eingebundenen Produkte von Drittanbietern, darf ausschliesslich auf Open-Source-Komponenten basieren. Es dürfen keine Lizenzgebühren für Softwarekomponenten anfallen.
Cloud-Infrastruktur	Die neuen Applikationen, Datenbanken und Pipelines müssen zwingend in einer Cloud-Infrastruktur betrieben werden (als Beispiele werden u.a. AWS-Cloud-Services, S3 Buckets, Kubernetes und Docker genannt).
Zentrales Identity Management	Die Authentifizierung der Nutzer hat zwingend über eIAM (oder vergleichbare SSO-Provider wie AGOV) der Bundesverwaltung zu erfolgen.
Browser- und Plugin-Freiheit	Die Web-App (Frontend) muss auf einer der gängigen Plattformen vollständig im Webbrowser funktionieren, ohne dass zusätzliche Plugins installiert werden müssen.
Einheiten und Standards	Sensormesswerte müssen systemintern zwingend in SI-Einheiten abgespeichert und dargestellt werden. Ursprüngliche Einheiten müssen jedoch bei Bedarf weiterhin verfügbar und abrufbar bleiben.
System-Schnittstellen	MONTEIS muss zwingend über Schnittstellen (APIs/Pipelines) mit den bereits existierenden Systemen BIM Mont Terri (3D-Geometrien/Infrastruktur), Fulcrum (mobiles Logbuch/Proben) und Citavi (Publikationsdatenbank) interagieren können.

2.2. Organisatorische Randbedingungen

Randbedingung	Beschreibung
Transparenz & Quellcode	Gemäss Vorgaben der Bundesverwaltung (bzw. EMBAG) muss der gesamte Source-Code der Applikationen auf GitHub publiziert werden und anschliessend für die Öffentlichkeit frei zugänglich sein.
Agile Methodik & Etappierung	Die Entwicklung hat strikt nach agilen Software-Entwicklungsmethoden zu erfolgen. Zudem ist ein etappiertes Vorgehen vorgeschrieben: Zuerst wird ein MVP (Priorität 1) entwickelt, gefolgt von optionalen Erweiterungen (Priorität 2 und 3).
Sprache	Die Kommunikations-, Applikations-, Daten- und Dokumentationssprache ist zwingend Deutsch.

Randbedingung	Beschreibung
Präsenz vor Ort	Zwar können die Arbeiten primär in den eigenen Räumlichkeiten des Anbieters erfolgen, jedoch wird erwartet, dass der Auftragnehmer regelmässig (voraussichtlich alle 4 Wochen) an Besprechungen in den Räumlichkeiten der Bedarfsstelle (St-Ursanne) teilnimmt.

2.3. Konventionen und Sicherheitsvorgaben

Randbedingung	Beschreibung
IKT-Grundschutz	Das System muss den obligatorischen minimalen Sicherheitsanforderungen des Bundes bezüglich Informationssicherheit (IKT-Grundschutz) genügen.
Vollständige Dokumentation	Das System, die Pipelines und der Code müssen vollständig dokumentiert werden und die Dokumentation muss während der gesamten Auftragsdauer gepflegt und à jour gehalten werden. Änderungen im System müssen nachvollziehbar dokumentiert werden.

3. Kontextabgrenzung

Dieses Kapitel beschreibt das System in seiner Umgebung. Es grenzt MONTEIS von seinen externen Nachbarsystemen ab und definiert die Schnittstellen und den Datenfluss zu den angebundenen Akteuren und Systemen.

3.1. Fachlicher Kontext

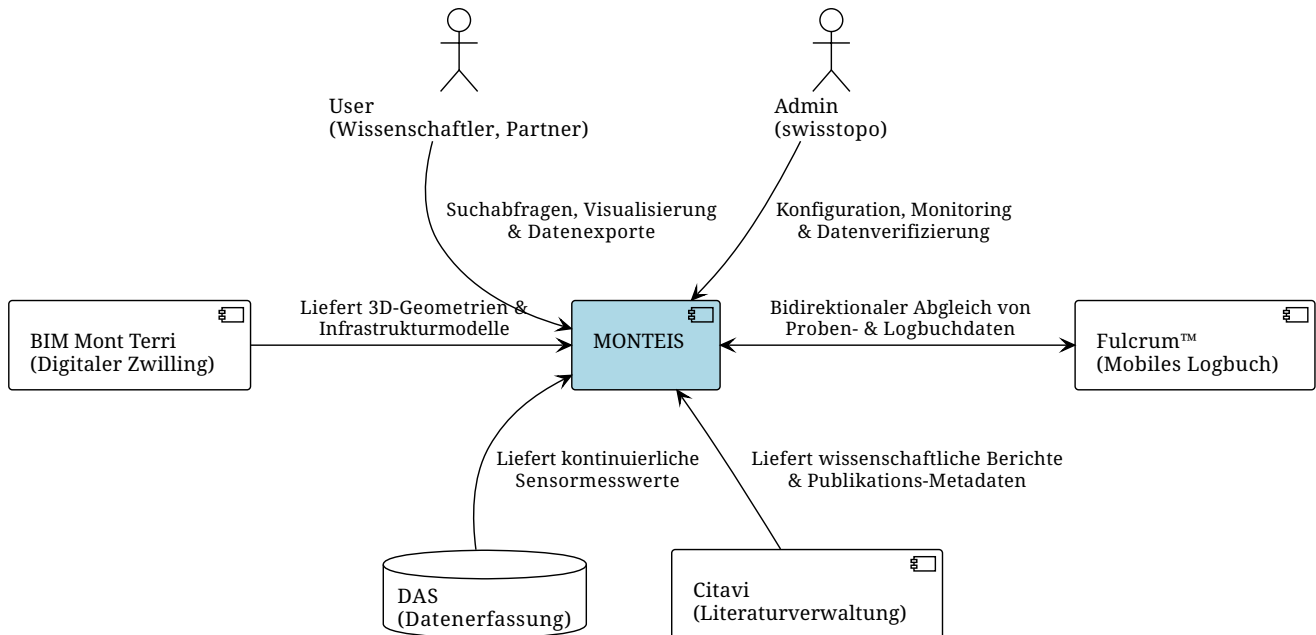
MONTEIS operiert nicht isoliert, sondern agiert als zentrales "Overarching System" (Informationsdrehscheibe) im Felslabor Mont Terri, das Daten aus verschiedenen spezialisierten Systemen bündelt.

Akteure:

- **Administrator (swisstopo):** Hat weitreichende Konfigurationsrechte. Er legt neue Experimente und Sensoren an, konfiguriert die Daten-Pipelines, richtet das Alerting ein und muss manuell hochgeladene Daten verifizieren.
- **User (Wissenschaftler, PIs, Projektpartner):** Greifen auf das System zu, um Daten über die 3D-Ansicht oder Filter zu suchen, lange Zeitreihen zu plotten, Exporte durchzuführen und Kommentare zu verfassen.

Nachbarsysteme:

Nachbarsystem	Typ	Fachliche Beschreibung & Informationsfluss
BIM Mont Terri	3D-Geometrie & Infrastruktur	Fungiert als digitaler Zwilling des Felslabors (Infrastruktur, Assets und Bohrlochdatenbank). MONTEIS bezieht von hier die 3D-Strukturen und Bohrlochgeometrien, um diese im eigenen 3D-Viewer darzustellen und für die räumliche Suche nutzbar zu machen.
Fulcrum™	Mobiles Logbuch / Vor-Ort-Erfassung	Cloud-Datenbank zur mobilen Erfassung von Ereignissen, Proben und Bohrlöchern direkt auf Tablets im Tunnel. Da Fulcrum das primäre Arbeits-Tool vor Ort bleibt, müssen Logbuch- und Assetdaten zwingend bidirektional mit MONTEIS abgeglichen werden.
Citavi	Publikations- & Berichtsdatenbank	Verwaltet wissenschaftliche Berichte, technische Notizen und Zeitschriftenartikel. Die darin enthaltenen Dokumente (PDFs) und Metadaten (z. B. Autor, Schlagwörter) werden periodisch in MONTEIS überführt, um sie direkt mit den jeweiligen Experimenten zu verknüpfen.
DAS (Data Acquisition Systems)	Sensorik / Datenerfassung	Lokale Erfassungssysteme (wie Geomonitor, Geoscope, Glözl etc.), die Millionen von Sensordaten (z. B. Druck, Temperatur) aufzeichnen. Diese Messwerte fließen kontinuierlich in MONTEIS ein.



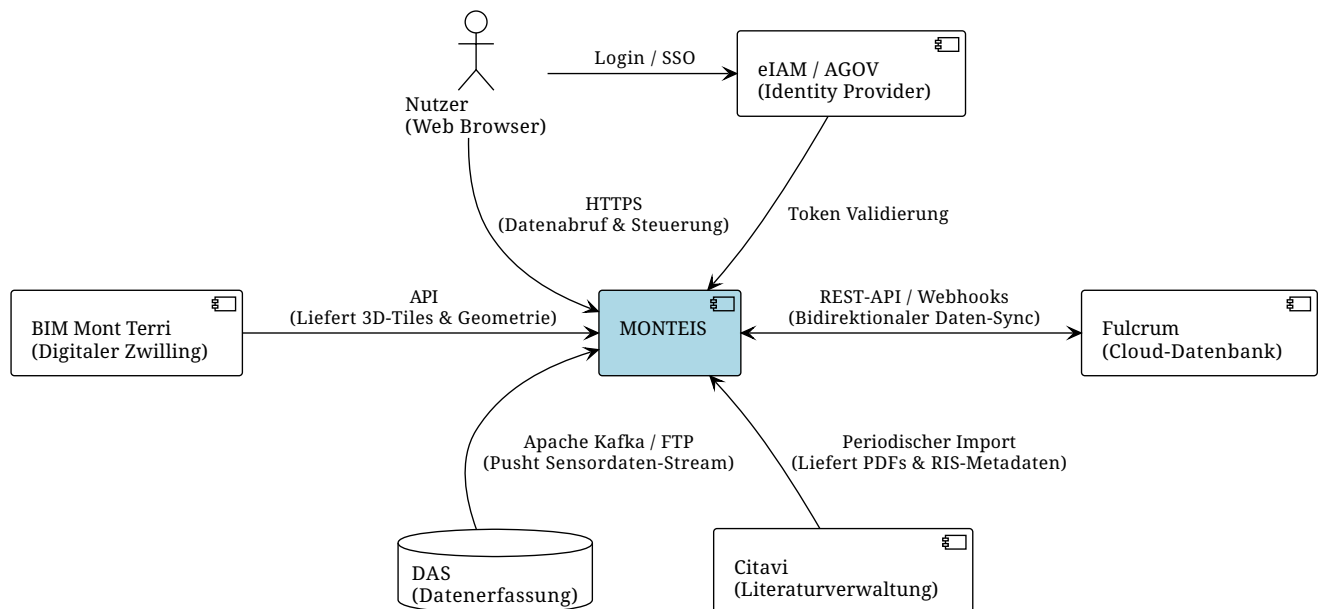
3.2. Technischer Kontext

Die technische Anbindung der Nachbarsysteme an MONTEIS erfolgt über klar definierte, standardisierte Schnittstellen. In dieser Sicht wird MONTEIS als Blackbox betrachtet, die detaillierten API-Spezifikationen (REST-Contracts) werden in der separaten API-Dokumentation (z. B. via Swagger/OpenAPI) gepflegt.

An den Systemgrenzen kommen folgende technische Integrationsmuster zum Einsatz:

- **Daten-Pipelines (Ingestion von DAS)**: Die Anbindung der lokalen Datenerfassungssysteme (DAS) erfolgt entweder über etablierte Dateitransfer-Protokolle (wie FTP) oder über moderne Event-Streaming-Technologien wie Apache Kafka, welches extrem geringe Latenzen (<10ms) bei hoher Bandbreite für die Hochfrequenz-Sensordaten bietet.
- **Format-Handling an der Systemgrenze**: Die eingehenden DAS-Schnittstellen müssen extrem heterogene Dateiformate (TXT, CSV, ASCII, XLS, JPG, TIF, LAS, LAZ, etc.) automatisiert entgegennehmen können.
- **REST-APIs & Webhooks (BIM & Fulcrum)**: Die Kommunikation mit den Umsystemen BIM und Fulcrum erfolgt über zustandslose REST-APIs, wobei strukturierte Daten im JSON-Format ausgetauscht werden. Für ereignisbasierte Updates (z. B. aus dem mobilen Logbuch Fulcrum) werden zusätzlich Webhooks eingesetzt.
- **Authentifizierungsschnittstelle (eIAM)**: Das System delegiert die Authentifizierung vollständig nach aussen. Das Login erfolgt über einen externen Call (z. B. via SAML oder OIDC) an das System **eIAM** oder **AGOV** der Bundesverwaltung.

Monteis mit Umsysteme



3.3. Mapping fachliche auf technische Schnittstellen

Die folgende Tabelle stellt den Bezug zwischen den ausgetauschten fachlichen Informationen (siehe 3.1) und deren technischer Implementierung (siehe 3.2) her:

Nachbarsystem	Fachliche Schnittstelle (Das "Was")	Technische Schnittstelle (Das "Wie")
DAS (Datenerfassung)	Kontinuierlicher Zufluss von rohen Sensormesswerten (z.B. Druck, Temperatur).	Daten-Stream via Apache Kafka (für Latenzen <10ms) oder Dateitransfer via FTP .
Fulcrum™	Bidirektionaler Abgleich von mobilen Logbucheinträgen, Proben und Asset-Daten.	Zustandslose Kommunikation via REST-API (Format: JSON) sowie ereignisbasierte Updates über Webhooks .
BIM Mont Terri	Bereitstellung der 3D-Infrastrukturgeometrien und Bohrlochdaten des Tunnelsystems.	Abfrage über eine API (z. B. für Cesium 3D-Tiles zur Visualisierung im Browser).
Citavi	Bereitstellung von wissenschaftlichen Berichten (PDFs) und Publikations-Metadaten.	Kein direkter API-Zugriff möglich; Umsetzung über manuellen oder periodischen Import von Austauschformaten (z. B. RIS und PDF-Dateien).
eIAM / AGOV	Sichere Identifikation und Authentifizierung der Nutzer (Admins, Wissenschaftler).	Externer Aufruf des föderativen Identity Providers der Bundesverwaltung via SAML / OIDC .

4. Lösungsstrategie

Dieses Kapitel fasst die grundlegenden Entwurfsentscheidungen und Lösungsansätze zusammen, die die Architektur von MONTEIS massgeblich prägen. Es beschreibt, mit welchen technologischen und strukturellen Konzepten die geforderten Qualitätsziele (insbesondere Skalierbarkeit, Performanz und Verfügbarkeit) erreicht werden.

4.1. Erreichung der wichtigsten Qualitätsziele

Die folgende Tabelle gibt einen Überblick (Executive Summary), mit welchen architektonischen Ansätzen und Taktiken die priorisierten Qualitätsziele erreicht werden. Detaillierte technische Beschreibungen folgen in den darauffolgenden Unterkapiteln.

Qualitätsziel	Szenario	Lösungsansatz & Architekturentscheidung	Link zu Details
Performanz & Skalierbarkeit	Ein Wissenschaftler möchte eine 10-jährige Zeitreihe eines Drucksensors visualisieren. Das System darf nicht einfrieren und muss bei 30 gleichzeitigen Nutzern performant bleiben.	Polyglot Persistence & Downsampling: Hochfrequente Messwerte (Zielvolumen: 200 Mrd. Messungen) landen in einer spezialisierten TimeScaleDB, Metadaten in PostgreSQL. Für lange Zeitreihen wird systemseitig ein Downsampling angewendet, um die Payload für das Frontend zu reduzieren.	→ Kap. 5.4 & 6.8
Verfügbarkeit & Robustheit	Ein Datenerfassungssystem (DAS) sendet fehlerhafte oder extrem grosse Datenmengen. Dies darf andere Experimente oder die Web-App nicht beeinträchtigen.	High Availability & Event-Streaming: Bereitstellung im Kubernetes-Cluster (AWS) mit High-Availability-Images. Apache Kafka (Connect) dient als asynchroner Puffer. Jede DAS-Pipeline operiert strikt isoliert als eigener Pod/Connector, sodass Ausfälle gekapselt bleiben.	→ Kap. 5.3.2 & 6.1
Sicherheit	Zugriffe auf sensible Experimentdaten müssen strikt geregelt und vor unbefugtem Zugriff geschützt sein.	Zentrales API-Gateway & RBAC via Keycloak: Direkte Zugriffe auf Pipelines oder Datenbanken sind unterbunden; Frontend und Backend fungieren als Nadelöhr. Die Authentifizierung (eIAM) wird über Keycloak angebunden. Rollen werden dort über Gruppenmitgliedschaft vergeben und als Claims im JWT an das Spring Boot Backend übergeben, welches die Berechtigungen durchsetzt.	→ Kap. 5.3.1 & 6.6

Qualitätsziel	Szenario	Lösungsansatz & Architekturentscheidung	Link zu Details
Datenintegrität	Historische Sensordaten und deren Umrechnungen müssen manipulationssicher und jederzeit nachvollziehbar gespeichert werden.	Trennung von Roh- und Prozessdaten: Sensordaten werden direkt und unverfälscht abgelegt. Die Normalisierung erfolgt ausschliesslich in der Pipeline. Ändert sich eine Berechnungsformel, müssen die Daten zwingend über die Pipeline aktualisiert werden. Rohdaten landen parallel in einem S3 Bucket, während das Schema der TimeScaleDB absichtlich schlank gehalten wird.	→ Kap. 5.3.2 & 6.2 & 6.9
Interoperabilität	Das System muss heterogene Daten aus Umsystemen (BIM, Fulcrum, Citavi) und diversen Sensoren verarbeiten.	Spezialisierte Backend-Komponenten & Kafka Sinks: Die Anbindung von Umsystemen erfolgt über dedizierte Spring Boot Komponenten. Die Heterogenität der DAS-Sensordaten wird durch das Kafka Connect Cluster abgefangen, welches vielfältige Formate entgegennehmen und standardisieren kann.	→ Kap. 5.3.2
Offenheit & Transparenz	Das System muss zukunftssicher und ohne Vendor-Lock-in durch Lizenzkosten betrieben werden können.	100 % Open Source: Alle eingesetzten Komponenten werden auf ihre Open-Source-Lizenz geprüft. Der Source Code wird zentral bei swisstopo auf GitHub geführt und gemanagt.	→ Kap. 4.3
Nachvollziehbarkeit	Uploads und Systemzustände müssen für Administratoren transparent und auditierbar sein.	Statusbasierte Ablage: Manuelle Uploads werden zunächst in einem isolierten <code>manual_uploads</code> -Ordner im S3-Bucket zwischengespeichert. Erst nach der expliziten Freigabe durch einen Administrator werden die Daten via Pipeline in die TimeScaleDB und den Cold Storage überführt.	→ Kap. 5.4 & 6.2
Benutzerfreundlichkeit	Nutzer benötigen individuelle Konfigurationsmöglichkeiten und eine intuitive Bedienung.	User-Centric Design & Customizing: UI-Designs und Workflows werden durch die Anwender validiert. Die Applikation erlaubt es jedem Nutzer, relevante Einstellungen (Alarmer, Darstellungseinheiten aus einem definierten Set) individuell zu konfigurieren.	→ Kap. 5.2

4.2. Fundamentale Architekturentscheidungen (Top-Level)

MONTEIS folgt einer verteilten, Cloud-nativen **Microservices-orientierten Architektur**, die in drei logische und physische Schichten unterteilt ist:

1. **Interface Layer:** Angular Web-Frontend.
2. **Business Layer:** Spring Boot Backend und Daten-Pipelines (Kafka Connect).
3. **Storage Layer:** TimeScaleDB, PostgreSQL, AWS S3 und S3 Glacier.

Entkopplung und Skalierung: Die verschiedenen Schichten und Services werden als separate Pods in einem Kubernetes-Cluster bereitgestellt. Dies ermöglicht eine granulare und individuelle horizontale Skalierung (z. B. Skalierung des Backends bei hohen Zugriffszahlen oder Erhöhung der Pipeline-Pods bei grossen Daten-Uploads). Die Kommunikation zwischen den Layern erfolgt über REST-APIs, während Datenbanken über effizientes Connection Pooling angebunden werden.

4.3. Technologie-Entscheidungen (Tech Stack)

- **Frontend:** Die Web-Applikation wird mit **Angular** umgesetzt, ergänzt durch **Angular Material**, **AG-Grid** und **Scion Workbench** für eine konsistente und moderne UI. Für die komplexe Darstellung der geologischen 3D-Modelle kommt **Giro3D** zum Einsatz.
- **Backend & API:** Als zentrales Verarbeitungssystem dient eine **Java Spring Boot** Applikation. Jede Datenanfrage oder -eingabe die durch einen User ausgelöst wird, passiert dieses Backend. Einzig die Pipeline kann autonom agieren.
- **Data Pipelines:** Die Ingestion der Sensordaten erfolgt über ein **Kafka Connect Cluster**, gefolgt von spezifischen Java- oder Python-Pods für die weitere Datenverarbeitung. Für Benachrichtigungen wird das AWS Alerting genutzt.
- **Storage-Split:** Zur performanten Bewältigung der bis zu 50 TB an Daten kommen vier Speichersysteme zum Einsatz:
 - **TimeScaleDB:** Für das performante Speichern und Abfragen von Echtzeit-Messpunkten.
 - **PostgreSQL (mit PostGIS):** Für zugehörige Metadaten und die vollständig georeferenzierte Ablage der Systemhierarchien.
 - **AWS S3:** Für Dateien aus Umsystemen, manuelle Uploads und die temporäre (24h) Zwischenspeicherung von Rohdaten.
 - **AWS S3 Glacier Deep Archive:** Für die kosteneffiziente Langzeitarchivierung. Ein täglicher Batch-Job überführt Rohdaten aus dem regulären S3 dorthin.

4.4. Integrations- und Schnittstellenstrategie

Isolierung der Pipelines: Um zu verhindern, dass ein fehlerhafter Sensordaten-Feed das System beeinträchtigt, kann das Input-Topic falls nötig mittels Broker vor dem Kafka Connect Cluster in verschiedene Output-Topics (z. B. ein Topic pro Sensor) aufgeteilt werden. Jede DAS-Pipeline verfügt über einen eigenen Connector. Im Extremfall existiert pro Sensor ein eigener Pipeline-Pod.

Dies reduziert die zu verarbeitende Datenmenge pro Pod massiv, garantiert höchste Performanz, isoliert Fehler und optimiert die Cloud-Kosten.

3D-Modellierung (IFC-Integration): Anstatt eine aufwändige direkte API-Kopplung für 3D-Modelle zu bauen, stellt MONTEIS dem Administrator eine Upload-Funktion für IFC-Dateien (z. B. aus BIM Mont Terri) zur Verfügung. Diese Modelle werden im Frontend über Giro3D und **Three.js** nativ in Web-Workern des Browsers gerendert. Dies lagert die Rechenlast auf den Client aus und schützt die Performance der restlichen Web-App.

4.5. Organisatorisches & DevOps

CI/CD & Deployment: Der Entwicklungsprozess stützt sich auf moderne, hochautomatisierte DevOps-Infrastrukturen:

- **Pipelines:** Bei jedem Commit wird Code-Formatting und automatisiertes Testing ausgeführt. Merge Requests durchlaufen zudem eine statische Codeanalyse (z. B. SonarQube).
- **Deployment-Flow:** Vom **main**-Branch erfolgt ein vollautomatisches Deployment auf die **dev**-Umgebung. Getaggte Versionen werden auf **staging** für das Kunden-Review bereitgestellt. Nach Abnahme wird das getestete Image auf die **prod**-Umgebung propagiert.
- **Security & Maintenance:** Mittels Weekly Builds und Rolling Updates wird das System aktuell gehalten. Container werden kontinuierlich via Trivy oder Dependency-Track gescannt. Für das vereinfachte Abhängigkeits-Management ist der Einsatz von **Renovate** im Repository vorgesehen.

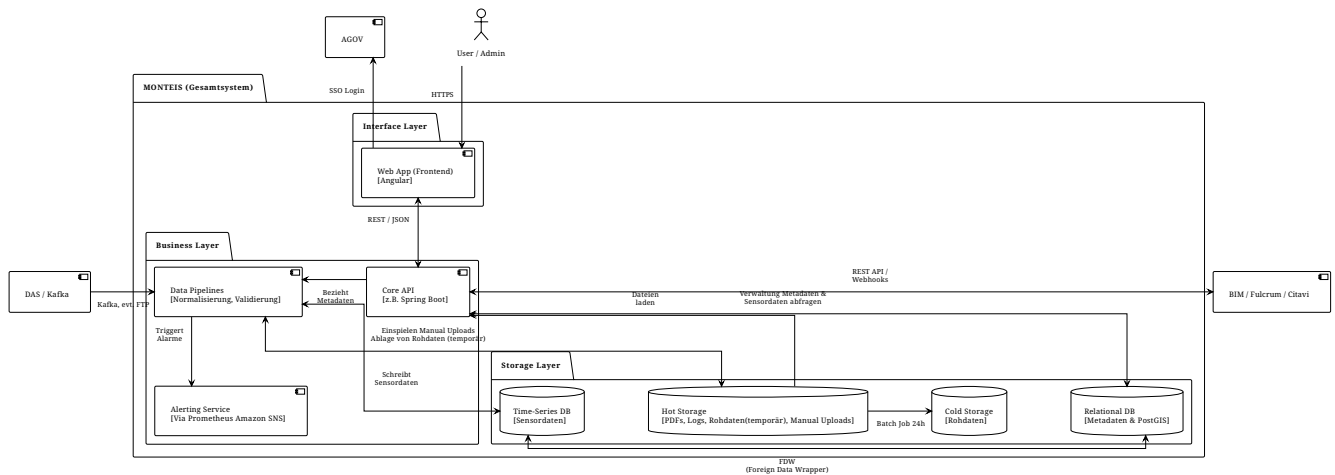
Open Source Compliance: Es wird prozessual (z. B. durch Lizenz-Scanning in der CI/CD-Pipeline) sichergestellt und dokumentiert, dass jede verwendete Komponente und Bibliothek unter einer anerkannten Open-Source-Lizenz steht.

5. Bausteinsicht

Dieses Kapitel beschreibt die statische Struktur des Systems MONTEIS. Die Bausteinsicht ist hierarchisch aufgebaut: Level 1 beschreibt die groben Architektur-Schichten (Gesamtsystem), Level 2 detailliert die inneren Komponenten und Zugriffslogiken dieser Schichten.

5.1. Whitebox Gesamtsystem (Level 1)

Das System folgt einer strikten 3-Schicht-Architektur. Die Kommunikation zwischen den Layern ist durch eindeutige Zuständigkeiten und gerichtete Datenflüsse geregelt.



5.1.1. Interface Layer (Frontend)

Stellt die Benutzeroberfläche für Wissenschaftler und Administratoren bereit. Nimmt Benutzerinteraktionen entgegen, visualisiert 3D-Modelle und rendert hochfrequente Zeitreihendiagramme.

5.1.2. Business Layer (Backend & Pipelines)

Das logische Herzstück des Systems. Diese Schicht ist dreigeteilt:

1. Ein synchrones **Spring Boot Backend** als zentrales API-Gateway und Metadaten-Verwalter.
2. Ein asynchrones **Pipeline & Worker-Ökosystem** (Kafka-basiert) für die massenhafte Verarbeitung und Speicherung der Sensordaten.
3. Ein **Alerting Service** für die individuelle Alarmierung mittels AWS SNS (Simple Notification Service).

5.1.3. Storage Layer (Datenbanken & Cloud Storage)

Nutzt den Ansatz der "Polyglot Persistence", um die massiven Datenmengen (Ziel: 200 Mrd. Messungen) performant und kosteneffizient abzulegen. Beinhaltet TimeScaleDB (Zeitreihen), PostgreSQL + PostGIS (Metadaten) sowie AWS S3 (Dateien und Rohdaten) und AWS S3 Glacier (Rohdaten).

5.2. Whitebox Interface Layer (Level 2)

TODO: Draft da die Architektur noch nicht abschliessend gemacht ist

Obwohl die genaue Modulstruktur des Frontends noch in der Spezifikationsphase ist, basiert die Architektur der Web-Applikation auf **Angular** und dem **Angular Material Design System**. Um die Komplexität der Darstellung zu managen, gliedert sich die UI logisch in folgende Hauptbereiche, die durch die **Scion Workbench** als Layout-Framework orchestriert werden:

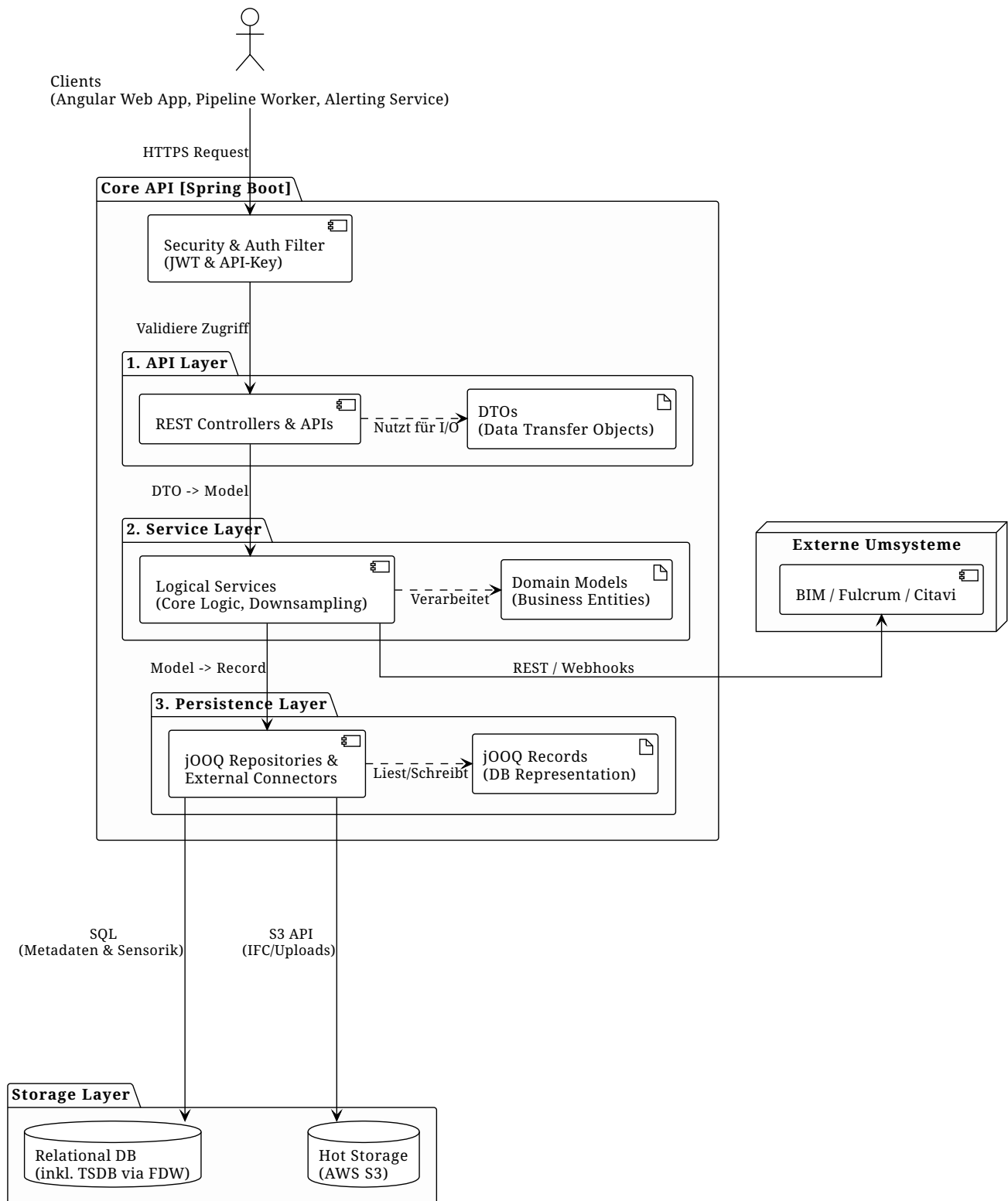
- **3D-Visualisierungs-Modul:** Nutzt **Giro3D**, um die Infrastruktur, Bohrlochgeometrien und Sensoren räumlich im Browser darzustellen, ohne die Performance der restlichen Applikation zu blockieren. Dies Wird erreicht mittels Worker vom Browser selber, damit können wir den Load des Rendering sauber auslagern auf das Client Device.
- **Data-Grid & Dashboard-Modul:** Integriert **ag-Grid** für die hochperformante tabellarische Darstellung und das Plotten der gefilterten Sensor-Zeitreihen.
- **Admin- & Konfigurations-Modul:** Schnittstelle für Administratoren zur Verwaltung von Experimenten, Freigabe von manuellen Uploads und Konfiguration von Alarmen.

5.3. Whitebox Business Layer (Level 2)

Der Business Layer besteht aus zwei stark entkoppelten Hauptsystemen: Dem Spring Boot Backend und den Kafka-basierten Daten-Pipelines.

5.3.1. Core API (Spring Boot Backend)

Core API (Spring Boot Backend)



Das Backend fungiert als Dreh- und Angelpunkt für jegliche synchrone Kommunikation. Es beinhaltet folgende logische Services:

- **Security & Auth Filter:** Der Authentifizierungs-Filter von Spring Security's OAuth2-Resource-Server verifiziert das Keycloak/eIAM-JWT (Signatur, Aussteller, Audience) und baut daraus über einen custom **Converter** (**MonteisJwtAuthenticationConverter**) in einem Schritt die vollständige **Authentication** auf: Rolle (über Keycloak-Gruppen vergeben) → **api:***-Authorities, sowie Sub und Experiment-IDs als eigener **MonteisPrincipal**, abgelegt im Standard-**SecurityContextHolder**.

Leseberechtigungen werden darüber transparent auf Datenbankebene per Row-Level-Security durchgesetzt, Schreibberechtigungen anhand der HTTP-Methode zentral im Filterchain (siehe Kap. 8 & 9). Ein schlanker `GET /api/me`-Endpunkt spiegelt die Schreibberechtigung an das Frontend, das damit rein kosmetisch schreibende UI-Elemente ausblendet.

- **Query & Downsampling Service:** Ein dedizierter Service, der Leseanfragen an die TimeScaleDB richtet und intelligente Downsampling-Algorithmen anwendet, um die abgerufene Datenmenge vor der Auslieferung an das Frontend zu reduzieren.
- **IFC Upload & Processing Service:** Nimmt hochgeladene 3D-Geometrie-Modelle (IFC-Dateien) entgegen und bereitet diese für das Rendering (via Giro3D/Three.js) im Frontend vor.
- **Umsystem-Connectoren:** * **Fulcrum-Client:** Konsumiert die API von Fulcrum für die bidirektionale Synchronisation.
- **Frontend-API:** Stellt dem Angular-Client alle benötigten Endpunkte zur Verfügung.
- **Pipeline-API:** Ein durch API-Keys gesicherter interner Endpunkt, über den die Pipeline-Worker Metadaten abrufen können.

5.3.2. Contracts (Geteiltes Schema-Modul)

Damit Core und Pipeline nicht mit divergierenden Vorstellungen desselben Konfigurations-Objekts laufen, existiert ein eigenständiges Maven-Modul `contracts`, das per OpenAPI Generator aus einer einzigen YAML-Spezifikation (siehe Kapitel 8, [API-Schnittstellen \(Contracts\)](#)) generiert wird. Es enthält u. a. das `SensorParameterConfig`-DTO (Kafka-Payload auf `internal-sensor-config`) sowie den `Das`-Enum, der die unterstützten Erfassungssysteme (z. B. `SOLEXPERS`) definiert. Sowohl `core` (Publisher) als auch `pipeline` (`SensorConfigCache`, Normalizer) hängen von `contracts` ab, statt eigene, potenziell abweichende Kopien dieser Typen zu pflegen.

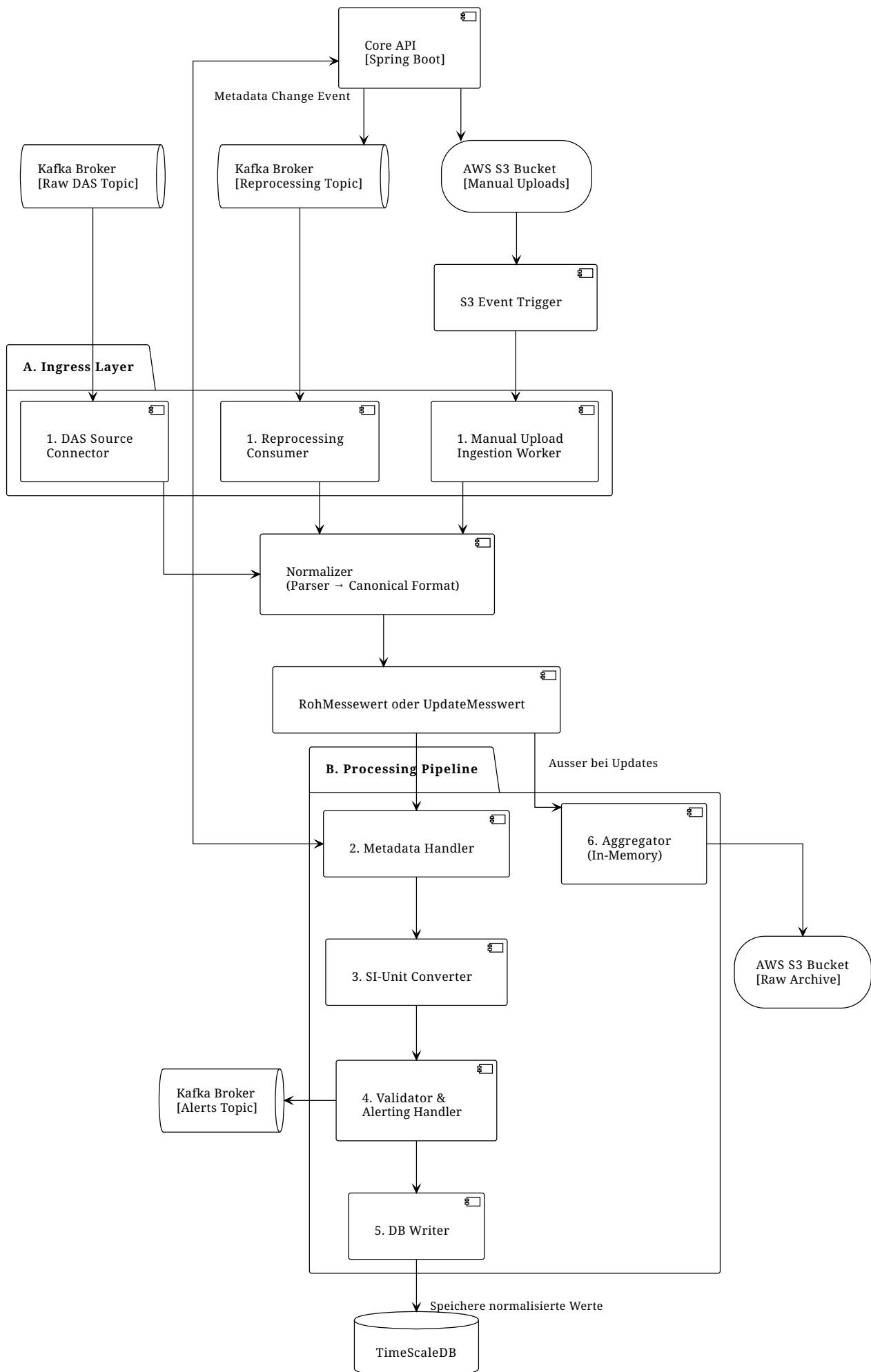
5.3.3. Data Pipelines & Alerting (Kafka & Worker)

Der Ingestion-Prozess für Sensordaten ist asynchron und ereignisgesteuert aufgebaut. Die Verarbeitung innerhalb eines Pipeline-Worker-Pods folgt einem strikten Ablauf:

1. **Format-Parser:** Eine neue Nachricht wird vom Kafka-Broker konsumiert und in ein Standard-Objekt (`NormalizedSensorData` mit dem zusammengesetzten DAS-Key `<DAS> <das_sensor_alias><das_parameter_alias>` als Sensor-Identifikator, `Timestamp`, `Value`) geparkt. Genauere Details zum Standardformat und zum DAS-Key sind in Kapitel 8.
2. **Metadata Handler:** Führt ein Cache-Lookup im `SensorConfigCache` anhand des DAS-Keys durch. Die Cache-Befüllung erfolgt nicht per Cache-Miss-REST-Call, sondern proaktiv über den `internal-sensor-config`-Kafka-Topic (Broadcast an jeden Pod bei jeder Konfigurationsänderung, siehe Kapitel 8).
3. **SI-Unit Converter:** Rechnet den Rohwert anhand einer in den Metadaten hinterlegten Formel (`JsonLogic`) in die standardisierte SI-Einheit um.
4. **Validator & Alerting Handler:** Prüft den umgerechneten Wert gegen definierte Upper-/Lower-Bounds. Bei einer Verletzung wird ein Event emittiert — jedoch ausschliesslich bei Live-Ingestion. Beim Reprocessing historischer Daten wird die Alert-Emission bewusst unterdrückt, um bereits bewertete Messwerte nicht erneut zu alarmieren. **(Hinweis: Architektonische Trennung des Alertings: Grenzwertverletzungen wirft der Worker, System-Alarme wie**

fehlende Daten werden direkt auf dem Kafka-Topic via Grafana und Amazon SNS überwacht).

5. **DB-Writer:** Persistiert den finalen Datensatz in die TimeScaleDB unter der stabilen `sensor_parameter_id` aus dem Cache-Eintrag (nicht dem DAS-Key), das Format wird in Kapitel 8 definiert.
6. **Aggregator (Memory):** Der Originaldatensatz wird im Speicher gehalten. Alle **n** Minuten flusht der Worker die gesammelten Rohdaten direkt in einen AWS S3 Bucket.



5.3.4. Notification & Alerting Service (Dedizierter Pod)

Um die Core API von ressourcenintensiven Hintergrundaufgaben zu entlasten und Probleme bei horizontaler Skalierung (z. B. doppelter E-Mail-Versand) zu vermeiden, wird das Alerting in einen isolierten Microservice ausgelagert.

- **Zweck/Verantwortung:** Zuständig für das dynamische Routing und den Versand von Benachrichtigungen an die Endnutzer.
- **Ablauf & Integration:**
 1. Als zentrale "Rule Engine" fungiert **Grafana**, welches Metriken (Prometheus) und Sensordaten (TimeScaleDB) aggregiert und überwacht.
 2. Bei Alarm-Auslösung sendet Grafana einen Webhook an diesen dedizierten Notification Service.
 3. Der Service fragt die User-Abonnements (Subscriptions) via REST API beim Backend an, welches die Spezifischen konfigurationen enthält.
 4. Anschliessend triggert er den physischen Versand der E-Mails oder SMS über **AWS SNS**.
- **Skalierung:** Dieser Service läuft isoliert vom Rest des Systems und kann so konfiguriert werden (z. B. als Single-Replica), dass Race-Conditions beim Nachrichtenversand architektonisch ausgeschlossen sind.

5.4. Whitebox Storage Layer & Zugriffslogik (Level 2)

Um Datenintegrität und das Single-Point-of-Truth-Prinzip zu wahren, gelten für den Storage Layer strikte Zugriffsregeln ("Data Governance").

Architektonische Besonderheit (Single Connection Pool via FDW): Obwohl Sensordaten (TimeScaleDB) und georeferenzierte Metadaten (PostgreSQL mit PostGIS) physisch in zwei getrennten Datenbanken liegen, nutzt das Spring Boot Backend zur Komplexitätsreduktion nur **einen einzigen Connection Pool**. Der Einstieg erfolgt über die Relationale Datenbank. Die TimeScaleDB wird transparent über einen **Foreign Data Wrapper (postgres_fdw)** für Leseoperationen eingebunden.

Die Durchsetzung der Datenintegrität und strikter Zugriffsverbote erfolgt nicht im Applikationscode, sondern "Bulletproof" direkt auf Datenbankebene. Hierfür werden spezifische Datenbank-Rollen (DB-Roles) und restriktive Privilegien (Grants) konfiguriert.

Datenbank / Storage	Schreib-Rechte (Write)	Lese-Rechte (Read)
TimeScaleDB (Zeitreihen)	Pipeline-Worker: Exklusives Schreibrecht für Sensordaten. Backend (nur Flyway): Das Backend darf zur Laufzeit keine Daten schreiben. Schreibzugriffe des Backends sind streng limitiert auf DDL-Schema-Migrationen durch Flyway (unter Verwendung eines separaten Deployment-DB-Users).	Spring Boot Backend (via FDW): Das Backend greift zur Laufzeit nicht direkt auf die TimeScaleDB zu, sondern liest die Zeitreihen ausschliesslich durch den FDW-Tunnel von der PostgreSQL aus.
PostgreSQL & PostGIS (Metadaten & Georeferenzierung)	Spring Boot Backend: Einziger Writer für System- und Sensorkonfigurationen zur Laufzeit. Backend (nur Flyway): DDL-Schema-Migrationen erfolgen über Flyway mit einem dedizierten Deployment-DB-User.	Spring Boot Backend (direkt): Liest Metadaten direkt und nutzt diese Datenbank als Startpunkt des FDW-Tunnels für Zeitreihen-Abfragen.
AWS S3 (Rohdaten & Uploads)	Pipeline-Worker: Schreiben temporäre Sensor-Rohdaten im Batch. Spring Boot Backend: Schreibt manuell hochgeladene Dateien in dedizierte Buckets. AWS Glue Jobs: Löschen die gepufferten Rohdaten-Files nach der erfolgreichen Batch-Verarbeitung und Verschiebung.	Spring Boot Backend: Liest manuell hochgeladene Dateien, um diese nach Freigabe an die Pipeline zu übergeben. AWS Glue Jobs: Lesen die Rohdaten für den täglichen Transfer in den Cold Storage.
AWS S3 Glacier (Cold Storage)	AWS Glue Jobs: Exklusives Schreibrecht zur langfristigen Archivierung.	Administratoren: Exklusiver Lesezugriff, rein für die Notfall-Wiederherstellung (Disaster Recovery) von Originaldaten.

Obwohl sämtliche Lesezugriffe des Backends auf PostgreSQL über dieselbe DB-Rolle (**core_app**) laufen, filtert eine zusätzliche Row-Level-Security-Schicht diese Zugriffe transparent pro Aufrufer nach Experiment-Zugehörigkeit — Admins sehen alles, reguläre Nutzer nur ihre eigenen Experimente. Details dazu in Kap. 8 ("Core-API Datenzugriff & Sicherheit") und in der Migration [db/meta/schema/V6__row_level_security.sql](#).

6. Laufzeitsicht

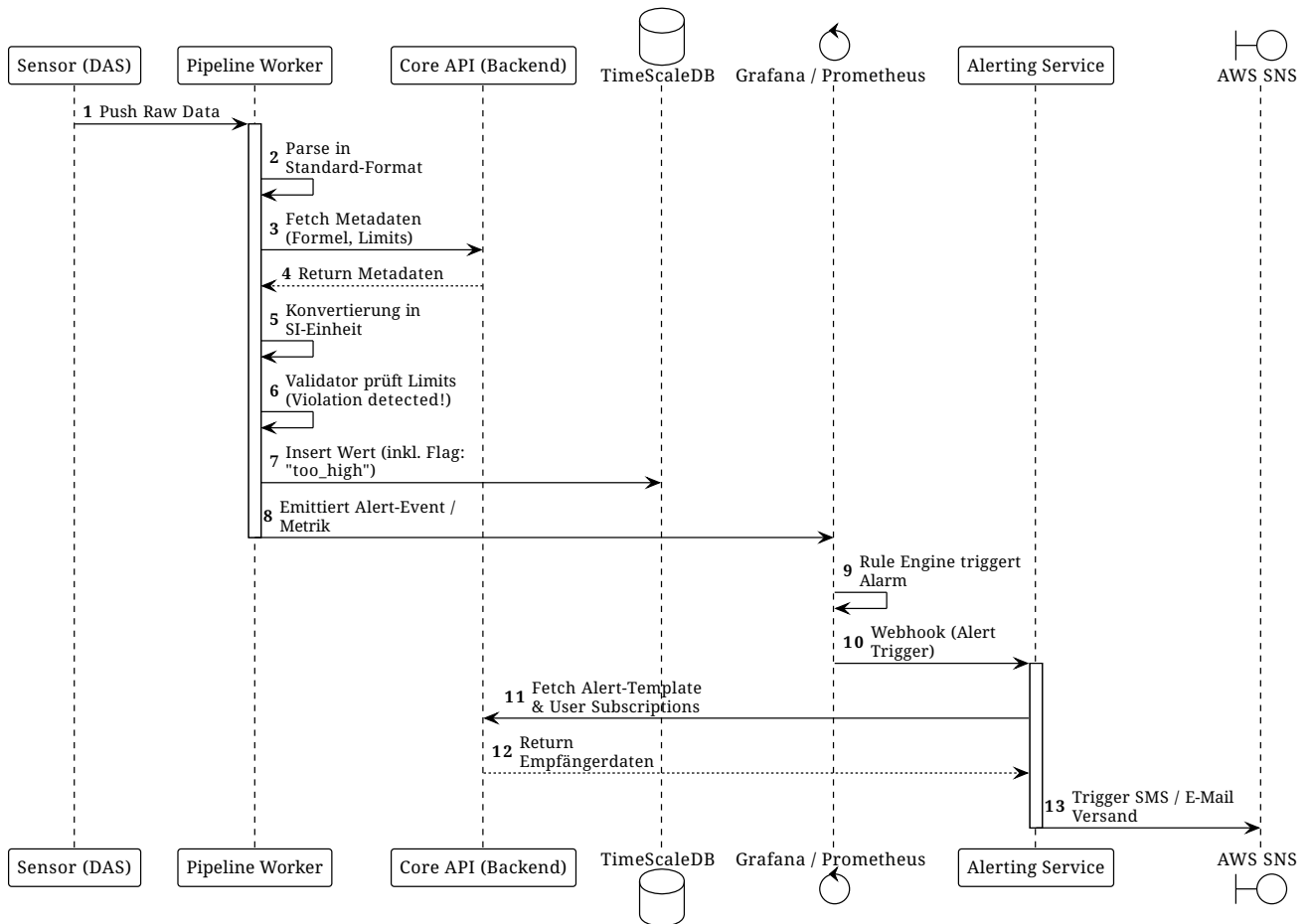
Dieses Kapitel visualisiert das dynamische Verhalten des Systems MONTEIS. Es zeigt das Zusammenspiel der in Kapitel 5 definierten Bausteine anhand der wichtigsten und architektonisch anspruchsvollsten Geschäftsprozesse. Die nachfolgenden Sequenzdiagramme dokumentieren den exakten Ablauf, die Reihenfolge der Aufrufe und die Zuständigkeiten der einzelnen Komponenten zur Laufzeit.

6.1. Szenario 1: Daten-Ingestion & Grenzwert-Alarm (Pipeline Flow)

Dieses Szenario beschreibt den Kernprozess der Datenerfassung: Ein Messwert trifft vom DAS im System ein, wird normalisiert und verletzt einen definierten Grenzwert.

Ablauf:

1. Die Pipeline empfängt den Messwert und parst ihn in ein Standardformat.
2. Der Worker ruft die Metadaten (inkl. Umrechnungsformeln und Limits) für diesen Sensor aus dem Backend/Cache ab.
3. Der Messwert wird in die SI-Einheit umgerechnet.
4. Der Validator stellt eine Grenzwertverletzung fest. Der Wert wird dennoch in die TimeScaleDB geschrieben, erhält aber ein entsprechendes Flag (`too_high`, `too_low`, `ok`), um im Frontend farblich markiert werden zu können.
5. Die Pipeline emittiert einen Event (z. B. via Log-Metriken), der von Prometheus/Grafana erfasst wird.
6. Grafana triggert via Webhook den Alerting Service, welcher die Templates und Abonnenten via Backend lädt und den Alarm via AWS SNS versendet.

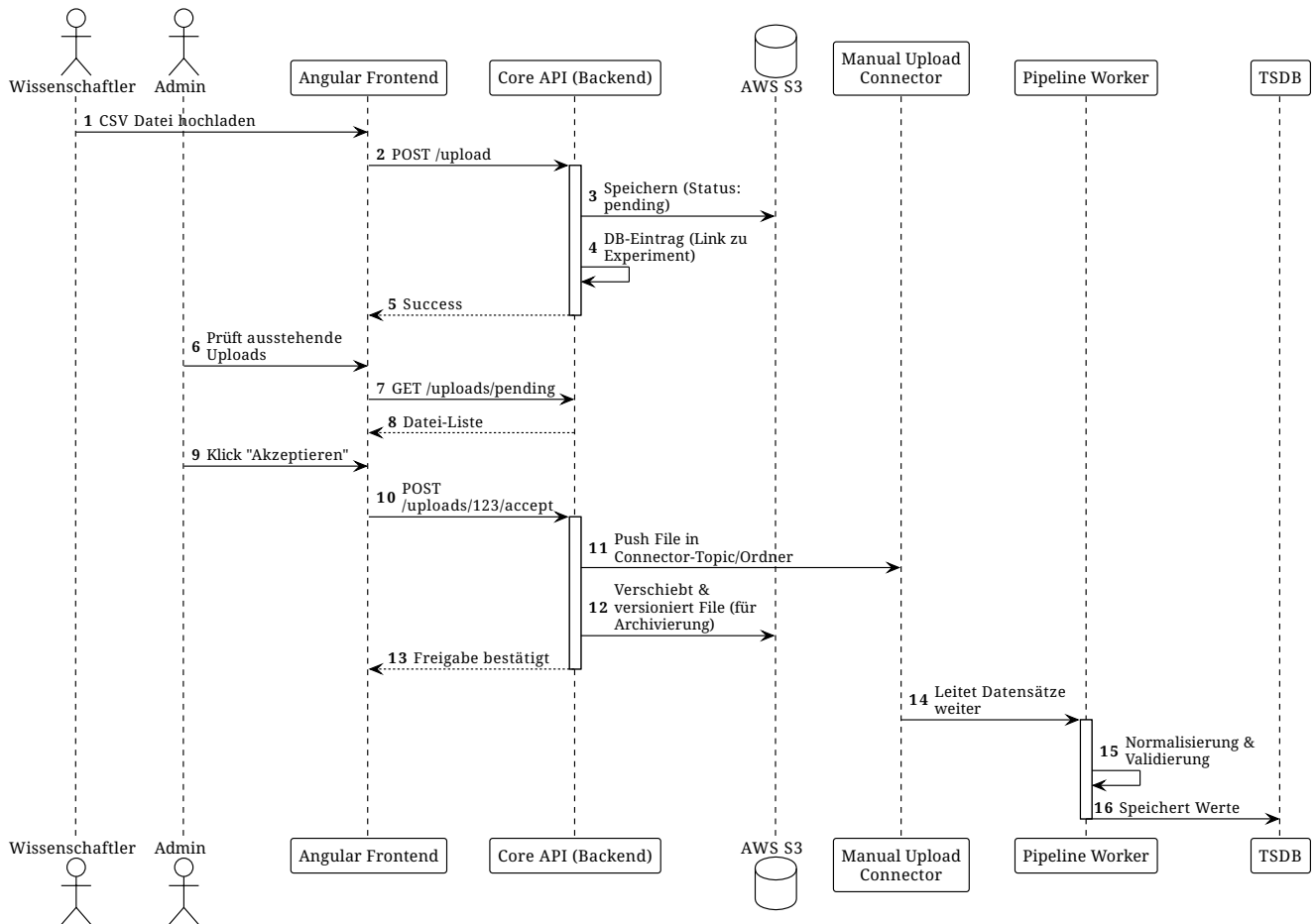


6.2. Szenario 2: Manueller Upload & Admin-Freigabeprozess

Nicht alle Daten fließen automatisch in das System. Historische Daten oder Zusatzmessungen werden von Wissenschaftlern per CSV hochgeladen. Um die Datenintegrität zu wahren, ist ein strikter Freigabeprozess vorgeschaltet.

Ablauf:

1. Der Wissenschaftler lädt eine CSV-Datei im Frontend hoch.
2. Das Backend speichert die Datei vorläufig in einem S3-Bucket und erstellt einen Datenbankeintrag (Status: **pending**).
3. Der Admin wird im UI benachrichtigt, lädt die Datei zur Prüfung herunter und kann sie bei Bedarf korrigiert neu hochladen.
4. Nach dem Klick auf "Akzeptieren" übergibt das Backend die Datei an den Manual Upload Connector der Pipeline.
5. Das Originalfile wird für die spätere Archivierung (AWS Glue) versioniert auf S3 verschoben.
6. Die Pipeline verarbeitet die Daten, als kämen sie von einem Live-Sensor.

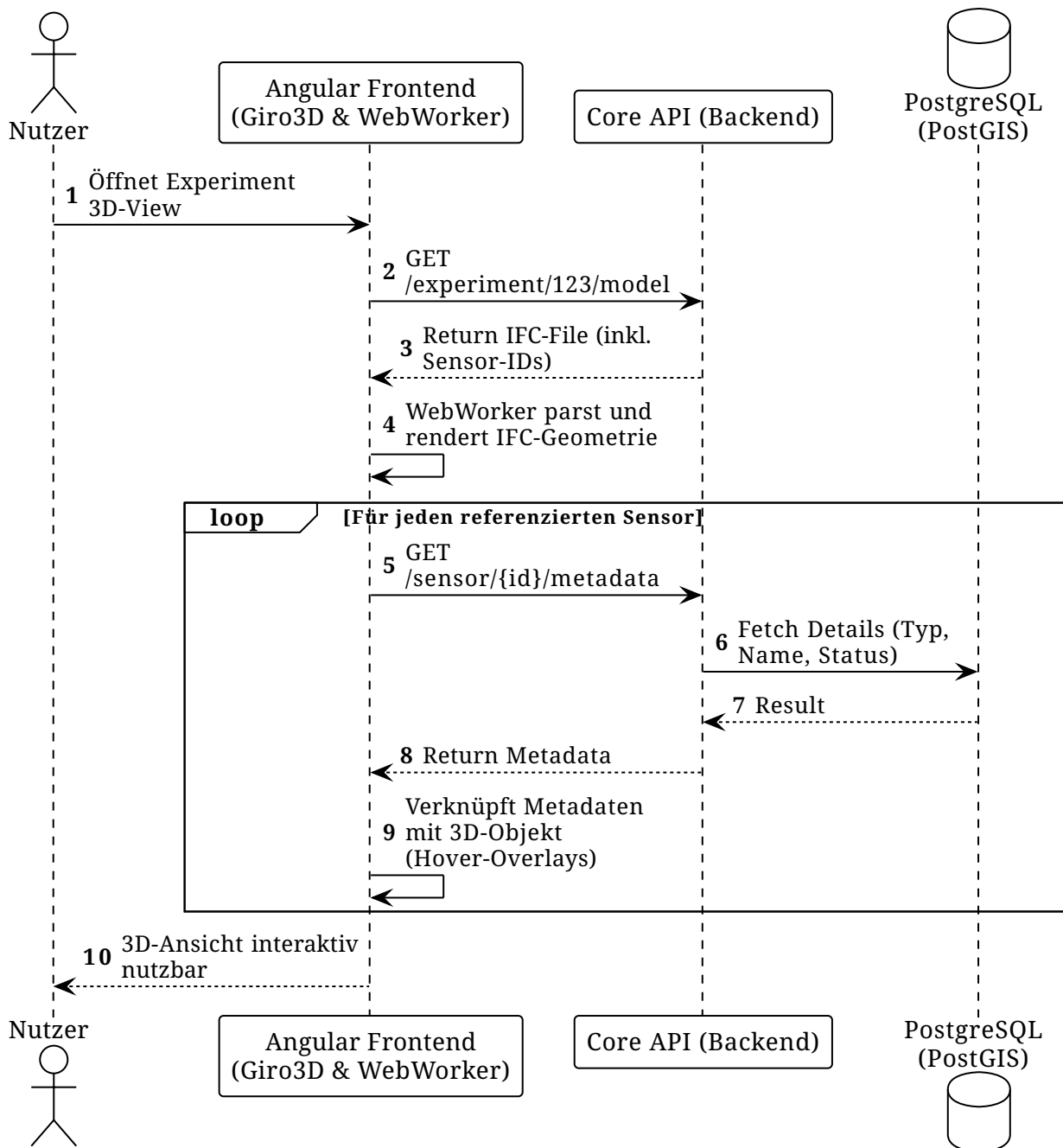


6.3. Szenario 3: Initialisierung des Digitalen Zwillings (3D View)

Der Digitale Zwilling ist ein Kern-Feature für die räumliche Orientierung im Felslabor. Die Komplexität des 3D-Renderings wird dabei architektonisch auf die Clients (Browser) ausgelagert, um das Backend zu schonen.

Ablauf:

1. Der Nutzer öffnet ein Experiment im Frontend.
2. Das Backend liefert das zugehörige 3D-Modell (IFC-File).
3. Das Frontend (**Giro3D**) lagert das ressourcenintensive Rendering an Web-Worker des Browsers aus.
4. Das IFC-Modell enthält die IDs oder Koordinaten der Sensoren. Das Frontend iteriert über diese IDs und lädt die Metadaten asynchron aus der PostGIS-Datenbank (via Backend) nach, um Overlays und Tooltips bei "Hover"-Aktionen darzustellen.



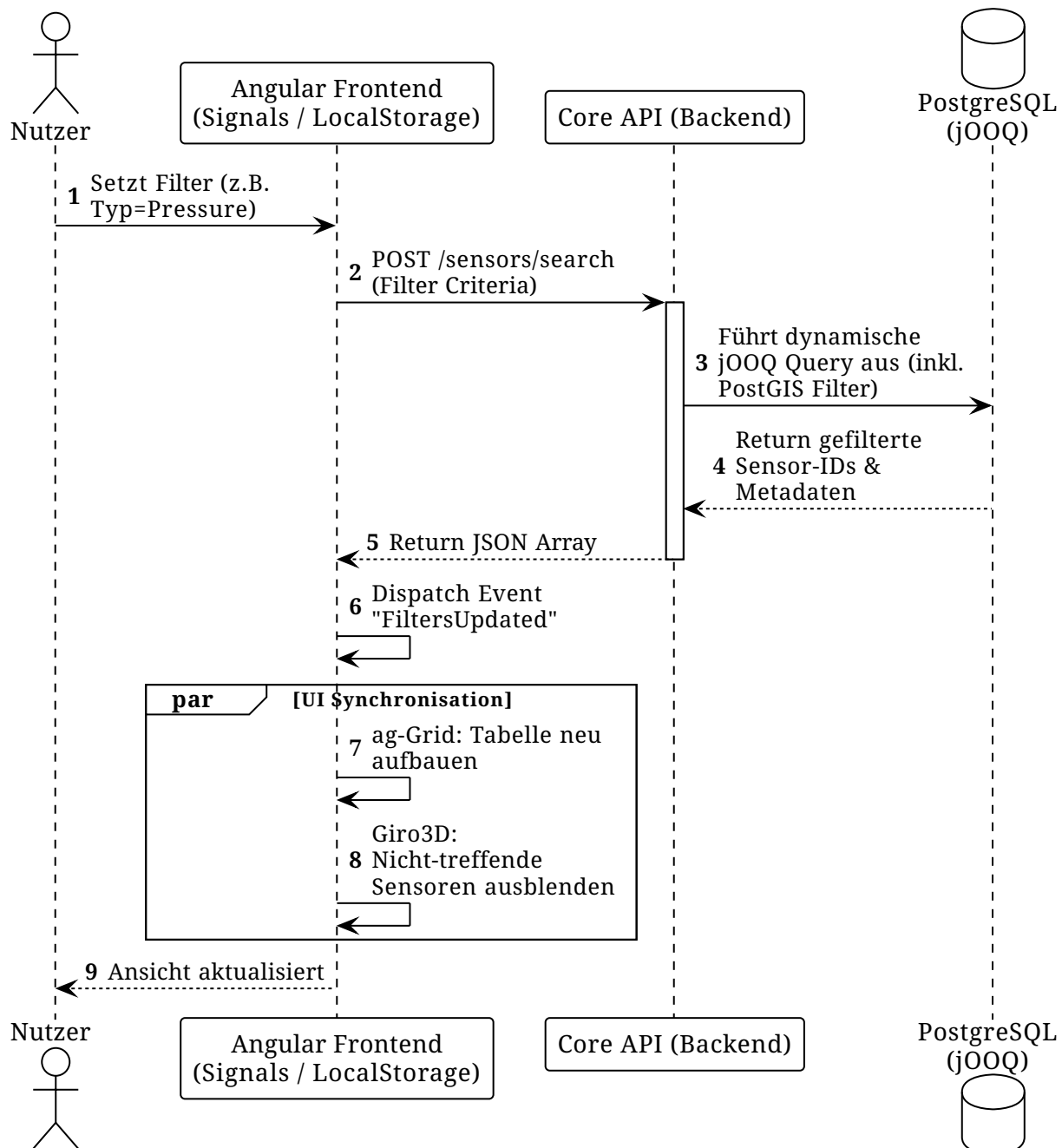
6.4. Szenario 4: Räumliches & Fachliches Filtern

Bei tausenden von Sensoren ist eine leistungsstarke Filterung essenziell. Die Filterkriterien wirken sich synchron auf die tabellarische Darstellung und den 3D-Viewer aus.

Ablauf:

- Der Nutzer definiert Filterkriterien (z. B. "Drucksensoren in Bohrloch X").
- Die Abfrage wird an das Backend gesendet, welches via `jQuery` dynamische Queries auf der Datenbank ausführt.
- Die gefilterte Liste der Sensor-IDs wird an das Frontend retourniert.
- Das Frontend nutzt Angular Signals und `localStorage`, um sowohl die Datentabelle (`ag-Grid`) als auch den Digitalen Zwilling (`Giro3D`) synchron zu aktualisieren – nicht relevante Sensoren

werden im 3D-Raum ausgeblendet.



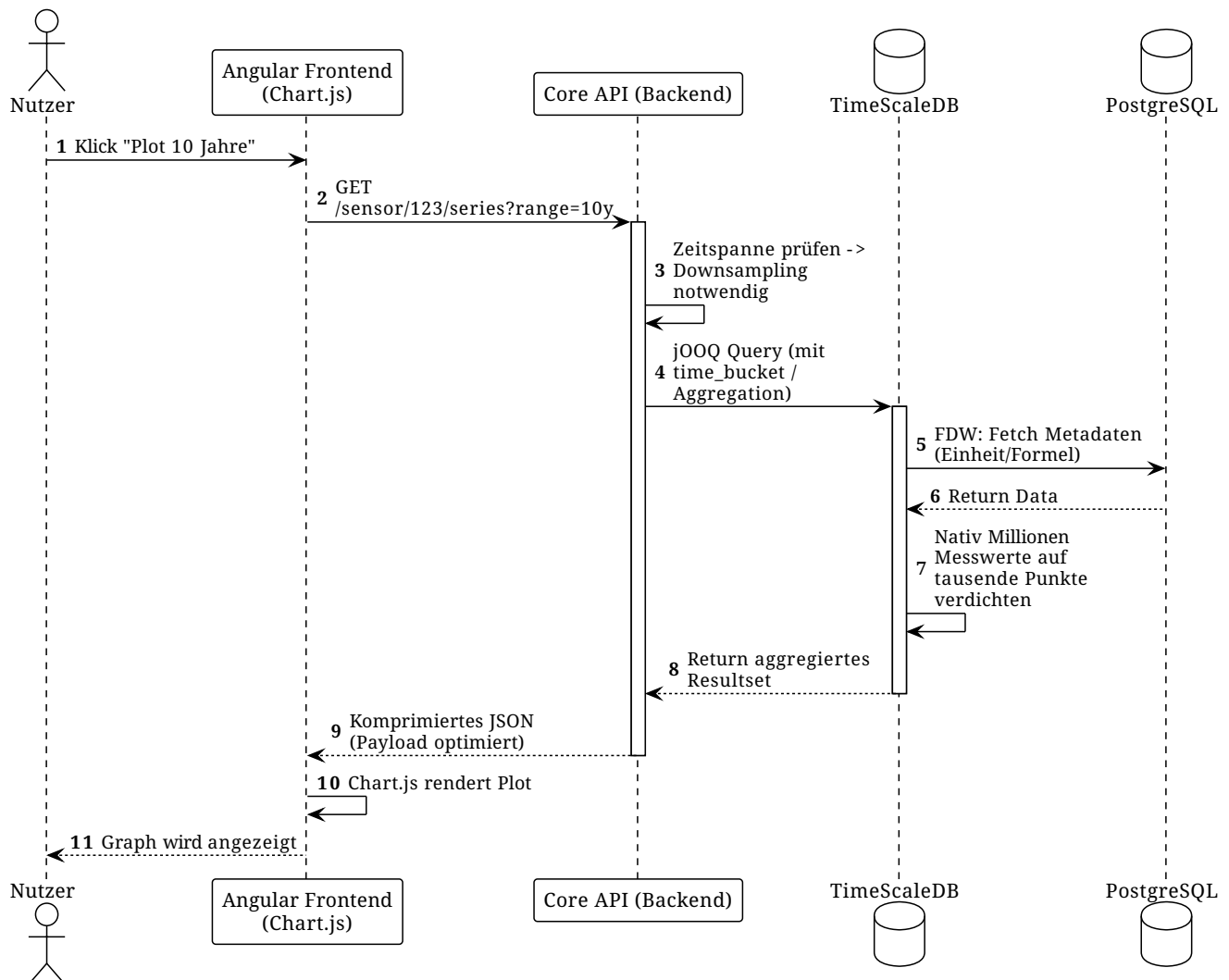
6.5. Szenario 5: Visualisierung langer Zeitreihen (FDW & Downsampling)

Das Abfragen von Zeitreihen, die Jahre zurückreichen, ist hochgradig datenintensiv. Hier kommt die Performance der **TimeScaleDB** in Kombination mit **postgres_fdw** und Downsampling zum Tragen.

Ablauf:

1. Der Nutzer wählt einen Sensor und einen grossen Zeitraum (z. B. 10 Jahre) für einen Plot (**Chart.js**) aus.
2. Das Backend prüft die Zeitspanne und leitet einen Downsampling-Request an die TimeScaleDB weiter. (Vordefinierte Buckets, z.B. Zeitspanne > 1 Jahr → Bucket=hourly)

3. Die TimeScaleDB aggregiert die Messwerte nativ (z. B. via `time_bucket`) und joined via `FDW` transparente notwendige Metadaten (wie Einheiten) aus der PostgreSQL.
4. Das Backend empfängt ein stark komprimiertes, aber visuell akkurates JSON-Array und liefert es an das Frontend aus.



6.6. Szenario 6: Authentifizierung, Security Context & Row-Level Security

Die Sicherheit und rollenbasierte Zugriffskontrolle ist eine Anforderung der Priorität 1. MONTEIS nutzt einen eigenen **Keycloak** als zentralen Identity Access Management (IAM) Server, welcher **AGOV** als Identity Provider (IdP) integriert. Das Spring Boot Backend fungiert als klassischer **Resource Server**. Leseberechtigungen (welche Experimente/Sensoren/Messwerte ein Nutzer sehen darf) werden nicht auf API-Ebene, sondern transparent auf Datenbankebene per **Row-Level Security** durchgesetzt.

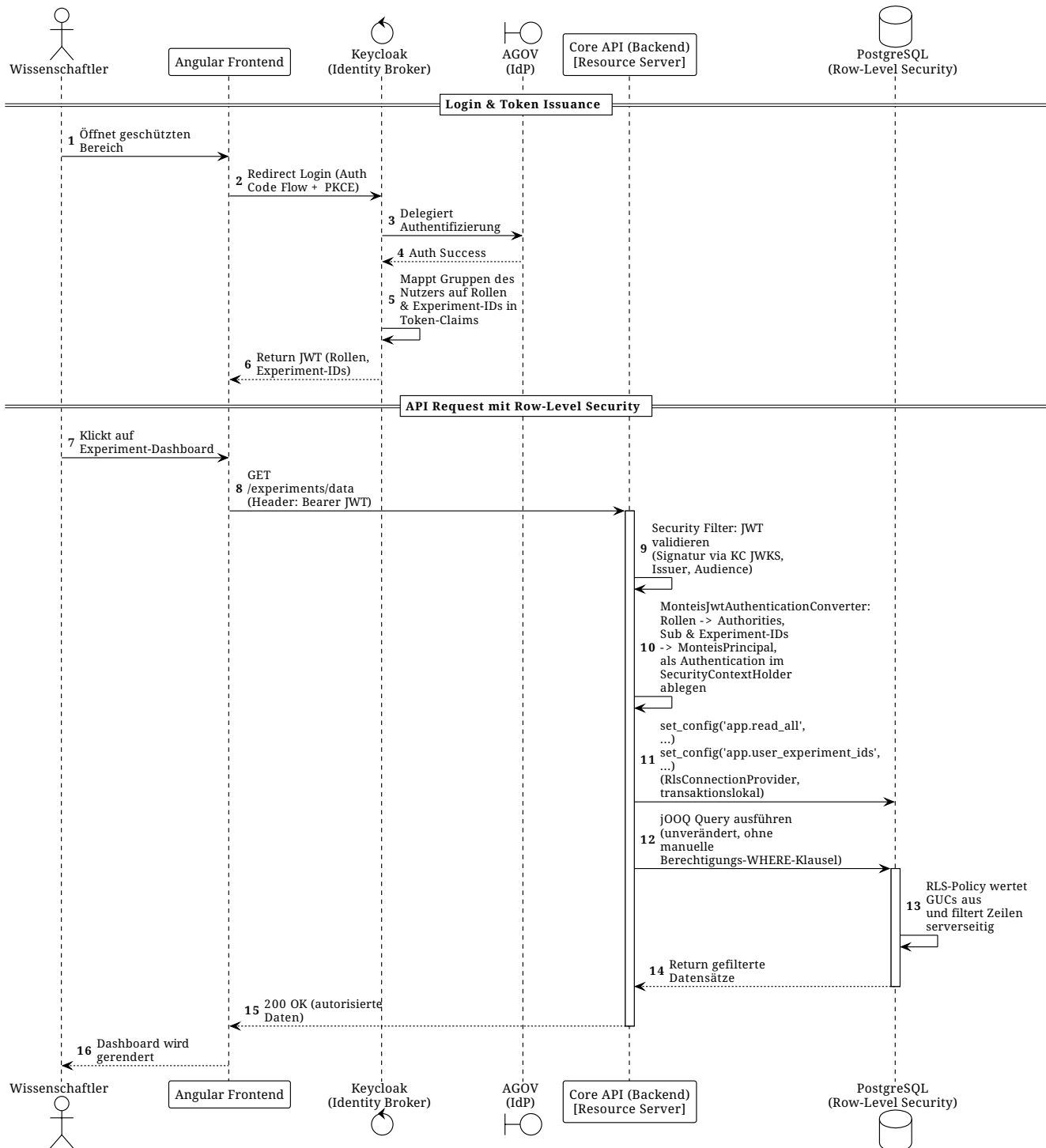
Ablauf:

1. Der Nutzer öffnet die Web-App und wird vom Angular Frontend an Keycloak weitergeleitet (OIDC Authorization Code Flow mit PKCE) und authentifiziert sich (delegiert an AGOV). Berechtigungen sind in Keycloak über Gruppenmitgliedschaft vergeben (z. B. eine Admin-Gruppe, eine Gruppe pro Experiment) statt direkt am Nutzer. Keycloak stellt ein JWT aus,

dessen Claims die Berechtigungen sowie die Experiment-Zugehörigkeiten des Nutzers enthalten.

2. Das Frontend sendet dieses JWT bei jedem REST-Call im **Authorization: Bearer**-Header mit.
3. Ein Security-Filter im Spring Boot Backend (OAuth2 Resource Server) fängt den Request ab, verifiziert Signatur, Aussteller und Audience des Tokens (via JWKS von Keycloak) und übergibt es an einen custom **Converter**, der daraus die vollständige **Authentication** baut und im Standard-**SecurityContextHolder** ablegt. Fehlt eine gültige Leseberechtigung, werden Experiment-IDs strikt auf "keine Rechte" verengt (**fail closed**), selbst wenn der Claim befüllt wäre.
4. Bei jeder jOOQ-Datenbankverbindung schreibt ein custom **ConnectionProvider** (**RlsConnectionProvider**) die aktuelle **Authentication** transaktionslokal als Postgres-GUCs auf die Connection.
5. Native **Row-Level-Security-Policies** auf **sensors/experiments** sowie eine **security_barrier**-View für die (per Foreign Data Wrapper eingebundenen) Sensormesswerte werten diese GUCs serverseitig aus und filtern Zeilen automatisch — der jOOQ-Code selbst enthält keine manuelle Berechtigungs-**WHERE**-Klausel. Admins sehen alles; reguläre Nutzer nur Daten aus Experimenten, deren Gruppe sie angehören.
6. Schreibende Requests (**POST/PUT/PATCH/DELETE**) werden unabhängig davon zentral im Filterchain (**SecurityConfig**) geprüft und erfordern eine Write-Berechtigung. Das Frontend fragt diese zusätzlich über **GET /api/me** ab, um schreibende UI-Elemente rein kosmetisch auszublenden — die eigentliche Durchsetzung bleibt serverseitig im Filterchain.

Die Experiment-Zugehörigkeit wird aktuell bewusst als JWT-Claim geführt, gespeist aus der Gruppenmitgliedschaft des Nutzers. Das ist einfach zu betreiben, solange die Zugehörigkeitsmengen klein bleiben — in der Praxis meist ein einzelnes Experiment pro Nutzer. Sollte das nicht mehr skalieren (viele Experimente pro Nutzer, häufig wechselnde Zuordnungen, oder schlicht Token-Größenlimits), ist die vorgesehene Ausweichlösung eine eigene **experiment_membership**-Tabelle (Schlüssel: **sub**), die der **MonteisJwtAuthenticationConverter** pro Request abfragt, statt die Zugehörigkeit aus dem Token zu lesen.



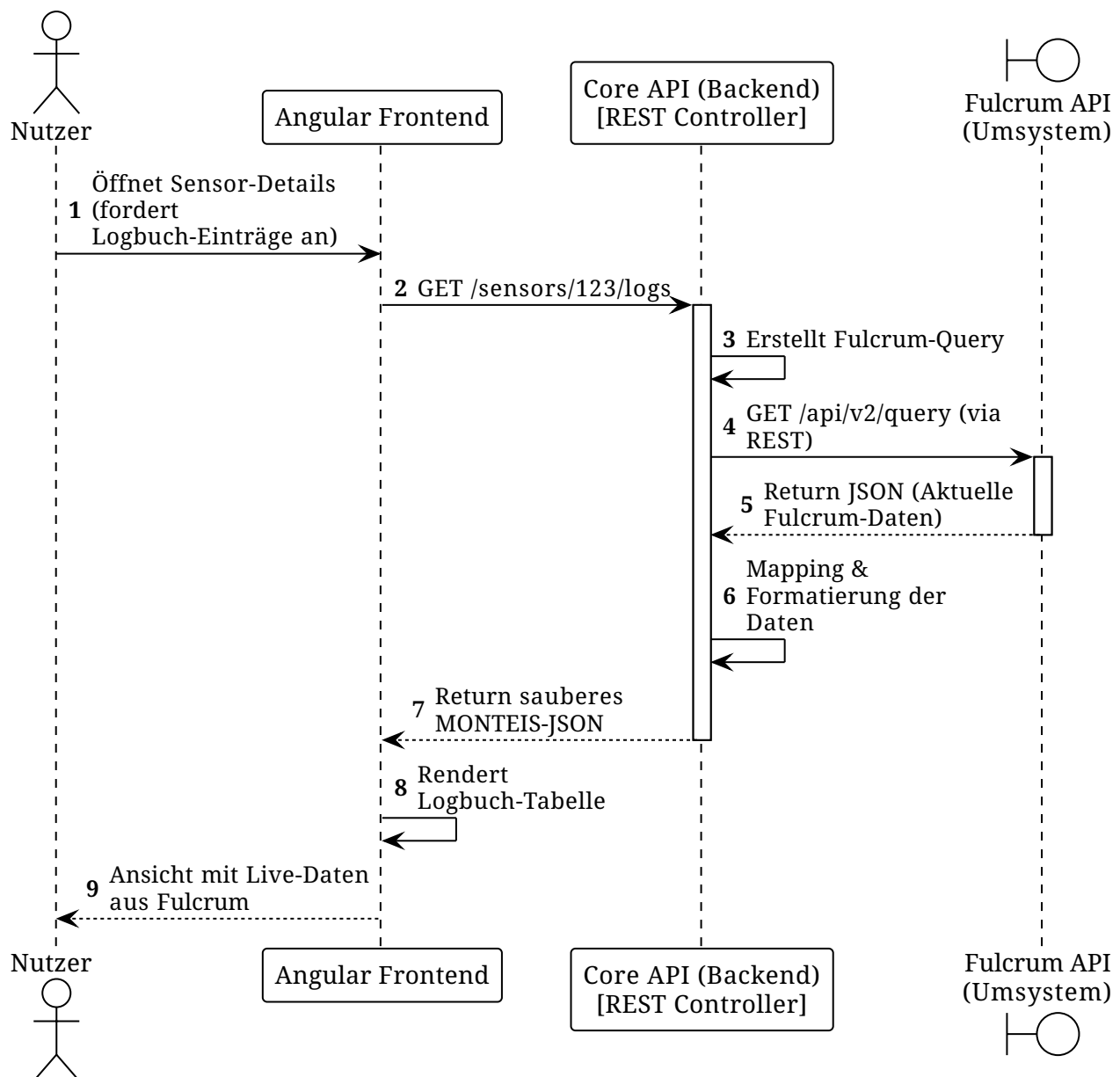
6.7. Szenario 7: On-Demand Datenabruf aus Umsystemen (Fulcrum Integration)

MONTEIS kommuniziert eng mit Umsystemen wie Fulcrum (dem digitalen Logbuch des Felslabors). Anstatt alle Log- und Asset-Daten asynchron und redundant in die MONTEIS-Datenbank zu spiegeln, wendet die Architektur für diese Daten das **On-Demand Fetching (Pull-Pattern)** an.

Das Spring Boot Backend fungiert hierbei als API-Gateway. Wenn externe Daten benötigt werden, werden diese "Just in Time" via REST abgerufen.

Ablauf:

1. Der Nutzer öffnet in der Angular Web-App die Detailansicht eines Experiments oder Sensors und fordert die aktuellen Logbuch-Einträge an.
2. Das Frontend sendet einen Request an die MONTEIS Core API.
3. Der REST Controller im Spring Boot Backend fängt den Request ab und formuliert eine synchrone Abfrage (z.B. via Fulcrum Query API) an das Umsystem Fulcrum.
4. Fulcrum retourniert die tagesaktuellen Log-Daten.
5. Die Core API mappt/formatiert die Daten in das interne MONTEIS-Format und liefert sie an das Frontend aus.
6. Der Nutzer sieht die absoluten Live-Daten aus Fulcrum, ohne dass diese physisch in der PostgreSQL von MONTEIS gespeichert werden mussten.



6.8. Szenario 8: TimeScaleDB Daten-Lebenszyklus (Downsampling & Kompression)

Um die enorme Menge an Messwerten (Ziel: 200 Milliarden) performant abfragbar zu halten und gleichzeitig die Speicherkosten der Datenbank zu minimieren, durchlaufen die Zeitreihendaten in der TimeScaleDB einen streng definierten, automatisierten Lebenszyklus.

Dieser Prozess nutzt die nativen TimeScaleDB-Features **Continuous Aggregates** (für das Downsampling) und **Hypertable Compression** (für die Reduktion des Speicher-Overheads).

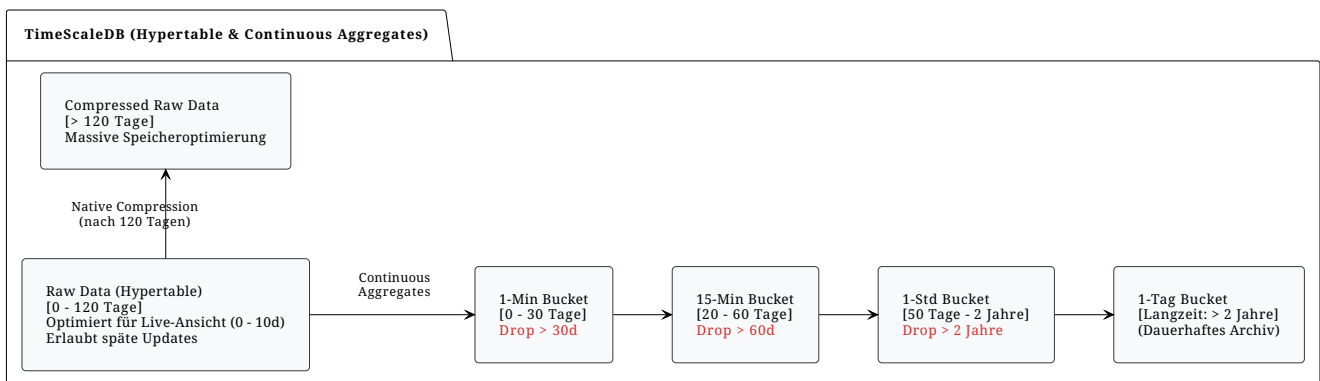
Die Aggregations- und Retentions-Regeln im Detail:

- **Raw Data (Hypertable):** Rohdaten werden primär für die Live-Ansicht (0 - 10 Tage) genutzt. Sie bleiben für 120 Tage unkomprimiert, um Nachlieferungen oder Updates von Messwerten problemlos zuzulassen. Nach 120 Tagen greift die native Kompression der TimeScaleDB (updates sind immer noch möglich aber weniger performant).
- **Continuous Aggregates (Downsampling):** Die Rohdaten werden im Hintergrund in materialisierte Views (Buckets) aggregiert.
- **Rolling Windows (Retention Policies):** Um ein unendliches Wachstum der fein granularen Views zu verhindern, werden alte Daten-Chunks in den Buckets automatisch gelöscht (Drop), sobald sie aus ihrem definierten Zeitfenster fallen.

Definierte Buckets & Zeitfenster:

- **1-Minuten Bucket:** Deckt die letzten 30 Tage ab. (Daten > 30 Tage werden verworfen).
- **15-Minuten Bucket:** Deckt den Zeitraum von Tag 20 bis Tag 60 ab. (Daten > 60 Tage werden verworfen).
- **1-Stunden Bucket:** Deckt den Zeitraum von Tag 50 bis 2 Jahre ab. (Daten > 2 Jahre werden verworfen).
- **1-Tages Bucket:** Deckt den Zeitraum ab 2 Jahren für die Langzeitspeicherung ab.

TimeScaleDB - Data Lifecycle, Downsampling & Retention



Neben der Datenbank-Optimierung verlässt sich MONTEIS auf zeitgesteuerte Hintergrundprozesse (Cron/Batch-Jobs), um die Speicherkosten für unveränderte Originaldaten zu minimieren. Um die rohen Originaldaten physisch zu sichern (für Compliance, Backups oder spätere Reprozessierung),

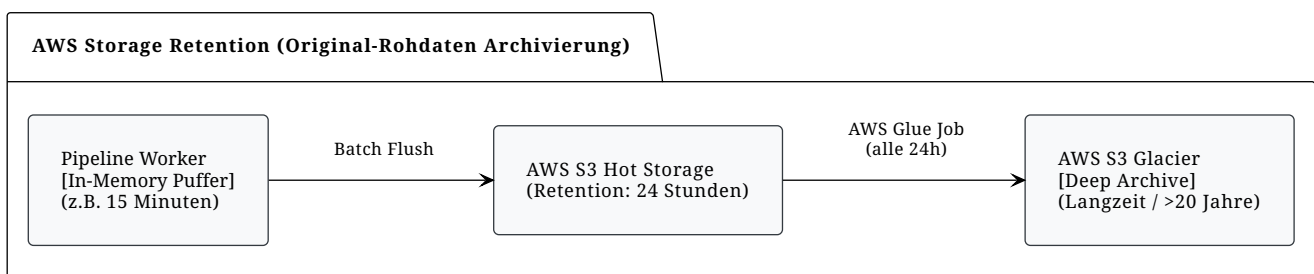
fließen diese parallel zur TimeScaleDB in einen Cloud-Objektspeicher.

Ablauf der Archivierung:

1. **Pufferung im Hot Storage:** Die Pipeline-Worker halten eine Kopie der Originaldatensätze kurzzeitig im Arbeitsspeicher und flushen diese regelmässig (z. B. alle 15 Minuten) als Batch-Files in einen regulären AWS S3 Bucket ("Hot Storage").
2. **AWS S3 Glacier Archiving (Cold Storage):** Mittels [AWS S3 Lifecycle Rules](#) werden die verteilten, temporären Rohdaten-Files aus dem "Hot Storage" (AWS S3 Standard Tier) in den weitaus günstigeren "Cold Storage" (AWS S3 Glacier) für die langfristige Speicherung verschoben. Die Daten werden semi-strukturiert (z. B. pro Sensor und Tag) abgelegt.

Hinweis: Die Daten im Glacier dienen rein der Notfall-Wiederherstellung oder historischen Nachprüfungen und werden vom regulären Frontend nicht abgefragt.

MONTEIS - Cloud Storage



6.9. Datenkonsistenz & Unabhängiges Routing (Kafka Consumer Groups)

6.9.1. Die Anforderung (Raw vs. Transformed)

Es muss immer gewährleistet sein, dass die Daten korrekt wiederhergestellt werden können und keine Daten verloren gehen. Aus diesem Grund stellen zwei unabhängige Flows sicher, dass die Original-Rohdaten in S3 verschoben und die transformierten und validierten Daten in die TimeScaleDB geschrieben werden.

6.9.2. Die Lösung: Fan-Out am Raw-Topic

Da Kafka als persistentes Logbuch (Append-Only Log) agiert, können mehrere Abonnenten dieselbe Nachricht unabhängig voneinander lesen. Der Datenstrom wird direkt am Eingangs-Topic aufgespalten:

1. **Storage Archiver (Raw Data Path):** Ein nativer Kafka Connect Sink (oder der POC Worker) lauscht auf den Raw-Topic mit einer eigenen Consumer Group (**s3-archive**). Er puffert die Rohdaten und schiebt sie periodisch in den **AWS S3 (Hot Storage)**. Von dort greifen automatisierte AWS Lifecycle Rules, die Daten nach einer definierten Zeit in den **AWS Glacier (Cold Storage)** zur Langzeitarchivierung verschieben.
 - **Im POC-Stadium:** Der Pipeline-Pod übernimmt diese Aufgabe vorübergehend mit. Eine

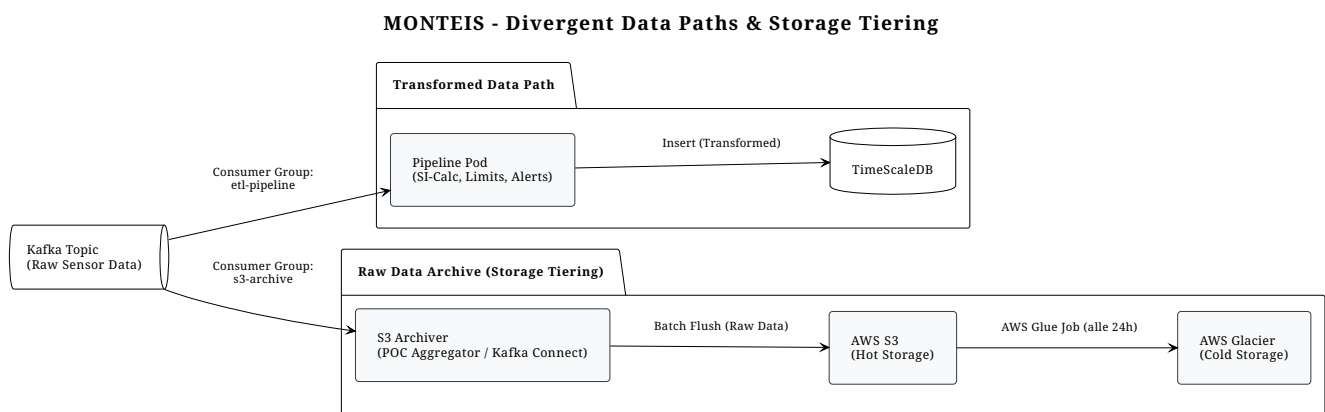
separate asynchrone Routine im Pod puffert die Daten ([Pipeline Pod - Stateful Polling Konzept](#)), formatiert sie im Hive-Partitioning-Stil (`sensor_id/year/month`) und lädt sie periodisch nach S3 hoch.

- **Final Solution (Kafka Connect):** Der S3-Upload-Code wird aus dem Pipeline-Pod entfernt. Stattdessen übernimmt ein nativer **Confluent S3 Sink Connector** diese Aufgabe. Da der POC die Datei- und Ordnerstrukturen von Kafka Connect exakt nachgeahmt hat, verläuft der Wechsel für nachgelagerte Systeme absolut transparent.

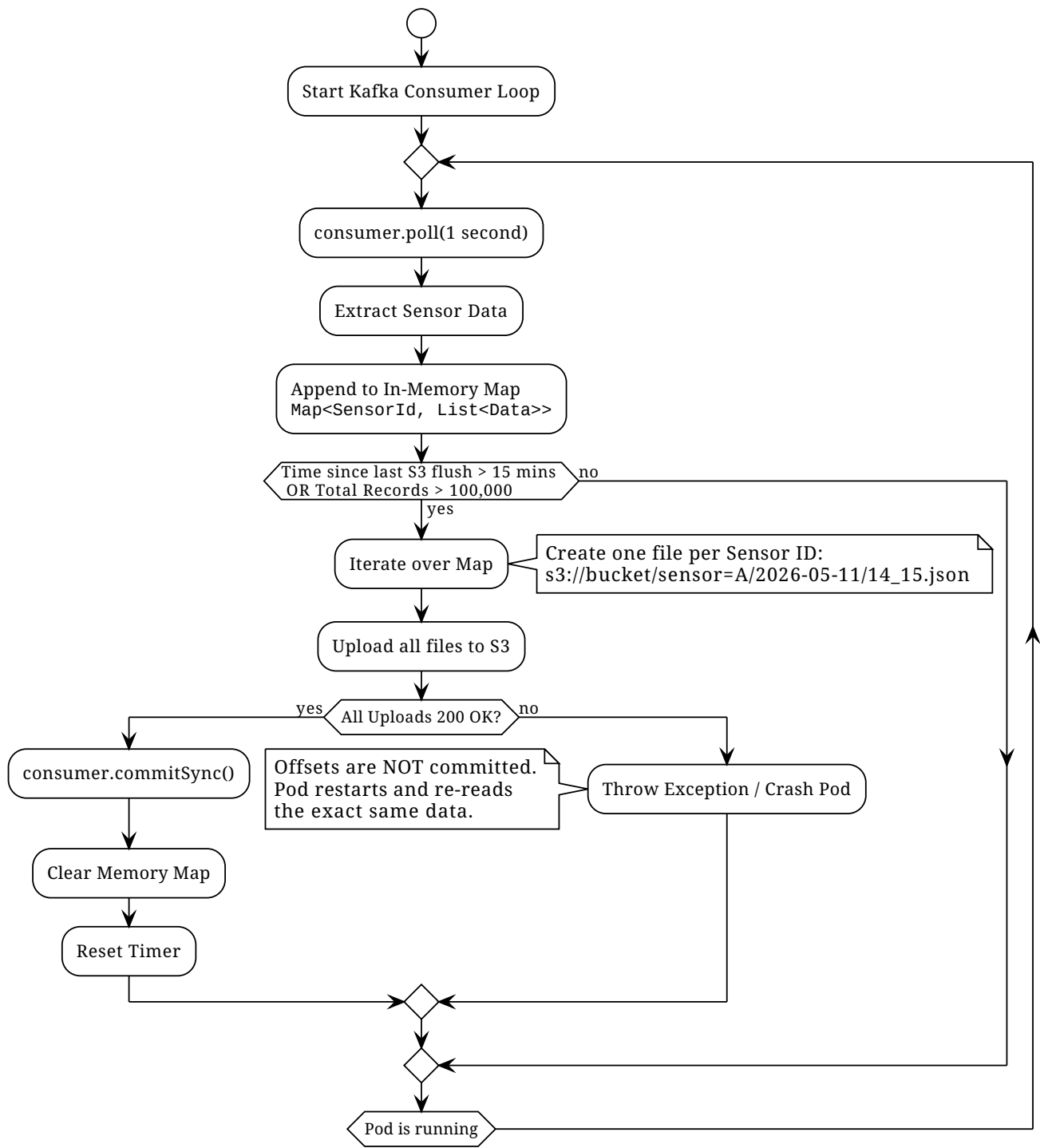
2. **Pipeline Pod (Transformed Data Path):** Der Pipeline-Worker lauscht auf denselben Raw-Topic mit einer völlig unabhängigen Consumer Group (`etl-pipeline`). Er transformiert die Daten, prüft Grenzwerte, triggert Alerts und speichert das finale Resultat in die **TimescaleDB**.

Fehlertoleranz durch getrennte Offsets: Kafka verwaltet den Fortschritt (Offset) für jede Consumer Group separat. Wenn der S3-Upload erfolgreich ist, die TimescaleDB jedoch offline geht, rückt nur das "Lesezeichen" der S3-Gruppe vor. Die Pipeline-Gruppe committet ihren Offset nicht. Nach Behebung des Datenbankausfalls wird der Pipeline-Pod gezwungen, die fehlgeschlagene Nachricht erneut zu verarbeiten. Es geht kein Wert verloren.

6.9.3. Architektur-Übersicht



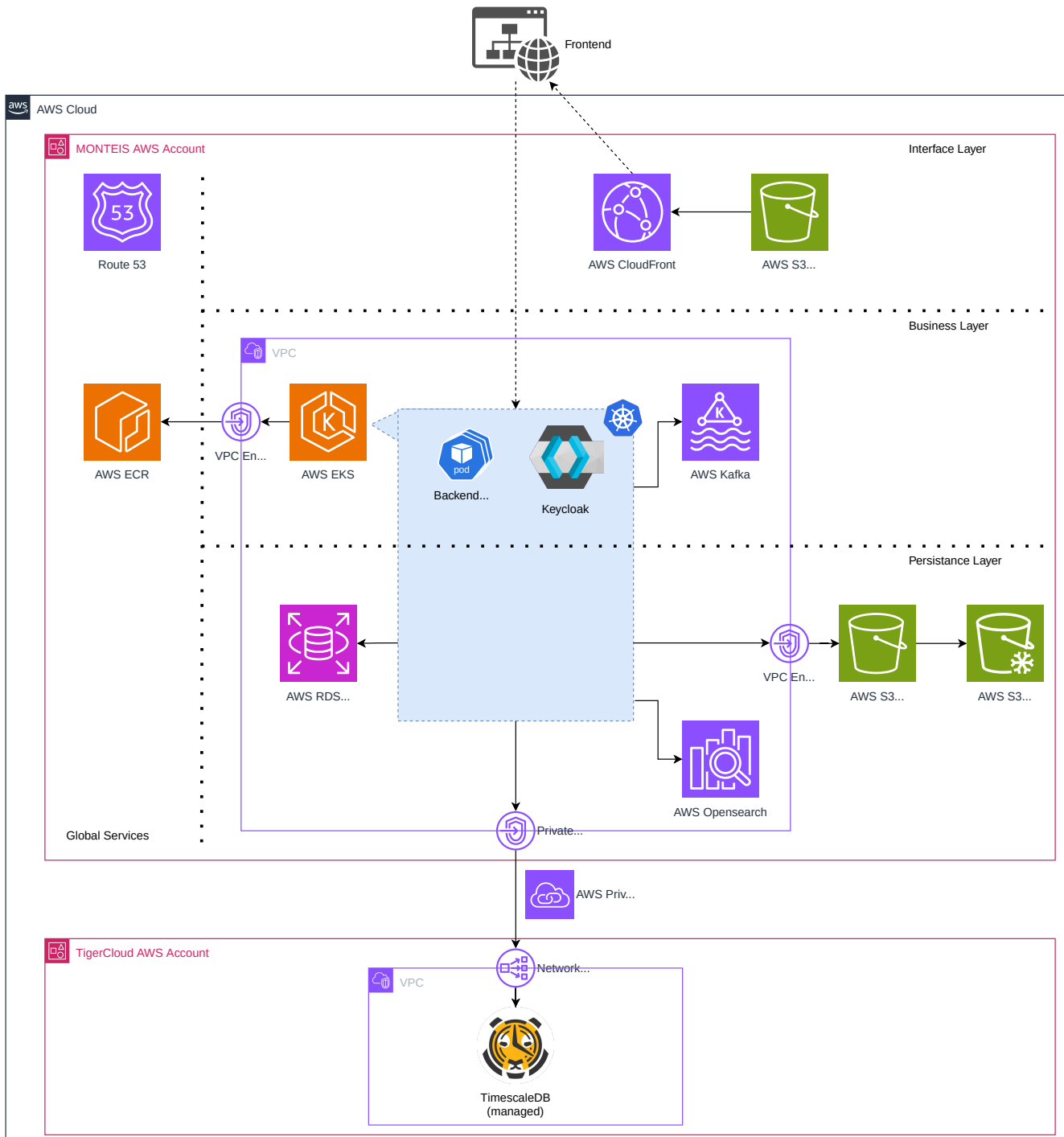
6.9.4. Pipeline Pod - Stateful Polling Konzept



7. Verteilungssicht

7.1. Infrastruktur Ebene 1: AWS Cloud Ressourcen

Das nachfolgende Diagramm zeigt die wichtigsten AWS Ressourcen (Dienste) für das MVP auf. Die Ressourcen sind logisch gruppiert in die drei Schichten: "Interface Layer", "Business Layer" und "Persistence Layer". Die Gruppierung "Global Services" dient der Verortung von global (= Layer-unabhängig) genutzten AWS Diensten. Zur besseren Einordnung ist zusätzlich das Kubernetes Cluster, bereitgestellt durch AWS EKS, angedeutet, um den Dienst "Keycloak" als zentralen Access- und Identity-Dienst einzuordnen.



7.1.1. Beschreibung AWS Cloud Ressourcen

Layer	Service	Aufgabe	Notiz
Global	Route53	Zentraler AWS DNS Service	
Global	AWS ECR	Container Registry für Build-Artefakte (z.B. Container Images)	
Interface	AWS CloudFront	Content-Delivery-Network (CDN) zur Bereitstellung Frontend	
Interface	AWS S3	Hosting Frontend Artefakte	
Business	AWS EKS	Managed Kubernetes Cluster	
Business	AWS Kafka	Managed Kafka Cluster	
Business	Keycloak	Open-Source Identity Provider & User-Management	
Business	VPC Endpoint	Sichere Netzwerk-Verbindung zwischen VPC und globalen AWS Services über privates AWS Netzwerk	Für MVP empfehlenswert Vorteile: Nutzung AWS-internes Netzwerk (kein öffentliches Internet!) + Kostenreduktion Traffic (vs. NAT Gateway) Ref: https://pcg.io/insights/vpc-endpoints-explanation-and-cost-comparison/
Business / Persistence	VPC	Virtual Private Cloud, dedizierter Netzwerkbereich für Compute und Storage Ressourcen	

Layer	Service	Aufgabe	Notiz
Persistence	AWS RDS for PostgreSQL	Managed Datenbank für Metadaten	
Persistence	TimescaleDB	Datenbank für Zeitreihen-Daten	
Persistence	AWS S3	Object-Storage (hot & cold)	
Persistence	AWS OpenSearch	Service für Indexierung und Volltext-Suche	
Persistence	AWS PrivateLink	Sichere Netzwerk-Verbindung zwischen zwei VPCs über privates AWS Netzwerk	Für MVP empfehlenswert https://docs.aws.amazon.com/whitepapers/latest/aws-vpc-connectivity-options/aws-privatelink.html https://www.tigerdata.com/docs/deploy/tiger-cloud/tiger-cloud-aws/security/vpc

8. Querschnittliche Konzepte

8.1. Architekturmuster: Domain-Driven Design & CQRS-Lite (Read/Write Flows)

Die Architektur der Applikation unterscheidet strikt zwischen schreibenden Operationen (Zustandsänderungen) und reinen Lese-Operationen (Datenaggregation und Views). Dieses Muster orientiert sich am CQRS-Prinzip (Command Query Responsibility Segregation) kombiniert mit Domain-Driven Design (DDD).

Diese Trennung verhindert "Frankenstein"-Domänenmodelle, maximiert die Performance bei Lesezugriffen und schützt komplexe Geschäftsregeln bei Schreibzugriffen.

8.1.1. Der Write-Flow (Commands & Mutationen)

Gilt für alle Endpunkte, die den Systemzustand verändern (POST, PUT, PATCH, DELETE).

Hierbei wird ein striktes Domain-Driven Design erzwungen. Das Ziel ist der Schutz der Geschäftslogik ("Always-Valid Domain Model").

- **Ablauf:** Web Request → Controller → WriteDto → Mapper → Domain Entity → Service → Repository (jOOQ) → mapper → Datenbank.
- **Regeln:**
 - DTOs werden vom Mapper in Rich-Domain-Objekte (z.B. Sensor, Formula) übersetzt.
 - Validierungsregeln und Business-Logik leben ausschliesslich in der Domäne.
 - Die Services und Repositories arbeiten ausschliesslich mit den Domänen-Objekten, niemals mit DTOs.
 - Repositories sind für das Mapping von der Domäne in die Datenbankstruktur (via jOOQ) zuständig.

8.1.2. Der Read-Only Flow (Queries & Dashboards)

Gilt für Endpunkte, die rein lesend auf aggregierte Daten zugreifen (z.B. das overview Modul, die Experiment-Detailansicht oder Tabellen-Ansichten über mehrere Fachbereiche hinweg).

Hierbei wird die Domänenschicht **bewusst übersprungen**. Da bei reinen Leseoperationen kein Zustand geschützt werden muss, wäre die Instanziierung von Domänenobjekten reine Ressourcenverschwendung.

Es werden zwei Ablauf-Varianten unterschieden, abhängig davon, ob zusätzliche Logik zwischen Controller und Datenbank nötig ist:

- **Direkter Ablauf (Standardfall):** Web Request → Controller → QueryRepository (jOOQ) → ReadDto.
 - Gilt, wenn der Endpunkt eine einzelne Query 1:1 durchreicht, ohne zusätzliche

Orchestrierung, Aggregation über mehrere Repositories oder fachliche Nachbearbeitung.

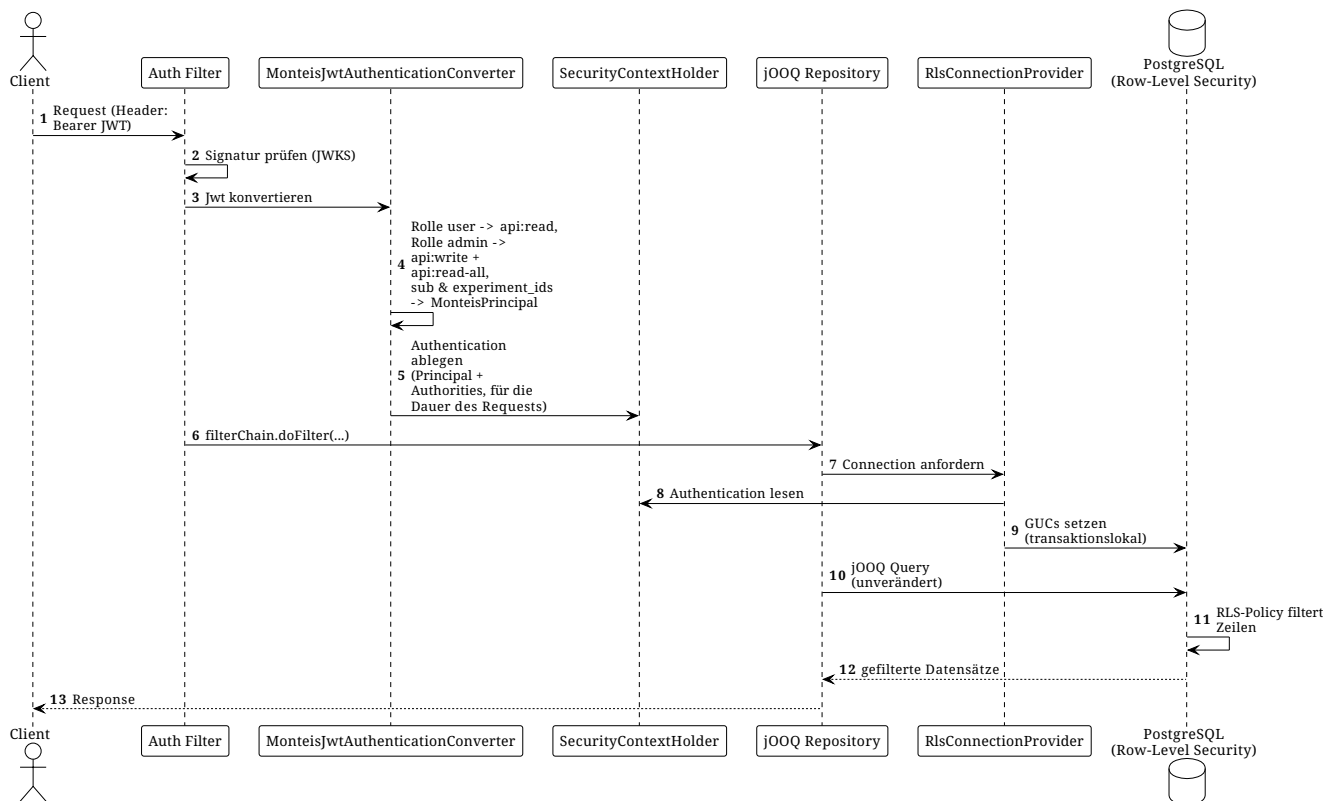
- Der Controller injiziert das `QueryRepository`-Interface direkt und ruft es auf. Ein Service, der nur `return queryRepository.find(...)` macht, ist reine Indirektion ohne Mehrwert und wird bewusst weggelassen.
- Beispiele: `SensorController#findAllFormulas` (nutzt `SensorQuery` direkt) und `ExperimentController#getMetrics` (nutzt `ExperimentQueryInterface` direkt).
- **Ablauf mit Service (Ausnahmefall):** Web Request → Controller → Service → QueryRepository (jOOQ) → ReadDto.
 - Nur nötig, wenn der Read-Endpoint echte Logik erfordert, die nicht ins Repository gehört, z.B. das Kombinieren mehrerer Query-Aufrufe, fachliche Berechnungen auf den gelesenen Daten oder Orchestrierung über mehrere Module hinweg.
 - Sobald ein solcher Service auf reine Delegation ohne jegliche Logik schrumpft, muss er entfernt und der Controller direkt auf das `QueryRepository` umgestellt werden.
- **Regeln (für beide Varianten):**
 - Es werden **keine** Domänen-Objekte (`Entities`) verwendet.
 - Das Repository nutzt die volle Stärke von jOOQ, um hochoptimierte SQL-Joins (z.B. über Sensoren, Experimente und Boreholes) auszuführen.
 - Module, die dieses Pattern nutzen, verwenden den Suffix `QueryRepository` (statt `Repository`), um visuell zu signalisieren, dass hier kein Aggregate-Root zurückgegeben wird.
 - Die Sicherheitsfilterung (Row-Level-Security) erfolgt unabhängig von dieser Entscheidung immer auf Datenbankebene über native Postgres-RLS-Policies (siehe Abschnitt "Core-API Datenzugriff & Sicherheit"), nie im Service oder Repository. Das Weglassen des Service hat daher keinen Einfluss auf die Zugriffskontrolle.

8.1.3. Zusammenfassung der Vorteile

- **Performance:** Native SQL-Joins direkt in DTOs ohne den Overhead von Objekt-Instanziierungen im Java-Speicher.
- **Saubere Domäne:** Fachobjekte wie `Sensor` bleiben fokussiert auf ihre eigenen Business-Regeln und werden nicht mit UI-spezifischen Feldern (z.B. `experimentName`) verschmutzt.

8.2. Core-API Datenzugriff & Sicherheit

Berechtigungsprüfungen finden nie im Service statt, sondern ausschliesslich an den Rändern des Systems (Filter) und in der Datenbank (Row-Level Security).



- **Security Context (`SecurityContextHolder`):** Ein Auth-Filter validiert das JWT (Signatur, Aussteller, Audience) und übergibt es an `MonteiJwtAuthenticationConverter`, der daraus die Berechtigungen sowie Sub und Experiment-IDs als `MonteiPrincipal` ableitet und beides als `Authentication` für die Dauer des Requests im Standard-`SecurityContextHolder` von Spring Security ablegt — kein eigener Propagierungsmechanismus. Fehlt eine gültige Leseberechtigung, werden Experiment-IDs strikt auf leer verengt (**fail closed**). Hintergrundjobs ohne Request (z. B. der Startup-Audit-Backfill) binden stattdessen explizit einen `SystemSecurityContext` mit Admin-Rechten.
- **Rollenvergabe via Keycloak-Gruppen:** Berechtigungen werden Nutzern nie direkt zugewiesen, sondern über Gruppenmitgliedschaft (Admin-Gruppe, Read-All-Gruppe, eine Gruppe pro Experiment). Details zu den konkreten Rollen/Claims siehe Keycloak-Realm-Config und `MonteiJwtAuthenticationConverter`.
- **Leseberechtigungen (Row-Level Security):** `RlsConnectionProvider` schreibt die aktuelle `Authentication` bei jeder Connection-Beschaffung transaktionslokal auf die Verbindung. Native RLS-Policies auf `sensors/experiments` sowie eine security-barrier-View für die per FDW eingebundene TimescaleDB-Tabelle filtern Zeilen serverseitig danach — der jOOQ-Code selbst enthält keine Berechtigungslogik. Weil der Kontext transaktionslokal ist, muss jede RLS-relevante Repository-Methode innerhalb einer Spring-Transaktion laufen (`@Transactional(readOnly = true)`).
- **Schreibberechtigungen (RBAC):** Alle DB-Zugriffe laufen über dieselbe Rolle (`core_app`), RLS kann also keine nutzerspezifischen Schreibrechte abbilden. Schreibzugriffe bleiben auf DB-Ebene permissiv; die Autorisierung erfolgt stattdessen zentral im Spring-Security-Filterchain (`SecurityConfig`) anhand der HTTP-Methode: `POST/PUT/PATCH/DELETE` erfordern eine Write-Berechtigung, `GET` bleibt für alle authentifizierten Nutzer offen — siehe ADR in Kap. 9. Das Frontend fragt diese Berechtigung über `GET /api/me` ab und blendet Schreib-Buttons rein kosmetisch aus, wenn sie fehlt.

- **Frontend-seitige Sichtbarkeit (rein kosmetisch):** Der Endpunkt `GET /api/me` (`UserController`) spiegelt lediglich wider, ob die aktuelle `Authentication` die `api:write`-Authority besitzt (`CurrentUserDto#canWrite`). Das Angular-Frontend fragt dies über den `PermissionsService` beim App-Start einmalig ab und blendet darüber z. B. "Sensor erstellen/bearbeiten"-Buttons für Nutzer ohne Schreibrecht aus. Das ist ausschliesslich UX-Komfort — die eigentliche Durchsetzung bleibt vollständig serverseitig im Filterchain (siehe oben); ein Bypass dieser UI-Ausblendung führt zu keinem unautorisierten Zugriff.

8.3. Core-API Optimistic Locking und Versionierung/Auditierung

8.3.1. Technischer Hintergrund: Optimistic Locking

Im Gegensatz zum Pessimistic Locking (bei dem Datensätze explizit auf Datenbankebene über `SELECT ... FOR UPDATE` gesperrt werden), geht das Optimistic Locking davon aus, dass parallele Schreibzugriffe auf denselben Datensatz selten sind. Die Konflikterkennung wird bis zum eigentlichen Schreibvorgang verzögert.

Der Mechanismus: Die Tabelle erhält eine zusätzliche Spalte (z.B. `VERSION` als Integer oder einen Zeitstempel).

Der Lesezugriff: Beim Laden eines Datensatzes wird diese Version mitgelesen.

Der Schreibzugriff: Beim Speichern wird ein `UPDATE` abgesetzt, das die alte Version in der `WHERE`-Klausel voraussetzt und die Version im `SET`-Block inkrementiert: `UPDATE tabelle SET daten = 'neu', version = 2 WHERE id = 123 AND version = 1;`

Die Prüfung: Die Datenbank meldet zurück, wie viele Zeilen aktualisiert wurden (Rows Affected). Ist dieser Wert 0, bedeutet dies, dass ein anderer Prozess den Datensatz in der Zwischenzeit modifiziert hat. In diesem Fall wirft der ORM oder Query-Builder (hier jOOQ) eine Exception (z.B. `DataChangedException`), was in der Regel zu einem Rollback der Datenbanktransaktion führt.

8.3.2. Architektur und Trennung der Zuständigkeiten

Die gewählte Schichtenarchitektur trennt das Concurrency Management (jOOQ) strikt vom Audit-Logging (JaVers).

REST-Schicht (DTOs): Dient als Ein- und Ausgangspunkt. Das DTO transportiert die Nutzdaten inklusive der vom Client zuletzt gelesenen Version.

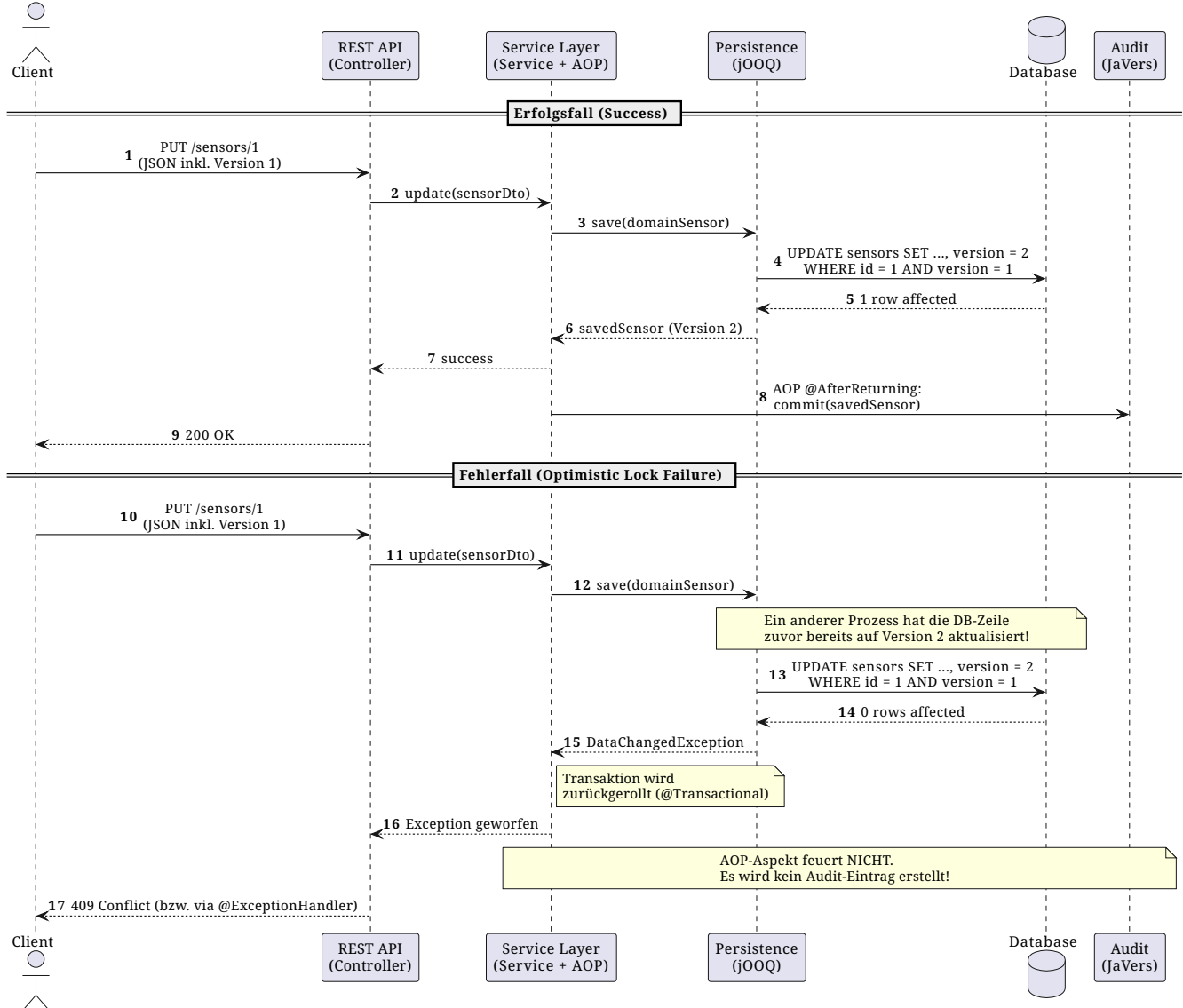
Service-Schicht (Domain Models): Führt die reine Geschäftslogik aus. Diese Schicht arbeitet mit domänenspezifischen Java-Objekten. Ein Spring AOP-Aspekt (Aspektorientierte Programmierung) überwacht hier die Service-Methoden (z.B. getriggert durch `@AfterReturning` oder eine Custom Annotation).

Persistenz-Schicht (jOOQ Records): Übernimmt das Mapping vom Domain Model auf den jOOQ `UpdatableRecord`. Durch den expliziten Aufruf von `record.update()` generiert jOOQ das notwendige SQL für das Optimistic Locking vollautomatisch und wertet die betroffenen Zeilen aus. (Neue Datensätze werden regulär über `record.insert()` gespeichert).

Audit-Schicht (JaVers): Arbeitet vollkommen passiv. Der AOP-Aspekt greift sich das resultierende Domain Model nur dann, wenn die Methode der Business-Schicht erfolgreich und ohne Exceptions beendet wurde, und committet den Status-Diff in die JaVers-Tabellen.

8.3.3. Erfolgs- und Fehlerfall

Das folgende Diagramm veranschaulicht den Datenfluss und das Zusammenspiel der Komponenten, insbesondere warum fehlgeschlagene Updates nicht zu fehlerhaften Audit-Einträgen führen.



Wenn bei der Versionierung etwas schief geht, erfolgt ein Rollback der Übergeordneten Transaktion, damit die gespeicherten Daten und die zugehörige Versionierung konsistent bleiben (Wenn etwas gespeichert wird, ist es in der Versionierung ersichtlich).

8.3.4. Aspekt für die Versionierung

Damit die Service-Schicht nur minimale Kenntnisse von der Versionierung haben muss, wird ein Spring AOP Aspekt verwendet. Methoden werden lediglich mit einer eigenen Annotation versehen (@AuditChanges), damit das betroffene Objekt vom Aspekt aufgegriffen und sauber in den Versionstabellen von JaVers abgelegt wird. Hier gilt zu beachten, dass @AfterReturning verwendet

wird, damit beim Einfügen eines neuen Objekts dessen eindeutige ID bereits gesetzt wurde. Hier ist ein Beispiel, wie der Aspekt aussehen könnte:

```
import org.aspectj.lang.annotation.AfterReturning;
import org.aspectj.lang.annotation.Aspect;
import org.javers.core.Javers;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class JaversAuditAspect {

    private final Javers javers;

    public JaversAuditAspect(Javers javers) {
        this.javers = javers;
    }

    @AfterReturning(pointcut = "@annotation(AuditChanges)", returning = "result")
    public void auditReturnValue(Auditable result) {

        if (result == null) {
            return;
        }

        String currentUser = "SYSTEM";
        if (SecurityContextHolder.getContext().getAuthentication() != null) {
            currentUser = SecurityContextHolder.getContext().getAuthentication().getName();
        }

        javers.commit(currentUser, result);
    }
}
```

Voraussetzung:

- Aspekt muss als Komponente und Aspekt annotiert sein
- Dependency muss im pom.xml definiert sein:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

8.3.5. Entwickler-Leitfaden: Neue Entitäten für Audit & Locking anbinden

Wenn ein neues Fachobjekt (z. B. **Experiment**) in die Architektur integriert wird, müssen folgende Schritte zwingend eingehalten werden, damit Optimistic Locking (via jOOQ) und die Historisierung (via JaVers) reibungslos ineinandergreifen:

1. Datenbank & jOOQ (Persistenz-Schicht)

- **Schema:** Die neue Datenbanktabelle benötigt zwingend eine Versions-Spalte (z. B. **version INTEGER NOT NULL DEFAULT 1**).
- **jOOQ Generator:** Die Konfiguration im **pom.xml** (`<recordVersionFields>VERSION|version</recordVersionFields>`) sorgt dafür, dass jOOQ das Optimistic Locking vollautomatisch anwendet.
- **Dirty Tracking (Wichtig):** Global in der Spring Boot Konfiguration muss das jOOQ-Setting **RecordDirtyTracking.MODIFIED** aktiviert sein. Dadurch prüft jOOQ via **Objects.equals()**, ob sich ein Wert **wirklich** geändert hat, bevor ein Datensatz als "dirty" markiert wird.

2. DTO-Schicht (Data Transfer Objects als Java Records)

- **Marker Interface:** Das DTO muss zwingend das Interface **Auditable** implementieren, damit der AOP-Aspekt es verarbeiten kann.
- **JaVers-Annotationen & Konstanten:** Das DTO muss die **@Id**-Annotation auf dem Primärschlüssel tragen. Die Nutzung von **@TypeName** wird zwingend vorgeschrieben, um den Audit-Log von Java-Refactorings zu entkoppeln. Der Wert für **@TypeName** muss als **public static final String** im Record definiert werden (z. B. **JAVERS_TYPE_NAME = "Sensor"**), damit Repositories bei nativen DB-Queries typsicher darauf zugreifen können.
- **State-Transport:** Das DTO muss die Version(en) zwingend als Attribut enthalten (z. B. **version** und **formulaVersion**).

3. Repository-Schicht

- **Strikte Trennung:** **INSERT**- und **UPDATE**-Logik müssen in dedizierte Methoden aufgeteilt werden (Auf das generische **record.store()** wird verzichtet, da Optimistic Locking nur für Updates relevant ist).
- **Das Update-Pattern:** Jede Update-Methode muss exakt diesen Ablauf befolgen:
 1. Den aktuellen Datensatz (Baseline) aus der DB laden.
 2. **Versions-Rewind (Locking Prep):** Die Version des geladenen jOOQ-Records explizit auf die Version des eingehenden DTOs zurücksetzen (z. B. **sensorRecord.setVersion(dto.version())**).
 3. Neue Werte aus dem DTO blind in den Record setzen (via Setters). Dank **RecordDirtyTracking.MODIFIED** ignoriert jOOQ Setters mit identischen Werten völlig.
 4. **record.update()** aufrufen. jOOQ generiert nun automatisch die **WHERE id = ? AND version = ?** Klausel auf Basis der "rewound" Version. Liefert der Client eine veraltete Version, schlägt dieses Update fehl und jOOQ wirft nativ eine **DataChangedException**.
 5. **Ghost-Updates verhindert:** Wenn sich keine Nutzdaten geändert haben, führt **update()** dank des **MODIFIED**-Trackings kein SQL-Update aus und die Version bleibt unverändert.

JaVers registriert dies, erkennt keine Differenz und legt automatisch keinen leeren Audit-Log an.

4. Service-Schicht

- **Trigger:** Die speichernde Service-Methode muss zwingend mit `@Transactional` und der eigenen Annotation `@AuditChanges` versehen werden.
- **Rückgabewert (Kritisch!):** Die Methode *muss* das frisch gespeicherte DTO zurückgeben. Der AOP-Aspekt auditiert exakt diesen Rückgabewert, um sicherzustellen, dass die aktualisierte Version und generierte ID korrekt im Audit-Log landen.

8.3.6. Umgang mit initialen Daten (Flyway Seeding & Backfill)

Da initiale Test- oder Stamm-Daten oft via Flyway-SQL-Skripte direkt in die Datenbank (`INSERT INTO ...`) geschrieben werden, umgehen sie den Spring Service Layer und somit den JaVers AOP-Aspekt vollständig. Für JaVers existieren diese Datensätze nicht, was bedeutet, dass beim ersten regulären Update über die API die Historie des Ursprungszustands verloren geht.

Um diesen "Blindspot" zu beheben, nutzt die Architektur einen **Startup Backfill Service**:

- **Event-Trigger:** Ein `@EventListener(ApplicationReadyEvent.class)` prüft beim Hochfahren der Applikation auf un-auditierte Daten.
- **jOOQ Streaming & NOT EXISTS:** Um Out-Of-Memory (OOM) Fehler und das "N+1 Query"-Problem bei grossen Datenmengen (z.B. >100.000 Sensoren) zu vermeiden, nutzt das Repository einen Datenbank-Cursor (`fetchStream()`). Über eine native `WHERE NOT EXISTS` Klausel direkt auf der JaVers-Tabelle (`jv_global_id`) wird die Filterung komplett in die Datenbank verlagert.
- **Dynamische Typisierung:** Der Abgleich in jOOQ greift dabei sicher auf die statische Konstante des DTOs zurück (z. B. `WriteSensorDto.JAVERS_TYPE_NAME`).
- **Sicherer Commit:** Gefundene unauditierte Datensätze werden vom Backfill-Service iterativ mit dem Autor `SYSTEM_SEEDER` in JaVers als `INITIAL` Snapshot committet.

8.4. Core-API Validierung (Validation & Error Handling)

Die Validierung ist streng in zwei Schichten unterteilt und wird über einen zentralen Mechanismus so verarbeitet, dass das Frontend (Angular) immer eine einheitliche, übersetzungsfertige Struktur erhält, während die Backend-Domäne kompromisslos geschützt bleibt.

- **Syntaktische Validierung (Formale DTO-Prüfung):**
 - **Wo:** Im Web Layer (Controller-Eingang) auf den Request DTOs.
 - **Wie:** Mit Jakarta-Annotationen (z.B. `@Min`, `@NotBlank`) direkt auf den Feldern oder über massgeschneiderte Cross-Field Validatoren (z.B. `@ValidFormulaState`). Als Message wird kein Text, sondern ein flacher Translation-Key (z.B. `validation.required`) hinterlegt.
 - **Zweck:** Fängt formale Datenfehler ab, bevor sie gemappt werden. Resultiert in einem **422 Unprocessable Content**.

- **Hinweis zu Typen:** Falsche Datentypen (z.B. String statt Zahl) werden bereits durch den Jackson-Deserializer vor der Validierung abgefangen. Dies resultiert aus Sicherheitsgründen ("Fail Secure") in einem **400 Bad Request** mit einer generischen Fehlermeldung und einer Trace-ID, um keine internen Details preiszugeben.
- **Semantische Validierung (Business-Logik & Always-Valid Domain Model):**
 - **Wo:** Direkt im Domain Model (Entities, z.B. im Konstruktor oder Setter der Klasse `Formula`).
 - **Wie:** Die Domäne prüft fachliche Regeln bei der Instanziierung. Ist ein Zustand ungültig (z.B. eine Formel enthält kein 'x'), wirft die Entität einen `ValidationError`. Es ist physisch unmöglich, ein ungültiges Domänenobjekt im Backend zu erzeugen.
 - **Feld- vs. Objekt-Fehler (Cross-Field Validation):**
 - **Field Errors:** Verletzen Regeln eines einzelnen Attributs. Der Fehler wird mit dem exakten Feldnamen (z.B. `expression`) geworfen und im Frontend zielgenau unter dem entsprechenden Input-Feld angezeigt.
 - **Object / Form Errors:** Verletzen Regeln, die das Zusammenspiel mehrerer Attribute betreffen (z.B. "Wenn eine ID existiert, muss auch eine Version mitgegeben werden"). In diesem Fall wird der Fehler gezielt mit einem übergeordneten Identifier wie `form` geworfen. Der Frontend-Service fängt dies ab und weist den Fehler der gesamten `FormGroup` zu, sodass er als globaler Formularfehler dargestellt werden kann.
 - **Zweck:** Kapselung der Geschäftsregeln. Schlägt dies fehl, resultiert dies ebenfalls in einem **422 Unprocessable Content** mit dem betroffenen Formular-Feldnamen (oder dem übergeordneten `form` Identifier) und Message-Key.
- **Der Magische `@RestControllerAdvice` (Zentraler Exception Handler):**
 - Dies ist das Herzstück der Fehlerbehandlung. Er erweitert den `ResponseEntityExceptionHandler`, um die nativen HTTP-Semantiken von Spring (wie 405 Method Not Allowed) beizubehalten, tauscht aber den Body gegen unser einheitliches `ErrorDTO` aus.
 - Er definiert globale `@ApiResponse`-Annotationen, wodurch Swagger/OpenAPI automatisch die TypeScript-Interfaces für das Frontend generiert.
 - **Für 422 Validierungsfehler:**
 - Er extrahiert *dynamisch* den Namen des fehlerhaften Feldes.
 - Er liest alle Parameter aus den Annotationen (z.B. das `min` Limit aus `@Min`) komplett ohne hartcodierte Strings aus.
 - **Für 400 (Client Errors) und 500 (Server Crashes):**
 - Es gilt das Prinzip der "Information Hiding". Bei Jackson-Parsingfehlern oder NullPointern wird eine UUID (Trace-ID) generiert, geloggt und an das Frontend gesendet. Der Benutzer sieht nur eine generische Meldung (z.B. "System Error"), aber Entwickler können den Fehler anhand der Trace-ID sofort in den Logs finden.

Das Resultat für das Frontend:

Egal ob eine DTO-Annotation fehlschlägt oder die Domäne eine Exception wirft, Angular erhält bei **422-Fehlern** immer den gleichen Payload: Den flachen Feldnamen, den Translation-Key und ein

dynamisches Parameter-Objekt. Bei generischen Fehlern wird das Feld **global** verwendet.

```
[
  {
    "field": "age",
    "actualValue": 16,
    "messageKey": "validation.min",
    "params": {
      "value": 18
    }
  },
  {
    "field": "global",
    "actualValue": null,
    "messageKey": "error.server.internal",
    "params": {
      "traceId": "123e4567-e89b-12d3-a456-426614174000"
    }
  }
]
```

- **Übersetzungsfile im Frontend:**

Im Angular Public Folder werden die Übersetzungsfiles platziert. Sie enthalten dieselben Schlüssel, die das Backend verwendet, und binden die dynamischen Parameter über Interpolation ein.

```
{
  "validation.min": "Der aktuelle Wert ist {{ actualValue }}, er muss aber  
mindestens {{ value }} sein.",
  "validation.required": "Dieses Feld darf nicht leer sein.",
  "error.server.internal": "Ein unerwarteter Fehler ist aufgetreten. Bitte den  
Support mit der Referenznummer {{ traceId }} kontaktieren."
}
```

8.5. Architektur- und Skalierungskonzept: Data Pipeline

8.5.1. Architekturübersicht

Das folgende Diagramm veranschaulicht den Datenfluss vom externen Partner bis zur TimeScaleDB. Es zeigt die physische Gruppierung in einen "Core Pod" und einen "Pipeline Pod" und verdeutlicht die Entkopplung von Ingress (Adapter), Verarbeitung (Processor), Cache-Befüllung und Reprocessing.

Topic Name	Typ / Ort	Beschreibung
<code>internal-sensor-config</code>	Lokal (Compacted)	Wird von der Core API befüllt (<code>cleanup.policy=compact</code>) — beim Erstellen eines Sensor-Parameters immer, bei Aktualisierung nur wenn sich Formel oder Grenzwerte ändern. Publiziert wird pro <code>sensor_parameter</code> mit dem zusammengesetzten DAS-Key (<code><DAS><das_sensor_alias><das_parameter_alias></code> , siehe DAS-Mapping: Inbound-Key vs. Persistenz-Referenz) als Kafka-Message-Key, da Formel und Grenzwerte seit der Aufteilung in Sensor vs. Sensor_Parameter auf dieser Ebene liegen. Die Payload trägt zusätzlich <code>sensor_parameter_id</code> als stabile Persistenz-Referenz. Dient als Source-of-Truth für den In-Memory-Cache der Worker (<code>SensorConfigCache</code>) und als Trigger für das Reprocessing.



Die `version` eines Sensor-Parameters wird über den jOOQ-Optimistic-Locking-Mechanismus bei **jeder** Aktualisierung erhöht — unabhängig davon, ob diese Änderung eine Publikation auslöst. Da nur Formel- oder Grenzwert-Änderungen publiziert werden, können auf dem Topic `internal-sensor-config` daher Versionssprünge auftreten (z. B. Version 1 gefolgt von Version 4, wenn zwischenzeitliche Aktualisierungen wie eine Namensänderung keine Publikation ausgelöst haben). Dies ist beabsichtigt und unkritisch: Das Reprocessing (siehe [SensorReadingRepository](#)) vergleicht Versionen ausschliesslich mit einem strikten `<`-Operator und setzt keine lückenlose Versionsfolge voraus.

8.5.3. Skalierungsstrategien (High Throughput)

Da das externe Quell-Topic 6 Partitionen besitzt, ist die gesamte Pipeline auf maximale Parallelität über genau diese 6 Partitionen ausgelegt.

- **Kafka Connect (Ingestion):** Der MirrorSourceConnector ist mit `"tasks.max": "6"` konfiguriert. Kafka Connect baut exakt 6 parallele Threads auf, um das externe Topic ohne Bottlenecks abzusaugen.
- **Ingress Adapter (`SolExpertsRawSensorDataNormalizationService`):** Übernimmt das Entpacken der herstellerspezifischen Payloads. Nutzt `@KafkaListener(concurrency = "6")`, um die Last optimal auf die Partitionen zu verteilen.
- **Datenbank-Schicht (`TimeScaleDB`):** Um die Datenbank bei hohen Datenraten nicht zu überlasten, konsumiert der Processor (`SensorDataBatchListener`) im **Batch-Modus** (`spring.kafka.listener.type=batch`, `max-poll-records=200`). Das Schreiben erfolgt im Batch-Modus über ein hochoptimiertes jOOQ Multi-Row-Insert mit nachgelagerter ON CONFLICT (timestamp, sensor_parameter_id) DO UPDATE-Logik, um maximale Idempotenz bei minimalem Verbindungs-Overhead zu gewährleisten.

8.5.4. Datenintegrität & Zero-Data-Loss Architektur

Das Monteis-Pipeline-System verarbeitet Sensordaten über zwei primäre Datenströme: Den **Process-Flow** für Echtzeitdaten und den **Reprocess-Flow** für historische Datenkorrekturen. Ein zentrales Element dieser Architektur ist das Zusammenspiel aus manuellen Kafka-Acknowledgments, idempotenten Datenbank-Operationen und gezieltem Poison-Pill-Handling.

Gemeinsame Basis: Orchestrator & Kafka-Offsets

Beide Flows stützen sich auf gemeinsame Komponenten und eine strikte Kafka-Konfiguration, die garantiert, dass keine Nachricht im Nirvana verschwindet.

- **Shared Transformation Orchestrator:** Um mathematische Abweichungen zu verhindern, teilen sich der Ingestion-Flow und der Backfill-Flow exakt dieselbe Logik. Dies garantiert, dass Standardisierung und Validierungs-Flags systemweit identisch berechnet werden.
- **Cache Broadcast Pattern:** Damit jeder Pipeline-Pod einen synchronen Metadaten-Cache besitzt, generiert der `SensorConfigCacheHydrationListener` beim Start eine zufällige UUID als Consumer Group (`cache-hydrator-#{UUID}`). Dadurch wird jedes Metadaten-Update an **jeden** laufenden Pod gebroadcastet.
- **Deaktiviertes Auto-Commit (`enable-auto-commit: false`):** Spring Boot darf Kafka-Offsets niemals im Hintergrund bestätigen. Das System verlässt sich ausschliesslich auf manuelle Bestätigungen (`ack-mode: manual`).
- **Das Guarded Acknowledgment Pattern (At-Least-Once):** In allen Listnern wird der Kafka-Offset erst explizit committet (`ack.acknowledge()`), **nachdem** die Daten sicher in die nächste Schicht oder die TimeScaleDB persistiert wurden. Fällt ein Pod aus, pausiert Kafka und liefert das Event nach dem Neustart erneut aus.

Explizite Separation of Concerns für Container-Factories

Um das Risiko von hängenden Offsets oder Datenverlust bei zustandslosen `@SendTo`-Integrationen zu eliminieren, verwendet das System zwei spezialisierte Container-Factories:

- `manualAckFactory`: Reserviert für rein infrastrukturseitige oder datenbanknahe Konsumenten (z. B. `SensorConfigCacheHydrationListener` oder `SensorDataBatchListener`). Sie erzwingt die manuelle Injektion von Acknowledgment und ein explizites `ack.acknowledge()` im Code.
- `forwardingMessageFactory`: Speziell konzipiert für zustandslose EIP-Ingress-Adapter mit deklarativer `@SendTo`-Weiterleitung. Sie nutzt den `AckMode.RECORD` und stellt sicher, dass der Offset erst bestätigt wird, nachdem alle transformierten Datensätze vom Broker erfolgreich quittiert wurden.

8.5.5. 4.2. Echtzeit-Verarbeitung (Process-Flow)

Echtzeit-Verarbeitung (Process-Flow)

Dieser Flow verarbeitet neue, normalisierte Sensordaten, standardisiert sie und speichert sie in der TimeScaleDB.

- **Kafka Batching:** Der Konsum erfolgt in Batches, um den Durchsatz zu maximieren.

- **Idempotente UPSERTs:** Der `SensorDataBatchProcessor` nutzt `jOOQs ON CONFLICT (timestamp, sensor_parameter_id) DO UPDATE`. Wenn Kafka einen Batch nach einem Absturz erneut sendet, überschreibt die Datenbank die Zeilen einfach mit exakt denselben, korrekten Daten. Es entstehen keine Duplikate.
- **Poison Pill Handling (Batch-Rettung):** Wenn ein einzelner Datensatz innerhalb eines 200er-Batches fehlerhaft ist (z. B. unlesbarer Timestamp), fängt der `SensorDataBatchProcessor` die `TransformationException` lokal ab, loggt den fehlerhaften Rohwert und überspringt ihn (`return null`). Die verbleibenden 199 validen Datensätze werden erfolgreich in der DB gespeichert, anstatt den gesamten Batch in eine Dead Letter Queue (DLQ) zu leiten.

Historische Daten-Reprozessierung (Reprocess-Flow)

Dieser Flow wird durch ein Aktualisierungs-Event der Sensorkonfiguration (`SensorConfig`) getriggert und ist darauf ausgelegt, potenziell Hunderttausende historischer Zeitreihendaten rückwirkend an neue mathematische Formeln anzupassen. Die Architektur trennt dabei die Steuermechanismen strikt von den ressourcenintensiven Datenbankoperationen, um den Speicherbedarf konsistent niedrig zu halten und Deadlocks zu verhindern.

- **High-Performance Guard:** Der `SensorReprocessingOrchestrator` führt initial eine leichte `EXISTS`-Abfrage auf der Datenbank aus. Sind für die betroffene Formelversion keine veralteten Datensätze vorhanden, terminiert der Prozess ressourcenschonend innerhalb weniger Millisekunden.
- **Feingranulare Transaktionsgrenzen via `TransactionTemplate`:** Um die Belegungsdauer wertvoller Datenbankverbindungen im HikariCP-Pool zu minimieren, wird im `HistoricalReadingChunkProcessor` bewusst auf eine deklarative Methodensperre via `@Transactional` verzichtet. Auch wenn die mathematische Auswertung der normalisierten Werte dank des hochperformanten Parsington-Expression-Cachings im Arbeitsspeicher nur wenige Millisekunden beansprucht, trennt die Architektur die Netzwerk-I/O-Phasen strikt voneinander:
 1. **Transaktionsfreies Lesen:** Die historischen Datensätze werden ohne aktive Schreibtransaktion über das Netzwerk geladen.
 2. **In-Memory-Transformation:** Die Auswertung der vorab geparsten Parsington-Formeln und das Stream-Mapping erfolgen vollständig im lokalen Arbeitsspeicher der JVM (ohne gebundene JDBC-Connection).
 3. **Programmatischer DB-Commit:** Erst für das abschließende Schreiben wird ein dediziertes Spring `TransactionTemplate` geöffnet. Dieses kapselt das `jOOQ-Bulk-Update` in eine extrem kurze, atomare Schreibtransaktion, was die Netzwerk-Lesezeiten komplett aus der Transaktionsdauer eliminiert und den Durchsatz bei parallelen Workflows maximiert.
- **Schutz vor Race-Conditions (Partitioned Kafka Consumer Groups):** Da das Triggern der Reprozessierung ereignisgesteuert über das Topic `internal-sensor-config` erfolgt, greift auf Infrastrukturebene ein natürlicher Schutz vor Nebenläufigkeitskonflikten. Sämtliche Konfigurations-Updates werden zwingend mit dem eindeutigen, zusammengesetzten DAS-Key (`<DAS><das_sensor_alias><das_parameter_alias>`) als Kafka-Message-Key publiziert. Durch die intrinsischen Eigenschaften von Partitioned Kafka Consumer Groups garantiert der Broker, dass alle Events für einen spezifischen Sensor-Parameter strikt sequenziell und stets von genau demselben Konsumenten-Thread verarbeitet werden. Parallele, kollidierende Reprocess-Flows für dieselben Zeitreihendaten (Write Skews oder Lost Updates über Pod-Grenzen hinweg) sind

somit architektonisch ausgeschlossen, weshalb auf der Datenbankebene auf zusätzliche, aufwändige Sperrmechanismen (wie Pessimistic oder Optimistic DB-Locking) verzichtet wird.

- **Iteratives Chunking (JVM-Speichersicherheit):** Um Out-Of-Memory-Fehler (OOM) auf den Kubernetes-Pods zu vermeiden, wird die Verarbeitung über eine dynamische `do-while`-Schleife im Orchestrator gesteuert. Jede Abfrage erzwingt ein striktes, über die `application.properties` konfigurierbares `LIMIT` (z. B. 1000 Datensätze pro Chunk) und nutzt einen Cursor auf Basis des ältesten `timestamp` des vorherigen Chunks (`ORDER BY timestamp DESC`), um direkt an der letzten Position fortzusetzen statt bereits verarbeitete Zeilen erneut zu scannen — dies verhindert Überlappungen oder Lücken zwischen aufeinanderfolgenden Chunks. Die Schleife iteriert so lange, bis die Anzahl der geladenen Datensätze `0` erreicht.
- **Poison Pill Handling (Die Endlosschleifen-Falle):** Schlägt die mathematische Transformation eines historischen Datensatzes fehl (z. B. durch eine Division durch Null aufgrund korrupter Rohwerte), darf dieser Datensatz unter keinen Umständen unmodifiziert übersprungen werden. Andernfalls würde die SQL-Abfrage dieselbe fehlerhafte Zeile im nächsten Chunk-Durchlauf erneut laden, was in einer Endlosschleife resultieren würde.

Die Loop-Breaker Lösung: Die `TransformationException` wird gezielt abgefangen. Um die Schleife zu durchbrechen, wird die `version` des betroffenen Datensatzes explizit auf die aktuelle Zielversion angehoben und der `norm_value` auf `null` gesetzt:

```
} catch (TransformationException ex) {
    log.error("POISON PILL REPROCESSING: Math failed for historical record of sensor
{}.". " +
        "Raw Value: [{}]. Bumping version to bypass infinite loop. Reason: {}",
        reading.getId(), reading.getRawValue(), ex.getMessage(), ex);

    // 1. Versions-Inkrement erzwingen: Verhindert das erneute Erfassen im nächsten
    // Chunk-Durchlauf
    reading.setVersion(sensorConfig.getVersion().shortValue());

    // 2. norm_value auf null setzen: Verhindert die Auslieferung korrupter
    // Berechnungen an nachgelagerte Systeme
    reading.setNormValue(null);

    return reading;
}
```

8.6. Integration neuer externer Datenquellen

Die Architektur verwendet das **Adapter-Pattern**, um neue Datenquellen nahtlos als eigenständige "Plug-and-Play"-Komponenten zu integrieren. Um eine maximale Entkopplung und Testbarkeit zu gewährleisten, setzt die Pipeline konsequent auf **Komposition statt Vererbung** (Composition over Inheritance). Die Kern-Pipeline (alles ab dem `internal-normalized` Topic) bleibt von der Herkunft und dem ursprünglichen Format der Daten vollständig isoliert.

Wenn ein neues externes System angebunden werden soll (z. B. Sensordaten im CSV-Format auf einem FTP-Server), ist der folgende standardisierte Prozess durchzuführen:

8.6.1. Ingress Layer konfigurieren (Kafka Connect)

Um die Architektur strikt ereignisgesteuert (event-driven) zu halten, sollte die Spring Boot Applikation externe Systeme nicht selbst aktiv pollen.

- Konfiguration eines **Kafka Connect Source Connectors** (z. B. SFTP Plugin oder SpoolDir/FilePulse).
- Der Connector pollt die externe Quelle und schreibt diese Rohdaten in ein neues, dediziertes Raw-Topic (z. B. `internal-raw-ftp-sensors`).

8.6.2. Raw Data Model definieren

Analog zur `RawSolExpertsSensorData`-Klasse muss ein spezifisches Java-Record für die Payload der neuen Datenquelle erstellt werden (z. B. `RawFtpCsvSensorData`).

8.6.3. Normalizer Component implementieren (Strategy Pattern)

Um das Messaging-Plumbing (Kafka) vollständig von der Datenübersetzung zu entkoppeln, setzt die Pipeline auf das **Strategy Pattern**. Für jede neue Datenquelle wird eine spezifische Implementierung des generischen Interfaces `VendorDataNormalizer<T>` als Spring `@Component` bereitgestellt.

- **Zustandslose Transformation:** Der Normalizer nimmt das rohe Hersteller-Record entgegen und übersetzt es in eine Liste von standardisierten Spring `Message<NormalizedSensorData>`-Objekten.
- **Kritisch – DAS-Key als Kafka Partitioning Key (`KafkaHeaders.KEY`):** Damit die nachgelagerte Pipeline (insbesondere der `SensorDataBatchListener`) Messwerte desselben Sensor-Parameters in strikter zeitlicher Reihenfolge und ohne Race-Conditions verarbeiten kann — und damit dieselbe Nachricht sich im `SensorConfigCache` verlustfrei auflösen lässt — muss beim Aufbau der ausgehenden Nachricht **zwingend** der zusammengesetzte DAS-Key (`<DAS><das_sensor_alias><das_parameter_alias>`, siehe [DAS-Mapping: Inbound-Key vs. Persistenz-Referenz](#)) als Kafka-Key gesetzt werden. Der DAS-Enum-Wert ist dabei je Adapter fix (z. B. `SOLEXPerts`), `das_sensor_alias` entspricht dem Geräte-Identifizier der Rohnachricht (`deviceName`) und `das_parameter_alias` dem Feldnamen innerhalb der `values`-Map. Dies geschieht über den `MessageBuilder` mittels `.setHeader(KafkaHeaders.KEY, dasKey)`.

Beispiel-Implementierung des Normalizers:

```
@Component
public class SolExpertsSensorDataNormalizer implements
VendorDataNormalizer<RawSolExpertsSensorData> {

    private static final Das DAS = Das.SOLEXPerts;

    @Override
    public List<Message<NormalizedSensorData>>
normalizeToMessages(RawSolExpertsSensorData rawPayload) {
        if (rawPayload == null || rawPayload.values() == null ||
```

```

rawPayload.values().isEmpty()) {
    return List.of();
}

List<Message<NormalizedSensorData>> outboundMessages = new ArrayList<>();

for (Map.Entry<String, Double> entry : rawPayload.values().entrySet()) {
    String dasParameterAlias = entry.getKey();

    // Composite DAS mapping key:
    <DAS>__<das_sensor_alias>__<das_parameter_alias>
    // Resolved against sensor.das_sensor_alias /
    sensor_parameter.das_parameter_alias
    // in the SensorConfigCache - see "DAS-Mapping: Inbound-Key vs.
    Persistenz-Referenz".
    String dasKey = "%s__%s__%s".formatted(DAS, rawPayload.deviceName(),
    dasParameterAlias);

    NormalizedSensorData canonicalPayload =
        new NormalizedSensorData(dasKey, rawPayload.ts(),
    entry.getValue());

    // CRITICAL: Set Kafka partitioning key to guarantee ordering in
    downstream topics
    // and to match the key format used by SensorConfigCache/internal-sensor-
    config.
    Message<NormalizedSensorData> message = MessageBuilder
        .withPayload(canonicalPayload)
        .setHeader(KafkaHeaders.KEY, dasKey)
        .build();

    outboundMessages.add(message);
}

return outboundMessages;
}
}

```



Der **Das**-Enum-Typ ist im geteilten Maven-Modul **contracts** definiert und wird sowohl von **core** (Publisher, **sensor.das**) als auch von **pipeline** (Normalizer, **SensorConfigCache**) referenziert — es existiert damit genau eine Quelle der Wahrheit für die unterstützten Erfassungssysteme.

8.7. 5.4. Ingress Listener implementieren (Deklarativer EIP-Adapter)

Der eigentliche Konsument lauscht auf das Raw-Topic und agiert als leichtgewichtige, zustandslose Enterprise Integration Pattern (EIP) Pipe. Er delegiert die Übersetzung vollständig an den

injizierten `VendorDataNormalizer`.

- **Klassen-Namenskonvention & Stereotyp (ArchUnit-konform):** Da Konsumenten von Kafka-Nachrichten reine Infrastruktur-Komponenten und keine Domänen-Services sind, muss die Klasse zwingend mit `@Component` (niemals `@Service`!) annotiert sein und im Namen auf `Listener` enden (z. B. `RawSolExpertsSensorDataListener`). Dies wird durch die automatisierte ArchUnit-Testsuite strikt überwacht.
- **Event-Ingestion & Factory-Anbindung:** Die Methode wird mit `@KafkaListener` annotiert und lauscht auf das Raw-Topic. Hierbei muss zwingend die auf Ingress-Routing spezialisierte Container-Factory angegeben werden: `containerFactory = "forwardingMessageFactory"`.
- **Deklaratives Routing via `@SendTo`:** Statt Nachrichten manuell über einen Publisher zu versenden und asynchrone Futures zu verwalten, setzt die Pipeline auf deklaratives Routing. Die Listener-Methode wird mit `@SendTo("${app.kafka.topics.normalized})"` annotiert. Das Spring-Framework fängt die zurückgegebene `List<Message<NormalizedSensorData>>` ab, splittet sie und publiziert jede Nachricht einzeln in das Ziel-Topic.
- **Guarded Acknowledgment (Zero-Data-Loss Garantie):** Das sichere Bestätigen der Offsets übernimmt die zugrundeliegende Infrastruktur vollautomatisch und atomar:
 1. Die `forwardingMessageFactory` empfängt die vom Normalizer generierte Nachrichtenliste.
 2. Sie leitet alle enthaltenen Nachrichten unter Beibehaltung der gesetzten `KafkaHeaders.KEY`-Header an das Ziel-Topic (`internal-normalized`) weiter.
 3. Da die Factory auf `AckMode.RECORD` konfiguriert ist, wird der Kafka-Offset des ursprünglichen Raw-Topics **erst und ausschließlich dann committet**, wenn der Kafka-Broker den erfolgreichen Erhalt und die Speicherung **aller** transformierten ausgehenden Nachrichten bestätigt hat. Stürzt der Pod während der Verarbeitung ab, erfolgt kein Commit und das Raw-Event wird nach dem Neustart sicher erneut verarbeitet.

Beispiel-Implementierung des Ingress-Listeners:

```
@Component
public class RawSolExpertsSensorDataListener {

    private static final Logger log =
        LoggerFactory.getLogger(RawSolExpertsSensorDataListener.class);

    private final VendorDataNormalizer<RawSolExpertsSensorData> normalizer;

    public
    RawSolExpertsSensorDataListener(VendorDataNormalizer<RawSolExpertsSensorData>
    normalizer) {
        this.normalizer = normalizer;
    }

    @KafkaListener(
        topics = "${app.kafka.topics.raw-solexperts}",
        groupId = "solexperts-adapter-group",
        concurrency = "${app.kafka.concurrency:6}",
        containerFactory = "forwardingMessageFactory")
```



```

    @SendTo("${app.kafka.topics.normalized}")
    public List<Message<NormalizedSensorData>>
processRawMessage(RawSolExpertsSensorData rawPayload) {
    log.info("Received raw SolExperts message: {}", rawPayload);

    // Delegation an den Normalizer: Spring Kafka übernimmt das Splitten und
Routing der Liste
    return normalizer.normalizeToMessages(rawPayload);
}
}

```

8.7.1. Alternative: Zero-Code Ingestion via Kafka Connect SMTs

Für einfache, flache Datenformate (wie standardisierte CSV-Dateien via FTP) wird ein **Zero-Code-Ansatz** bevorzugt:

- **Single Message Transforms (SMTs):** Der Kafka Connect Source Connector führt einfache Transformationen (wie Spaltenumbenennungen oder Datentyp-Casts) direkt "on-the-fly" durch.
- **Direktes Routing:** Der Connector überspringt das Raw-Topic und schreibt die standardisierten Payloads direkt in das `internal-normalized` Topic.
- **Vorteile:** Keine zusätzliche Java-Codebasis, keine Unit-Tests und eine reduzierte Infrastruktur-Komplexität.

Die Implementierung eines dedizierten Java-basierten Normalisierungsservices bleibt somit ausschliesslich für Datenquellen reserviert, die komplexe Restrukturierungen, dynamisches Key-Mapping (z. B. Auflösen von verschachtelten Key-Value-Maps) oder bedingte Logik erfordern.

8.8. Datenbankschemata

Im Folgenden ist das aktuelle logische und physische Datenbankschema dokumentiert. Wie in Kapitel 5 beschrieben, verwenden wir als primären Einstiegspunkt eine PostgreSQL-Datenbank mit der PostGIS-Erweiterung für die Verwaltung der Metadaten und Geoinformationen.

8.8.1. Sensor vs. Sensor_Parameter

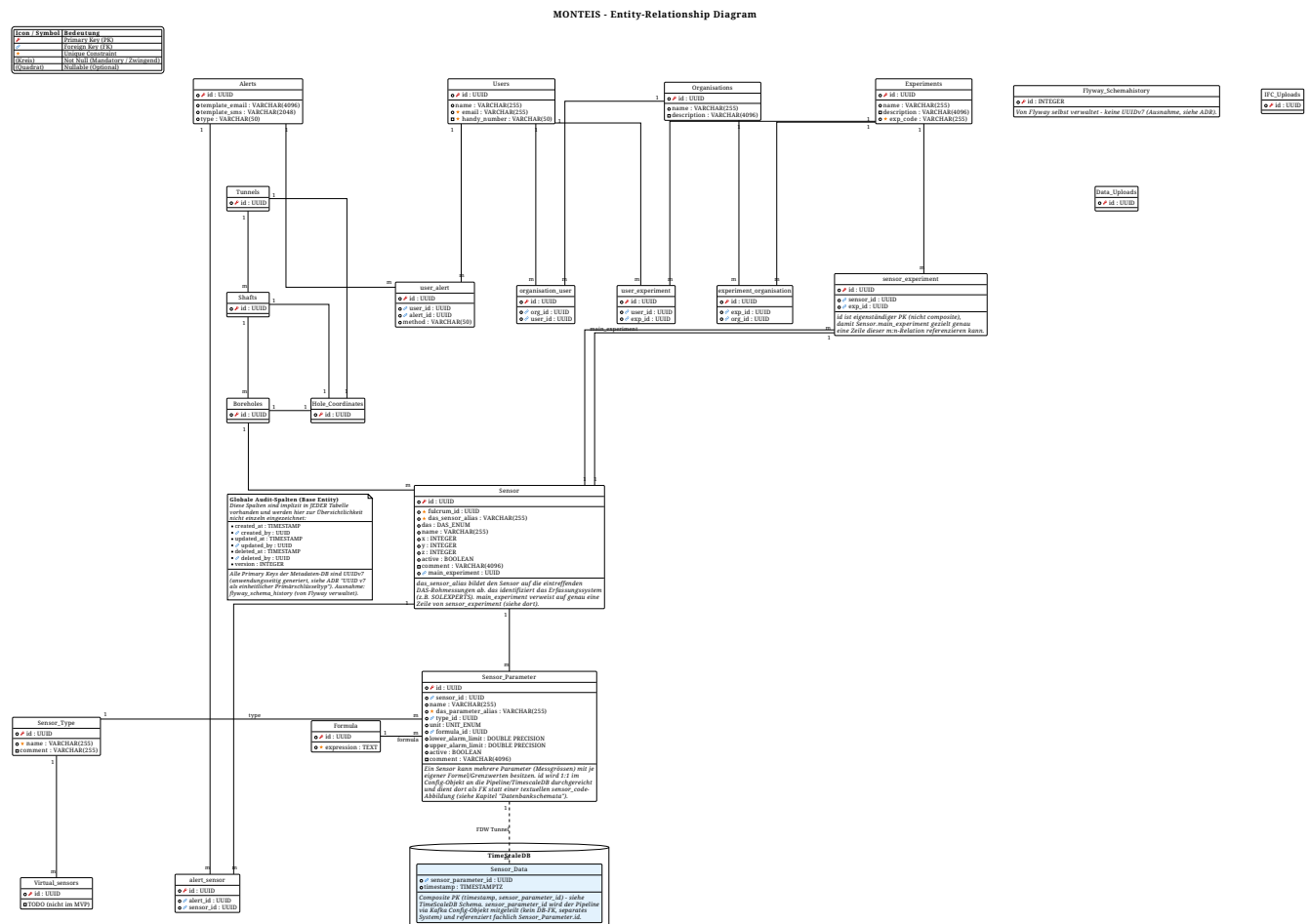
Ein physischer Sensor ist nicht mehr zwangsläufig auf eine einzelne Messgrösse beschränkt: Ein Gerät kann mehrere Parameter (z.B. Temperatur und Druck) liefern, die je eigene Formel, Einheit und Alarmgrenzen besitzen, sich aber ein gemeinsames Set an Metadaten (Standort, DAS-Zuordnung, Fulcrum-Referenz) teilen. Aus diesem Grund ist das Metadatenschema in zwei Tabellen aufgeteilt:

- **sensor** — geteilte Stammdaten je physischem Sensor: Koordinaten (`x/y/z`), die Referenz auf das externe Aufmass-/Inventarsystem Fulcrum (`fulcrum_id`), das Erfassungssystem (`das`) sowie den `das_sensor_alias`, über den eintreffende Rohmessungen des DAS auf den Sensor gemappt werden. Ein Sensor kann mehreren Experimenten zugeordnet sein (`sensor_experiment`, m:n), referenziert über `main_experiment` aber genau eine dieser Zuordnungen als das federführende Experiment.

- **sensor_parameter** — eine Zeile je Messgrösse eines Sensors, mit eigenem **das_parameter_alias** (Mapping der Rohmessung auf den konkreten Parameter), Typ, Einheit, Formel (FK auf **formula**) und Alarmgrenzen.

Diese Trennung ersetzt die vorherige 1:1-Annahme "ein Sensor = eine Messgrösse" und macht `sensor_parameter.id` zur massgeblichen, stabilen Kennung einer einzelnen Messgrösse — sie wird unverändert über das Config-Objekt an die Pipeline durchgereicht und dort als Fremdschlüssel für die Ablage in der TimeScaleDB verwendet (siehe unten), anstelle der bisherigen, fehleranfälligen textuellen `sensor_code`-Abbildung.

Sämtliche Primärschlüssel der Metadaten-DB (inkl. `sensor.id` und `sensor_parameter.id`) sind UUIDv7 — siehe [Architekturentscheidungen](#).



8.8.2. DAS-Mapping: Inbound-Key vs. Persistenz-Referenz

Rohmessungen treffen ausschliesslich mit den Identifikatoren des jeweiligen Erfassungssystems (DAS) ein — nie mit einer internen `sensor_parameter.id`. Damit die Pipeline eine eintreffende Messung dennoch verlässlich einem `sensor_parameter`-Datensatz zuordnen kann, ohne bei jeder einzelnen Nachricht die Core API anzufragen, wird ein deterministischer, zusammengesetzter Inbound-Key verwendet:

```
<DAS>  <das sensor alias>  <das parameter alias>
```

Beispiel: SOLEXPERTS BH12-P03 temperature

- **Präfix DAS:** Der Enum-Wert des Erfassungssystems (z. B. `SOLEXPPTS`). Damit Core (Publisher) und Pipeline (Konsument) exakt dasselbe Vokabular verwenden, ist der `Das`-Enum **nicht** dupliziert, sondern einmalig im geteilten Maven-Modul `contracts` definiert und wird von beiden Modulen als Abhängigkeit eingebunden.
- **Mittelteil `das_sensor_alias`:** Der auf dem DAS verwendete Geräte-/Sensor-Identifizier (`sensor.das_sensor_alias`), z. B. der `deviceName` aus der Rohnachricht.
- **Suffix `das_parameter_alias`:** Der auf dem DAS verwendete Feldname der konkreten Messgrösse (`sensor_parameter.das_parameter_alias`), z. B. der Key innerhalb der `values`-Map einer Rohnachricht.

Dieser zusammengesetzte Key wird an zwei Stellen verwendet:

1. **Als Kafka-Message-Key** auf `internal-sensor-config`, analog zum bisherigen Pattern (bisher `sensor.code`, siehe [Übersicht der Kafka Topics](#)). Der `Normalizer` einer Datenquelle (siehe [Integration neuer externer Datenquellen](#)) baut denselben Key beim Verarbeiten einer Rohnachricht auf und verwendet ihn ebenfalls als Kafka-Partitionierungs-Key für die nachgelagerten Topics.
2. **Als Lookup-Key im `SensorConfigCache`** der Pipeline-Worker: Der Cache-Miss/Hit erfolgt weiterhin wie bisher über eine simple Map, nur ist der Key jetzt der zusammengesetzte DAS-Key statt eines flachen `sensorId`-Strings.

Der eigentliche Cache-Wert (das `SensorConfig/SensorParameterConfig`-Objekt, siehe [API-Schnittstellen \(Contracts\)](#)) transportiert zusätzlich `sensor_parameter_id`. **Genau diese UUID — nicht der DAS-Key — wird beim Schreiben in die TimeScaleDB als Fremdschlüssel persistiert** (siehe `sensor_reading.sensor_parameter_id` weiter unten). Die restliche Transformation (Formel-Anwendung, SI-Standardisierung, Grenzwert-Validierung) bleibt dabei unverändert.

Diese Entkopplung von Inbound-Key (volatil, DAS-spezifisch) und Persistenz-Referenz (stabil, systemübergreifend eindeutig) ist bewusst gewählt: Ändert sich z. B. die Benennung eines Geräts auf dem DAS (Vendor benennt `das_sensor_alias` um) oder wird ein Sensor auf ein neues Erfassungssystem migriert, aktualisiert Core lediglich `sensor.das/das_sensor_alias/sensor_parameter.das_parameter_alias` und publiziert die neue Zuordnung (neuer DAS-Key → dieselbe `sensor_parameter_id`) auf `internal-sensor-config`. Der Pipeline-Cache re-keyed sich dadurch selbständig; bereits geschriebene historische Messwerte bleiben unverändert über dieselbe `sensor_parameter_id` referenziert und gehen bei einer Remapping-Änderung nicht verloren.

8.8.3. TimeScaleDB Schema (Zeitreihen & Downsampling)

Das Schema der TimeScaleDB für die hochfrequenten Zeitreihendaten umfasst neben der Hypertable auch die Continuous Aggregates (Downsampling-Buckets) und die Retention-Policies für das Daten-Lebenszyklus-Management.

```
-- 1. Enum für Range-Status
-- Bleibt speichereffizient (4 Bytes) und hochgradig komprimierbar
CREATE TYPE range_category AS ENUM ('too_low', 'correct', 'too_high');

-- 2. Base Table erstellen (Raw Data)
```

```

-- Hinweis: sensor_parameter_id referenziert fachlich sensor_parameter.id
-- in der Metadaten-DB (kein DB-FK, getrenntes System) - der Pipeline via
-- Kafka Config-Objekt mitgeteilt, statt wie bisher ueber einen sensor_code.
CREATE TABLE sensor_reading (
    timestamp          TIMESTAMPTZ NOT NULL,
    sensor_parameter_id UUID NOT NULL,
    raw_value          DOUBLE PRECISION NOT NULL,
    norm_value         DOUBLE PRECISION,
    version            SMALLINT DEFAULT 1,
    status             range_category,

    -- Composite Primary Key: Erzwingt Eindeutigkeit pro Sensor-Parameter &
    -- Zeitstempel
    PRIMARY KEY (timestamp, sensor_parameter_id)
);

-- 3. In eine Hypertable konvertieren (Partitionierung nach 'timestamp')
SELECT create_hypertable('sensor_reading', 'timestamp');

-- 4. Native Kompression für Originaldaten aktivieren (nach 120 Tagen)
ALTER TABLE sensor_reading SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'sensor_parameter_id'
);
SELECT add_compression_policy('sensor_reading', INTERVAL '120 days');

-- =====
-- CONTINUOUS AGGREGATES (Downsampling Buckets)
-- =====

-- Bucket 1: 1-Minuten-Aggregation
CREATE MATERIALIZED VIEW sensor_reading_1m
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 minute', timestamp) AS bucket,
       sensor_id,
       AVG(norm_value) AS avg_value,
       MAX(norm_value) AS max_value,
       MIN(norm_value) AS min_value
FROM sensor_reading
GROUP BY bucket, sensor_id;

-- Bucket 2: 15-Minuten-Aggregation
CREATE MATERIALIZED VIEW sensor_reading_15m
WITH (timescaledb.continuous) AS
SELECT time_bucket('15 minutes', timestamp) AS bucket,
       sensor_id,
       AVG(norm_value) AS avg_value
FROM sensor_reading
GROUP BY bucket, sensor_id;

```

```

-- Bucket 3: 1-Stunden-Aggregation
CREATE MATERIALIZED VIEW sensor_reading_1h
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', timestamp) AS bucket,
       sensor_id,
       AVG(norm_value) AS avg_value
FROM sensor_reading
GROUP BY bucket, sensor_id;

-- Bucket 4: 1-Tages-Aggregation (Langzeitarchiv)
CREATE MATERIALIZED VIEW sensor_reading_1d
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 day', timestamp) AS bucket,
       sensor_id,
       AVG(norm_value) AS avg_value
FROM sensor_reading
GROUP BY bucket, sensor_id;

-- =====
-- RETENTION POLICIES (Rolling Windows)
-- =====

-- Alte Aggregationen verwerfen, sobald sie aus ihrem Zeitfenster fallen:
SELECT add_retention_policy('sensor_reading_1m', INTERVAL '30 days');
SELECT add_retention_policy('sensor_reading_15m', INTERVAL '60 days');
SELECT add_retention_policy('sensor_reading_1h', INTERVAL '2 years');
-- Anmerkung: Die 'sensor_reading_1d' View behält Daten dauerhaft.

-- =====
-- REFRESH POLICIES (Background Jobs für Continuous Aggregates)
-- =====

-- 1-Minuten-Bucket: Aktualisiert sich jede Minute im Hintergrund
SELECT add_continuous_aggregate_policy('sensor_reading_1m',
    start_offset => NULL,
    end_offset   => INTERVAL '1 minute',
    schedule_interval => INTERVAL '1 minute');

-- 15-Minuten-Bucket: Aktualisiert sich alle 15 Minuten
SELECT add_continuous_aggregate_policy('sensor_reading_15m',
    start_offset => NULL,
    end_offset   => INTERVAL '15 minutes',
    schedule_interval => INTERVAL '15 minutes');

-- 1-Stunden-Bucket: Aktualisiert sich stündlich
SELECT add_continuous_aggregate_policy('sensor_reading_1h',
    start_offset => NULL,
    end_offset   => INTERVAL '1 hour',
    schedule_interval => INTERVAL '1 hour');

```

```
-- 1-Tages-Bucket: Aktualisiert sich einmal pro Tag (z.B. Nachts)
SELECT add_continuous_aggregate_policy('sensor_reading_1d',
    start_offset => NULL,
    end_offset   => INTERVAL '1 day',
    schedule_interval => INTERVAL '1 day');
```

8.9. API-Schnittstellen (Contracts)

Im folgenden Abschnitt sind die API-Endpoints als OpenAPI-Spezifikation (YAML) definiert. Diese Spezifikation dient während der aktiven Entwicklungsphase als fester Vertrag (Contract) zwischen dem Backend, dem Frontend und der Ingestion-Pipeline. Die YAML-Blöcke können direkt in Werkzeuge wie den Swagger Editor kopiert werden, um Mocks und Client-Code zu generieren.

Das `SensorParameterConfig`-Schema wird zusätzlich über den OpenAPI Generator in Java-Klassen im geteilten Maven-Modul `contracts` übersetzt. Core (Publisher) und Pipeline (Konsument/`SensorConfigCache`) hängen beide von `contracts` ab und teilen sich so — inklusive des `Das`-Enums — exakt dieselbe, versionierte Typdefinition für das auf `internal-sensor-config` publizierte Kafka-Config-Objekt, ohne Duplikation.

Hinweis: Nach Abschluss der Entwicklungsarbeiten wird dieser statische Block aus dem Architektur-Dokument entfernt und durch einen Link auf die live generierte, dynamische API-Dokumentation (z. B. via Swagger UI des Backends) ersetzt.

```
openapi: 3.0.3
info:
  title: Sensor Configuration API
  version: 1.0.0
  description: API to retrieve sensor parameter calibration, formula (via JSONLogic),
    and boundary data.
paths:
  /sensor-parameters/config:
    get:
      summary: Retrieve sensor parameter metadata
      description: Fetches the JSONLogic formula, boundaries, and version information
        for a specific sensor parameter.
      operationId: getSensorParameterConfig
      parameters:
        - name: dasKey
          in: query
          required: true
          description: The composite DAS mapping key
            ("<DAS>__<das_sensor_alias>__<das_parameter_alias>"),
              i.e. the same value used as the Kafka message key on internal-sensor-
            config
              (see "DAS-Mapping: Inbound-Key vs. Persistenz-Referenz").
      schema:
        type: string
        example: "SOLEXPerts__BH12-P03__temperature"
      responses:
```

```

    '200':
      description: Successful response containing the sensor parameter
configuration.
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/SensorParameterConfig'
    '400':
      description: Bad Request - Missing or invalid dasKey.
    '404':
      description: Not Found - The specified dasKey does not exist.
components:
  schemas:
    SensorParameterConfig:
      type: object
      required:
        - das
        - das_sensor_alias
        - das_parameter_alias
        - sensor_parameter_id
        - formula
        - upper_bound
        - lower_bound
        - version
      properties:
        das:
          type: string
          description: The DAS (data acquisition system) enum value this parameter is
sourced from. Defined
            once in the shared "contracts" module and referenced by both core
(publisher) and pipeline
            (SensorConfigCache), so both sides agree on the same vocabulary.
          example: "SOLEXPerts"
        das_sensor_alias:
          type: string
          description: The device/sensor identifier used on the DAS
(sensor.das_sensor_alias). Forms the
            middle segment of the composite DAS mapping key.
          example: "BH12-P03"
        das_parameter_alias:
          type: string
          description: The field name used on the DAS for this specific measurement
(sensor_parameter.das_parameter_alias).
            Forms the suffix of the composite DAS mapping key.
          example: "temperature"
        sensor_parameter_id:
          type: string
          format: uuid
          description: The unique identifier (UUIDv7) of the sensor parameter. Sent
unchanged to the pipeline
            and used as the foreign key when persisting readings in TimeScaleDB,

```

decoupled from the
das/das_sensor_alias/das_parameter_alias mapping used only for inbound
routing - replacing the
previous brittle sensor_code-based mapping and allowing DAS-side remapping
without orphaning
historical readings.
example: "018f2e6a-7c3e-7c3e-8b3e-1a2b3c4d5e6f"
formula:
type: string
description: The dynamic mathematical expression used to standardize raw
readings into SI units.
Must use standard infix notation and include the variable 'x'
(representing the raw value).
default: "x"
example: "x * 1.8 + 32"
upper_bound:
type: number
format: double
description: The maximum limit for this sensor parameter used for alerting.
example: 150000.5
lower_bound:
type: number
format: double
description: The minimum limit for this sensor parameter used for alerting.
example: -100.0
version:
type: integer
format: int16
description: The version number of this specific configuration record.
example: 2

8.10. Teststrategie

Um die Stabilität und Datenintegrität von MONTEIS zu gewährleisten, verfolgen wir eine schichtenbasierte Teststrategie. Der Fokus liegt darauf, die gesamte Funktionalität und das fachliche Verhalten der Applikation rigoros abzusichern, ohne durch fragile Layout-Tests unnötigen Wartungsaufwand zu erzeugen.

Um eine konstant hohe Code-Qualität zu garantieren, wird systemübergreifend **SonarQube** eingesetzt. Eine Mindestabdeckung dient als Guardrail, wobei die Aussagekraft der Tests (fachliche Relevanz, Fehlererkennungsfähigkeit) höher gewichtet wird als die reine Prozentzahl.

8.10.1. Frontend Testing

Im Angular-Frontend liegt der Fokus auf der Überprüfung der isolierten UI-Logik (z. B. State-Management, Filter-Logik, Datenaufbereitung für Charts).

- **Technologie:** Wir verwenden das moderne, von Angular unterstützte **Vitest** für die Unit-Tests.
- **Fokus:** Es wird ausschliesslich Logik getestet. Reine Darstellungsaspekte (Layout, CSS, Pixel-

Perfektion) werden bewusst von Unit-Tests ausgeschlossen.

8.10.2. Backend Testing (Spring Boot)

Das Backend wird strikt in verschiedene Test-Ebenen unterteilt (Test-Pyramide), um schnelle Feedback-Zyklen und realitätsnahe Datenbanktests optimal zu kombinieren. Als Basis dient hierbei die native **Spring Boot Test Suite**.

- **Web-Schicht (Controller Tests):** Für die REST-Schnittstellen verwenden wir das Spring Boot Slice-Testing (`@WebMvcTest`) in Kombination mit `MockMvc`. Dadurch testen wir das korrekte HTTP-Routing, Input-Validierungen, HTTP-Statuscodes und die JSON-Serialisierung/-Deserialisierung. Die zugrundeliegende Business-Logik (Services) wird auf dieser Ebene via `@MockBean` gemockt, um den Application Context klein und performant zu halten.
- **Business-Logik (Unit Tests):** Die eigentliche Fachlogik (Services, Mapper und Configurations) wird isoliert über klassische Unit-Tests mit **JUnit 5** und **Mockito** getestet. Externe Systemgrenzen und die Datenbank (Persistenzschicht) werden auf dieser Ebene konsequent gemockt.
- **Persistenzschicht (Integration Tests):** Die Persistenzschicht (jOOQ-Repositories) wird von den normalen Unit-Tests ausgeschlossen. Stattdessen nutzen wir **Spring Boot Testcontainers**, um die echten Datenbanken als isolierte Docker-Container hochzufahren.
 - **Architektur-Fokus (FDW):** Um den reibungslosen Datenabruf zu testen, fahren die Testcontainers sowohl eine PostgreSQL- als auch eine TimeScaleDB-Instanz hoch. Im Rahmen des Test-Setups wird der FDW-Tunnel zwischen den Containern initialisiert.
 - **Testdaten-Management:** Das Schema und die initialen Testdaten für die Container werden pro Test-Suite automatisiert über spezifische **Flyway-Skripte** aufgebaut.

8.10.3. End-to-End Testing (E2E)

Um die isolierten Komponenten-Tests zu komplementieren und das Zusammenspiel des gesamten Systems (Frontend + Backend + Datenbank) zu validieren, setzen wir ebenenübergreifende E2E-Tests ein.

- **Technologie: Playwright**
- **Infrastruktur & Setup:** Um eine verlässliche und reproduzierbare Testumgebung für die E2E-Tests zu schaffen, implementieren wir ein dediziertes **Spring Boot Testing-Profil**. Startet man das Backend mit diesem Profil, fährt es automatisch die benötigten Testcontainers (PostgreSQL, TimeScaleDB und Kafka) hoch und befüllt sie via Flyway, wodurch auch der Sensor-Config-Publish-Flow (`internal-sensor-config`) End-to-End getestet werden kann. Das Frontend kann anschliessend gegen dieses vollständig gekapselte Backend getestet werden.
- **Fokus:** Getestet wird das Verhalten der Applikation anhand der wichtigsten User-Workflows (z. B. Sensor-Filterung, Sensor erstellen, Experiment erstellen, etc.). Auch hier steht das funktionale Verhalten im Vordergrund, nicht das Layout.

8.11. Entwickler-Workflow & CI/CD Pipeline

Um eine konsistente Code-Qualität zu garantieren, den Review-Prozess zu verschlanken und die

Server-Ressourcen effizient zu nutzen, wird der Code-Lebenszyklus von lokalen Git-Hooks bis hin zu automatisierten GitHub Actions strikt orchestriert.

8.11.1. Lokaler Workflow (Git Hooks)

Bevor Code in das Repository gepusht wird, greifen lokale Git-Hooks , um formelle Standards bereits auf dem Entwickler-Rechner zu erzwingen:

- **Formatter Pre-Commit Hook:** Dieser Hook formatiert geänderte Dateien automatisch und fügt sie dem Staging-Bereich (`git add`) wieder hinzu. Dadurch wird sichergestellt, dass der Commit exakt den sauber formatierten Stand enthält.
 - **Frontend (.ts / .html):** Formatierung und Linting via **Prettier** und **ESLint**.
 - **Backend (.java):** Formatierung via **Spotless** unter strikter Verwendung des **Google Java Formats**.
- **Commit-Message Hook (commit-msg):** Prüft die Commit-Nachricht vor dem Speichern auf das Einhalten der **Conventional Commits** (TBD).

8.11.2. Continuous Integration (Pull Request Workflows)

Um die Ressourcen (und Kosten) der Pipeline zu schonen, werden ressourcenintensive CI-Jobs **nicht bei jedem einzelnen Push**, sondern ausschliesslich bei der Erstellung oder Aktualisierung von **Pull Requests (PRs)** ausgeführt.

- **Format Checker:** Prüft den gesamten Codebestand, um sicherzustellen, dass keine unformatierten Dateien die lokalen Hooks umgangen haben (z. B. durch `git commit --no-verify`).
- **Full Test Suite:** Führt die oben definierte Teststrategie (Unit, Controller, Integration via Testcontainers, E2E) vollständig aus.
- **Security Scanning (Trivy):** Eine dedizierte GitHub Action nutzt **Trivy** für automatisierte Sicherheitsüberprüfungen. Dabei werden die Projekt-Abhängigkeiten, Dockerfiles und Container-Images auf bekannte Schwachstellen (CVEs) und Fehlkonfigurationen (IaC) gescannt. Gefundene kritische Sicherheitslücken (z. B. Severity "HIGH" oder "CRITICAL") blockieren den Pull Request.
- **SonarQube Quality Gate:** Eine dedizierte GitHub Action (`sonar-scan`) analysiert den Code auf Bugs, Vulnerabilities und Code Smells. Anschliessend prüft die Action den Status des SonarQube Quality Gates. Ein Pull Request kann nur bei grünem Quality Gate gemerged werden.

8.11.3. Continuous Deployment & Release-Workflows

Sobald Code-Änderungen in den `main-branch` integriert werden oder offizielle Versionen anstehen, übernehmen automatisierte GitHub Actions das Deployment. Das Setup folgt strikt dem Build-Once-Prinzip für Produktions-Releases und einem GitOps-Ansatz für die Infrastruktur.

Es sind drei dedizierte Workflows definiert:

- **Workflow 1: Auto-Deploy auf DEV (Bleeding Edge)**

- **Trigger:** Automatisch bei jedem Push oder Merge in den **main**-Branch.
- **Ablauf:** Der Workflow baut sofort und kompromisslos ein neues Docker-Image aus dem aktuellen Stand des Codes und deployt dieses auf die **DEV**-Umgebung. Diese Stage spiegelt den aktuellsten Entwicklungsstand wider und muss nicht zwingend stabil sein.
- **Workflow 2: Create Release (Deployment auf STAGING)**
 - **Trigger:** Manuell (GitHub Workflow Dispatch).
 - **Ablauf:**
 1. Der ausführende Nutzer wählt die Art des Releases (Major, Minor oder Patch basierend auf dem letzten existierenden Git-Tag).
 2. Der Workflow aktualisiert die Version im Code (z. B. via `mvn clean install -Drelease=vX.X.X` im Backend).
 3. Er erstellt den neuen Git-Tag sowie ein offizielles GitHub Release.
 4. Die Anwendung wird kompiliert und das neue Docker-Image wird gebaut.
 5. Das frisch gebaute Image wird gestartet und durchläuft sofort die komplette End-to-End (E2E) Test-Suite.
 6. Nur wenn die E2E-Tests erfolgreich sind, wird das Image in die Container Registry (Amazon ECR) gepusht und automatisch auf der **STAGING**-Umgebung deployt.
- **Workflow 3: Promote to PROD (GitOps)**
 - **Trigger:** Manuell (GitHub Workflow Dispatch).
 - **Ablauf:** Dieses Skript baut bewusst **keine** neuen Images, um ungetestete Seiteneffekte zu vermeiden.
 1. Der Nutzer wählt einen bereits existierenden Release-Tag aus dem Dropdown.
 2. Der Workflow sucht in der Amazon ECR nach dem exakt passenden, auf Staging bereits validierten Docker-Image.
 3. Der Workflow checkt das separate Produktions-Infrastruktur-Repository aus.
 4. Er aktualisiert dort lediglich den Image-Tag in der `kustomize.yaml` und committet die Änderung.
 5. Durch diesen Commit im Infra-Repo wird das eigentliche Kubernetes-Deployment auf **PROD** (GitOps-Ansatz) angestossen.

9. Architekturentscheidungen

Kriterium	Beschreibung
Titel	ADR: Einsatz von Anchore Syft und Grant für Open-Source-Lizenzkonformität
Datum	08.06.2026
Status	entschieden
Kontext & Problem	Vermeidung von riskanten Open-Source-Lizenzen (z.B. Copyleft wie GPL-3.0) in Node.js- und Java-Projekten. Benötigt wird eine automatisierte, tiefgehende Prüfung innerhalb der CI/CD-Pipeline vor dem Merge.
Entscheidung	Einsatz von Anchore Syft zur Generierung der Software Bill of Materials (SBOM) im SPDX-JSON-Format und Anchore Grant zur lokalen Prüfung gegen eine .grant.yml-Richtlinie.
Begründung	Performance durch Entkopplung: Syft führt tiefe Netzwerkscans durch (Ergebnis wird gecached), Grant prüft das lokale SBOM in Millisekunden. Tiefe Auflösung: Native Unterstützung für die Auflösung von Maven-Parent-POMs und externen npm-Registries. SaaS-frei & Sicher: Die Prüfung läuft komplett lokal im GitHub Actions Runner ab. Keine Übertragung von Code oder Daten an Drittanbieter. Configuration as Code: Richtlinien sind versioniert und auditierbar direkt im Repository gespeichert.
Konsequenzen	Schnelle CI-Pipelines und sofortiges Blockieren bei Lizenzverstößen. Erfordert als Trade-off die manuelle Pflege einer ignore-packages-Liste für fehlerhafte Lizenz-Strings oder interne CI-Tools.

Kriterium	Beschreibung
Titel	ADR: Durchsetzung der Leseberechtigungen (Row-Level Security) auf Datenbankebene statt in der Applikationsschicht
Datum	17.07.2026
Status	entschieden
Kontext & Problem	Ein früheres Konzept sah einen jOOQ-seitigen VisitListener / RecordListener vor, der Queries abfängt und Berechtigungen im Code prüft. Nachteile: jeder neue Query-Pfad müsste ihn korrekt durchlaufen, die per Foreign Data Wrapper eingebundene TimescaleDB-Tabelle unterstützt ohnehin keine native RLS, und Berechtigungslogik im Code ist schwerer zu auditieren als deklarative DB-Policies.

Kriterium	Beschreibung
Entscheidung	Leseberechtigungen werden über native PostgreSQL Row-Level-Security-Policies auf <code>sensors/experiments</code> sowie eine security-barrier-View (<code>sensor_reading_secured</code>) für die FDW-Tabelle durchgesetzt. Ein custom jOOQ <code>ConnectionProvider</code> (<code>RlsConnectionProvider</code>) schreibt dafür bei jeder Connection-Beschaffung die aktuelle <code>Authentication</code> transaktionslokal auf die Verbindung; die Policies lesen diesen Kontext serverseitig aus. Die <code>Authentication</code> selbst wird von einem custom <code>Converter</code> (<code>MonteisJwtAuthenticationConverter</code>) aus den Berechtigungen und Experiment-IDs im JWT gebaut (<code>MonteisPrincipal</code>) und liegt für die Dauer des Requests im Standard- <code>SecurityContextHolder</code> von Spring Security, ohne eigenen Propagierungsmechanismus.
Begründung	Enforcement in der Datenbank ist unabhängig vom Zugriffspfad (jOOQ, native SQL, künftige Tools) und lässt sich nicht durch einen vergessenen Code-Check umgehen. Der transaktionslokale Kontext verhindert ein Durchsickern zwischen Anfragen bei Connection-Pooling.
Konsequenzen	RLS-relevante Repository-Methoden müssen innerhalb einer Spring-Transaktion laufen (<code>@Transactional(readOnly = true)</code>), sonst wird der Kontext nie gesetzt bzw. gelesen. Schreiboperationen bleiben auf DB-Ebene permissiv, da alle Zugriffe dieselbe Rolle (<code>core_app</code>) nutzen — siehe folgende ADR zur Trennung von Read- und Write-Autorisierung.

Kriterium	Beschreibung
Titel	ADR: Trennung von Lese- und Schreibautorisierung — RLS für Reads, RBAC für Writes
Datum	17.07.2026
Status	offen
Kontext & Problem	Alle DB-Zugriffe laufen über dieselbe Rolle (<code>core_app</code>); DB-Grants können daher nicht zwischen einzelnen Nutzern unterscheiden — sie eignen sich für die RLS-Lesefilterung, nicht aber für nutzerspezifische Schreibrechte.
Entscheidung (vorläufig)	Lesen bleibt vollständig RLS-gefiltert (siehe vorherige ADR). Schreiben ist stattdessen klassisch rollenbasiert (RBAC) auf Applikationsebene geregelt: Im Spring-Security-Filterchain (<code>SecurityConfig</code>) erfordert jeder <code>POST/PUT/PATCH/DELETE</code> -Request global eine Write-Berechtigung, die nur Admins besitzen. <code>GET</code> -Requests bleiben für alle authentifizierten Nutzer offen (weiterhin RLS-gefiltert).

Kriterium	Beschreibung
Begründung	Schreiben braucht nur ein einfaches rollenbasiertes Ja/Nein, während Lesen feingranular pro Experiment gefiltert werden muss. Das trennt die Verantwortlichkeiten klar: Datenbank filtert Daten, Applikation autorisiert Aktionen. Eine einzige Regel auf HTTP-Methoden-Ebene genügt aktuell, da es keine differenzierten Schreibrollen gibt.
Konsequenzen	Schreibrechte sind aktuell global (jeder <code>admin</code> darf überall schreiben), nicht pro Rolle/Experiment granularisiert. Sollte künftig feingranularere Schreibrechte nötig werden, müsste dies pro Controller-Methode (z. B. via <code>@PreAuthorize</code>) statt zentral im Filterchain erfolgen.

Kriterium	Beschreibung
Titel	ADR: UUID v7 als einheitlicher Primärschlüsseltyp für die Metadaten-DB
Datum	26.08.2026
Status	entschieden
Kontext & Problem	Mit der Aufteilung von <code>sensors</code> in <code>sensors</code> (gemeinsame Stammdaten) und <code>sensor_parameters</code> (je Messgrösse eigene Formel/Einheit/Grenzwerte, siehe Sensor vs. Sensor_Parameter) wächst die Anzahl referenzierter Tabellen und Fremdschlüssel in der Metadaten-DB spürbar. Bisher generierte PostgreSQL fortlaufende <code>BIGSERIAL</code> -Werte pro Tabelle. Das bringt zwei Probleme: fortlaufende Integer-IDs sind über Tabellen hinweg nicht unterscheidbar (Verwechslungsgefahr beim Debugging/Tracing) und verraten Rückschlüsse auf das Datenvolumen. Zusätzlich muss <code>sensor_parameter.id</code> künftig als stabile, global eindeutige Kennung über Systemgrenzen hinweg kommuniziert werden — sie wird 1:1 im Kafka-Config-Objekt an die Pipeline durchgereicht und dort als Fremdschlüssel beim Schreiben der Messwerte in die TimeScaleDB verwendet, anstelle der bisherigen textuellen <code>sensor_code</code> -Abbildung. Pro-Tabelle-lokale Integer-Sequenzen sind für eine solche Cross-System-Referenz ungeeignet.
Entscheidung	Sämtliche Primärschlüssel (und die davon abgeleiteten Fremdschlüssel) der Metadaten-DB werden auf UUID Version 7 (<code>UUIDv7</code>) statt <code>BIGSERIAL</code> umgestellt. Einzige Ausnahme ist die von Flyway selbst verwaltete <code>flyway_schema_history</code> -Tabelle, die ausserhalb unserer Kontrolle liegt.

Kriterium	Beschreibung
Begründung	Global eindeutig über alle Tabellen hinweg: Erleichtert Logging, Tracing und Debugging, da eine ID nie mit einer ID aus einer anderen Tabelle verwechselt werden kann. Zeitlich sortierbar: Im Gegensatz zu UUIDv4 kodiert UUIDv7 einen Millisekunden-Zeitstempel im höchstwertigen Teil. Dadurch bleiben B-Tree-Indizes auf dem Primärschlüssel weiterhin insertions-lokal und performant, ohne die Index-Fragmentierung, die rein zufällige UUIDv4-Werte verursachen würden. Keine Rückschlüsse auf Geschäftsvolumen: Fortlaufende Integer-IDs verraten z. B. wie viele Sensoren oder Experimente insgesamt existieren; für eine Anwendung mit potenziell externen Integrationen (Fulcrum) ist das unerwünscht. Stabile, kollisionsfreie Cross-System-Referenz: <code>sensor_parameter.id</code> muss ohne Rückfrage bei der Metadaten-DB eindeutig erzeugt und über Kafka an die Pipeline sowie fachlich in die TimeScaleDB weitergereicht werden können — eine Eigenschaft, die eine global (nicht nur pro Tabelle) eindeutige ID voraussetzt.
Konsequenzen	Storage-Overhead von 16 statt 8 Bytes pro ID; bei den erwarteten Tabellengrößen der Metadaten-DB (keine Zeitreihen) vernachlässigbar. Die Anwendung muss eine UUIDv7-fähige Generierungsbibliothek einbinden. Die bestehenden BIGSERIAL-basierten Migrationen (V1–V7) müssen im Rahmen der Sensor/Sensor_Parameter-Aufteilung entsprechend migriert werden. Fremdschlüssel-Spalten und ihre Indizes wachsen von 8 auf 16 Bytes, ohne relevante Performance-Auswirkung beim gegebenen Datenvolumen.

Kriterium	Beschreibung
Titel	ADR: Entkopplung von DAS-Ingest-Mapping (zusammengesetzter Key) und persistenter <code>sensor_parameter_id</code> -Referenz
Datum	26.08.2026
Status	entschieden

Kriterium	Beschreibung
Kontext & Problem	Eintreffende Rohmessungen tragen ausschliesslich Identifikatoren des jeweiligen Erfassungssystems (DAS) — z. B. einen Gerätenamen (<code>deviceName</code>) und einen Feldnamen innerhalb der Payload. Die Pipeline kennt beim Empfang einer Rohnachricht keine <code>sensor_parameter.id</code>; ein synchroner Rückruf an die Core API pro Nachricht wäre bei den angestrebten Datenraten (High-Throughput, siehe Kapitel 8, Skalierungsstrategien (High Throughput)) nicht praktikabel. Gleichzeitig sind DAS-seitige Identifikatoren volatil: Hersteller benennen Geräte um, ein Sensor kann auf ein anderes Erfassungssystem migriert werden, oder ein <code>das_sensor_alias</code> / <code>das_parameter_alias</code> wird in der Core API korrigiert. Würde die TimeScaleDB direkt mit diesen volatilen Strings referenzieren (wie bisher mit <code>sensor_code</code>), würden historische Messwerte bei einer Remapping-Änderung faktisch verwaist, da neue Werte unter einem neuen Schlüssel abgelegt würden.
Entscheidung	Es werden zwei getrennte Schlüssel für zwei getrennte Zwecke verwendet: Für das Ingest-Routing (Kafka-Key auf allen Sensordaten-Topics inkl. <code>internal-sensor-config</code>, sowie Lookup-Key im <code>SensorConfigCache</code>) dient ein deterministisch aus den DAS-Rohdaten zusammengesetzter String <code><DAS></code> <code><das_sensor_alias><das_parameter_alias></code> (siehe Kapitel 8, DAS-Mapping: Inbound-Key vs. Persistenz-Referenz). Für die Persistenz in der TimeScaleDB (<code>sensor_reading.sensor_parameter_id</code>) sowie für Idempotenz-Prüfungen (<code>ON CONFLICT</code>) wird ausschliesslich die stabile, unveränderliche <code>sensor_parameter.id</code> (UUIDv7, siehe vorherige ADR) verwendet. Der DAS-Key wird der Pipeline im Config-Objekt zusätzlich zur <code>sensor_parameter_id</code> mitgegeben (nicht daraus abgeleitet), sodass beide Werte unabhängig voneinander aktualisierbar bleiben.

Kriterium	Beschreibung
Begründung	Kein Rückruf pro Nachricht nötig: Der DAS-Key ist direkt aus der Rohnachricht ableitbar, ohne Kenntnis der internen <code>sensor_parameter.id</code> — das Cache-Lookup bleibt lokal und schnell. Historische Daten bleiben stabil referenziert: Ändert sich ein <code>das_sensor_alias/das_parameter_alias</code> (Remapping), aktualisiert Core nur die Zuordnung und publiziert einen neuen DAS-Key für dieselbe, unveränderte <code>sensor_parameter.id</code>. Bereits geschriebene Messwerte in der TimeScaleDB bleiben unverändert über die alte wie neue Zuordnung hinweg korrekt referenziert — es entstehen keine verwaisten Zeitreihen und kein Backfill/Merge ist nötig. Bessere Joins/Performance: Ein UUID-Fremdschlüssel ist kompakter und indexfreundlicher als ein zusammengesetzter, potenziell langer Text-Key, insbesondere für Joins zwischen TimeScaleDB (über den FDW-Tunnel) und der Metadaten-DB. Reaktionsfähig auf DAS-Wechsel: Da der DAS-Key nicht Teil der Persistenz ist, kann ein Sensor bei Bedarf komplett auf ein anderes Erfassungssystem umgezogen werden (neuer <code>das</code>-Wert, neuer <code>das_sensor_alias</code>), ohne die Historie zu fragmentieren.
Konsequenzen	Das Config-Objekt (<code>SensorParameterConfig</code>) transportiert zwangsläufig mehr Felder (<code>das</code>, <code>das_sensor_alias</code>, <code>das_parameter_alias</code>, <code>sensor_parameter_id</code>) statt eines einzelnen Identifiers. Der <code>Das</code>-Enum muss zentral im geteilten Modul <code>contracts</code> gepflegt werden, damit Core (Publisher) und Pipeline (Normalizer, <code>SensorConfigCache</code>) nicht auseinanderlaufen. Reprocessing-Jobs, die auf einen DAS-Key-Wechsel reagieren (z. B. rückwirkendes Neu-Labeling), müssen weiterhin über die <code>sensor_parameter_id</code> und nicht über den (potenziell mehrfach wechselnden) DAS-Key gesteuert werden.

10. Qualitätsanforderungen

10.1. Übersicht der Qualitätsanforderungen

10.2. Qualitätsszenarien

11. Risiken und technische Schulden

12. Glossar

Begriff	Definition
<i>jOOQ</i>	<i>jOOQ generiert Java-Code auf Basis der Datenbank und ermöglicht es, über seine Fluent-API typsichere SQL-Abfragen zu erstellen.</i>
<i>JaVers</i>	<i>JaVers ist der Industriestandard für Datenversionierung und Objektvergleiche in modernen Java-Projekten.</i>
<i>Scion-Workbench</i>	<i>Die Scion-Workbench ist ein Open Source verfügbares Layout Framework für nicht statische Angular Webapps.</i>
<i>AG-Grid</i>	<i>AG-Grid ist eine Library für Angular, speziell geeignet für anspruchsvolle filterbare Tabellen.</i>
<i>Syft</i>	<i>CLI Tool zur Generierung der Software Bill of Materials (SBOM) im SPDX-JSON-Format.</i>
<i>Grant</i>	<i>CLI Tool zur Prüfung der eingesetzten Lizenzen aller Dependencies gegen eine .grant.yml-Richtlinie.</i>